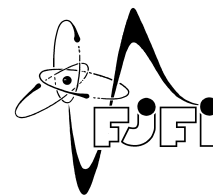




ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE
Fakulta jaderná a fyzikálně inženýrská



Transfromery pro Bayesovskou statistiku

Transfromers for Bayesian statistics

Výzkumný úkol

Autor: **Ruslan Guliev**
Vedoucí práce: **prof. Ing. Václav Šmídl, PhD.**
Akademický rok: 2025/2026

Poděkování:

Chtěl bych zde poděkovat především svému školiteli profesoru Václavu Šmídlovi za pečlivost, ochotu, vstřícnost a odborné i lidské zázemí při vedení mé práce. Dále děkuji všem ostatním, kdo tuto práci obohatili o své cenné rady.

Název práce:

Transfromery pro Bayesovskou statistiku

Autor: Ruslan Guliev

Studijní program: Aplikované matematicko–stochastické metody

Druh práce: Výzkumný úkol

Vedoucí práce: prof. Ing. Václav Šmídl, PhD., ÚTIA Av ČR

Abstrakt: Výpočet aposteriorních rozdělání je jádrem Bayesovské inference a existuje pro něj řada metod, například Markov chain Monte Carlo. Tyto metody však bývají výpočetně náročné a pro složitější úlohy prakticky nepoužitelné. Relativně nová architektura Prior-Data Fitted Networks (PFN) tento problém obchází jednoduše, transformer natrénovaný na syntetických datech z prioru vrací celé posteriorní prediktivní rozdělání v jediném dopředném průchodu. Metoda má navíc teoretické zázemí v matematické statistice. Cílem práce je architekturu podrobně popsat, replikovat její výsledky na úloze regrese Gaussovských procesů a empiricky ověřit teoretické hypotézy o jejím chování. V experimentech měříme, jak dobrá je aproximace přesného posterioru. Zkoumáme také přerozdělování práce mezi vrstvami transformera a robustnost metody při špatně zvoleném prioru. Závěrem metodu přeneseme na segmentaci obrazu a ověříme, že se její dobré vlastnosti zachovávají i tam.

Klíčová slova: amortizovaná inference, Bayesovská inference, Gaussovské procesy, Prior-Data Fitted Networks, segmentace obrazu, transformer

Title:

Transformers for Bayesian statistics

Author: Ruslan Guliev

Abstract: The computation of posterior distributions lies at the heart of Bayesian inference, and a number of methods exist for it, such as Markov chain Monte Carlo. These methods, however, tend to be computationally expensive and impractical for more complex tasks. The relatively new Prior-Data Fitted Networks (PFN) architecture sidesteps this problem in a simple way, a transformer trained on synthetic data sampled from the prior returns the entire posterior predictive distribution in a single forward pass. Moreover, the method is backed by results from mathematical statistics. The aim of this thesis is to describe the architecture in detail, replicate its results on Gaussian process regression, and empirically verify the theoretical hypotheses about its behaviour. In our experiments, we measure how good the approximation of the exact posterior is. We also study how the transformer layers redistribute the work among themselves and how robust the method is under a misspecified prior. Finally, we transfer the method to image segmentation and verify that its good properties carry over there as well.

Key words: amortized inference, Bayesian inference, Gaussian processes, image segmentation, Prior-Data Fitted Networks, transformer

Obsah

Úvod	10
1 Problematika výběru modelů	11
1.1 Obecný princip hledání modelu	11
1.2 Bayesův přístup k výběru modelu	12
1.3 GP regrese	13
1.4 Výběr modelu pro GP regresi	14
1.4.1 Variační inference	15
1.4.2 Type II – MLE	15
1.4.3 Markov Chain Monte Carlo	16
2 Prior–Data Fitted Networks	18
2.1 Úvod do Prior–Data Fitted Networks	18
2.2 Praktická motivace k PFN	20
2.3 Vnitřní architektura PFN	20
2.3.1 Fáze 1 – Meta–Learning	21
2.3.2 Fáze 2 – Vstup do modelu	22
2.3.3 Fáze 3 – Transformer vrstvy	24
2.3.4 Fáze 4 – Decoder, převod embeddingu na logity.	26
2.4 Teoretické statistické vlastnosti PFN	28
2.4.1 Statistický model	28
2.4.2 Od PPD k PFN	30
2.4.3 Učení PFN In–Context	31
2.4.4 PFN transformer	33
3 Experimentální průzkum vlastností PFN	36
3.1 Vnitřní struktura PFN transformeru a jeho základní principy	36
3.1.1 Struktura attention matic	37
3.1.2 Matematický rámec pro srovnávací experimenty	39
3.1.3 Srovnání s RBF kernelem a Nadaraya–Watsonovým estimátorem	40
3.1.4 Přesnost predikcí na skutečné GP realizaci	42
3.1.5 Konvergence střední hodnoty a kalibrace rozptylu	44
3.1.6 Shrnutí	45
3.2 PFN nad rodinou Gaussovských procesů: náhodné hyperparametry	46
3.2.1 Trénink nad rozdělením hyperparametrů	46
3.2.2 Struktura attention matic	47
3.2.3 <i>Attention</i> vs RBF kernel	48

3.2.4	Kalibrace rozptylu	49
3.2.5	Přesnost napříč korelačními délkami	50
3.2.6	Identifikace korelační délky z kontextu	52
3.2.7	Predikce na skutečné realizaci a role kontextu	53
3.2.8	Shrnutí	55
3.3	Specializace vrstev: čtení labelů a kernelové porovnávání	55
3.3.1	Korelační analýza attention napříč vrstvami	56
3.3.2	Které hlavy model skutečně potřebuje: head-knockout	60
3.3.3	Rozklad chyby a role hloubky	62
3.3.4	Shrnutí	63
3.4	Hloubka jako iterace řešiče: PFN versus gradientní sestup	64
3.4.1	Hloubka snižuje chybu, ale ne jako obyčejná Neumannova řada	65
3.4.2	Srovnání s gradientním sestupem v predikčním prostoru	66
3.4.3	Shrnutí	68
3.5	Robustnost při misspecifikaci prioru	68
3.5.1	Vliv velikosti kontextu	70
3.5.2	Vliv síly misspecifikace	72
3.5.3	Chování mimo trénovací rozsah korelační délky	73
3.5.4	Shrnutí	75
4	PFN pro segmentaci obrazu	76
4.1	Od jednorozměrné regrese k segmentaci obrazu	76
4.2	Kolaps k prioru u předtrénovaného modelu	78
4.2.1	Uspořádání experimentu	78
4.2.2	Kolaps k prioru	79
4.2.3	Výkon a saturace kontextem	80
4.2.4	Kalibrace	80
4.2.5	Shrnutí	82
4.3	Srovnání vlastního PFN s přesným posteriorem	82
4.3.1	Uspořádání experimentu	82
4.3.2	Shoda predikce s oraclem	83
4.3.3	Kalibrace	85
4.3.4	Rozklad chyby na bias a rozptyl	86
4.3.5	Vzdálenost od Bayesova optima	87
4.3.6	Kolaps k prioru nenastává	88
4.3.7	Shrnutí	89
	Závěr	91
	Literatura	91

Úvod

O Bayesovských metodách ve statistice a strojovém učení se často říká, že za nimi stojí budoucnost. Vskutku, výsledky této teorie jsou elegantní a užitečné a ve srovnání s frekventistickým přístupem bývají často jednodušší i přirozenější. Místo jediného bodového odhadu vracejí celé rozdělení, a s ním i poctivou informaci o tom, jak moc si můžeme být predikcí jisti. Řada těchto výsledků však zůstává krásnou teorií, která naráží na tvrdý strop, a tím je výpočetní náročnost. Přesnou inferenci umíme provést jen ve výjimečných případech a aproximační metody bývají pomalé nebo kladou na úlohu další požadavky.

Spolu s rychlým vývojem v oblasti strojového učení se postupně daří tato omezení překonávat. Jednou z metod, které tímto směrem míří, je architektura Prior-Data Fitted Networks (PFN), kterou publikovali Müller a Hollmann [2]. Její myšlenka je pozoruhodně přímá. Místo aproximace inference pro konkrétní data se transformer natrénuje na velkém množství syntetických úloh vzorkovaných z prioru a naučí se vracet celé posteriorní prediktivní rozdělení v jediném dopředném průchodu. Otázka, kterou si tato práce klade, zní: co se takový model doopravdy naučil a jak věrně Bayesovskou inferenci napodobuje?

Práce je rozdělena do čtyř kapitol. V první kapitole vybudujeme klasický základ, o který se bude opírat zbytek práce. Zavedeme Bayesovský výběr modelu a ukážeme, jak se přirozeně rozpadá do hierarchie úrovní. Jako hlavní nástroj regrese představíme Gaussovské procesy [4], u kterých lze posteriorní rozdělení spočítat přesně. Následně popíšeme tři standardní přístupy k aproximaci posteriorních rozdělení, variační inferenci, Type II MLE a metody MCMC. Právě s těmito metodami budeme PFN později porovnávat.

Ve druhé kapitole představíme samotné PFN. Nejprve vysvětlíme jeho hlavní myšlenku a srovnáním s metodami z první kapitoly ukážeme, v čem se od nich liší. Poté podrobně popíšeme architekturu i trénink a na příkladu Gaussovských procesů vysvětlíme, jak jeden dopředný průchod nahrazuje celou inferenci. Druhou polovinu kapitoly věnujeme statistické teorii, kterou pro PFN vybudoval Nagler [3]. Z ní plynou klíčové vlastnosti, které budeme v dalších kapitolách opakovaně testovat.

Třetí kapitola je experimentálním jádrem práce. Na sérii modelů natrénovaných na datech z Gaussovských procesů nejprve prozkoumáme vnitřní strukturu transformeru. Ověříme, že neprovádí pouhé průměrování hodnot podle podobnosti vstupů. Dále ukážeme, že model trénovaný na celé rodině hyperparametrů dokáže korelační délku identifikovat přímo z kontextu. Následně popíšeme přerozdělování práce mezi vrstvami a otestujeme hypotézu, že hloubka modelu odpovídá iteracím řešiče. Nakonec porovnáme PFN s klasickými metodami z první kapitoly v situaci, kdy mají všechny metody špatně zvolený prior.

Ve čtvrté kapitole přeneseme stejné otázky do dvou dimenzí, na segmentaci obrazu. Nejprve na předtrénovaném *in-context* segmenteru ověříme, zda se při malém kontextu objeví kolaps k prioru. Poté natrénujeme vlastní PFN na úloze s přesně spočitatelným posteriorem a změříme, jak moc dobře ho aproximuje, kde vzniká bias a rozptyl a jak se chová mimo trénovací rozdělení. Tím ověříme, zda Naglerovy předpovědi platí i mimo jednorozměrnou regresi.

Kapitola 1

Problematika výběru modelů

V této kapitole uvedeme čtenáře do problematiky výběru modelu. Ukážeme si zde jaké různé přístupy výběru modelu při modelování dat existují, jaké potíže se s tím vážou a lze-li tyto potíže elegantně vyřešit. V praxi se často setkáváme se situacemi, kdy musíme pro daná data zvolit nejvhodnější model z několika kandidátů. Například při analýze časových řad můžeme mít několik různých modelů, které by mohly popsat pozorovaná data, a potřebujeme rozhodnout, který konkrétní model z funkčního prostoru je nejvhodnější. Dnes existuje celá řada metod, jak tento konkrétní model určit, jak najít vhodné parametry modelu, ovšem všechny tyto metody jsou zpravidla dost výpočetně náročné. Avšak se to změnilo s příchodem prudkého rozvoje neuronových sítí a hlubokého učení. O neuronových sítích víme, že to je dobrý aproximační nástroj a ukázalo se totiž, že abychom mohli co nejefektivněji aproximovat hledaný model pro naše data, lze použít architekturu transformer, která, jak se postupem času dozvíme, je nejpreciznějším a nejrychlejším řešením tohoto problému. Zavedeme nejdříve obecnou problematiku výběru modelu, poté Bayesovský princip výběru modelu a nakonec si ukážeme, jak nám v tom může pomoci architektura transformer.

1.1 Obecný princip hledání modelu

V této sekci si ukážeme obecný princip výběru modelu. Představme si následující situaci: máme k dispozici naměřená data, např. z fyzikálního experimentu nebo ze sociální studie a chceme najít model, který tato data co nejlépe popisuje. Tento model může být například lineární nebo nelineární regrese, rozhodovací strom nebo neuronová síť. Dalším účelem modelu je schopnost generalizace na nová, neviděná data, tj. možnost predikce nebo možnost otestovat pravdivost modelu. V tomto okamžiku před námi stojí několik otázek:

- Jakou strukturu modelu musíme zvolit (např. lineární vs. polynomiální)
- Jak správně zvolit hodnoty parametrů a hyperparametrů modelu (např. síla regularizace nebo počet vrstev v neuronové síti)
- Jak moc složitý model zvolit, aby dobře fitoval trénovací data, ale zároveň dobře generalizoval na nová data

Nakonec z toho vyplývá i formulace daného problému: proces výběru modelu je proces systematického výběru optimálního modelu. Během tohoto procesu vzniká i základní *trade-off*, tedy kompromis mezi příliš jednoduchým a příliš složitým modelem. Zvolíme-li příliš jednoduchý model, nebude schopen zachytit strukturu dat a bude mít špatný fit i špatnou generalizaci (tzv. *underfitting*). Naopak zvolíme-li

příliš složitý model, bude perfektně fitovat (interpolovat) trénovací data, ale bude špatně generalizovat na nová data (tzv. *overfitting*). Cílem je najít optimální model z těchto hledisek.

Dnes existuje celá řada přístupů k výběru modelu. Jsou to tři hlavní přístupy: Bayesovský přístup (*marginal likelihood*), *cross-validation* a informační kritéria (AIC, BIC). *Cross-validation* je praktický a intuitivní přístup, který odhaduje generalizační chybu pomocí validačních dat. Jinými slovy, data rozdělíme na několik částí, pro každou část trénujeme model na ostatních datech a testujeme na ní. Nakonec průměrujeme chyby přes všechny části a vybereme model s nejnižší tzv. *cross-validační* chybou [4]. Dalším velmi populárním přístupem je Bayesovský přístup, který spočívá v tom, že spočítáme pravděpodobnost modelu danou daty. To za předpokladu stejných apriorních pravděpodobností modelů znamená, že spočítáme tzv. *marginal likelihood*, což je pravděpodobnost dat daná modelem, integrovaná přes všechny možné hodnoty parametrů modelu. Tento přístup má výhodu v tom, že automaticky penalizuje složité modely a nabízí principiální způsob, jak vyvážit *fit* a složitost modelu. Posledním přístupem jsou informační kritéria, jako je Akaike Information Criterion (AIC) a Bayesian Information Criterion (BIC) [1]. Obě kritéria penalizují složitost modelu, každé však míří jinam. BIC je rychlou a jednoduchou aproximací *marginal likelihood*, zatímco AIC cílí na predikční chybu na nových datech, a *marginal likelihood* tedy neaproximuje. Nás však bude v této práci nejvíce zajímat Bayesovský přístup, protože nám poskytuje nejvíce principů pro výběr modelu a je základem pro náš další postup.

1.2 Bayesův přístup k výběru modelu

Zde ukážeme Bayesův princip při výběru modelu. Je zvykem použít hierarchický princip při zavádění Bayesovského výběru modelu [4].

Označme tedy $\mathbf{w} \in \mathcal{W}$ jako parametry modelu, kde $\mathcal{W}$ je prostor parametrů modelu, mohou to být např. váhy v lineární regresi nebo v neuronové síti. Dále nechť $\boldsymbol{\theta} \in \Theta$ jsou hyperparametry modelu, kde Θ je prostor hyperparametrů modelu, např. počet vrstev v neuronové síti. Nakonec označme $\mathcal{H}_i \in \mathcal{H}$ jako různé struktury modelu, kde $\mathcal{H}$ je prostor modelů, např. různé architektury neuronových sítí. V Bayesovském přístupu k výběru modelu používáme tzv. Bayesovskou inferenci k odhadu pravděpodobnostního rozdělení parametrů, hyperparametrů a modelů na základě pozorovaných dat $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, kde $\mathbf{x}_i$ jsou vstupní data a y_i jsou odpovídající cílové hodnoty, anebo alternativně jej lze zapsat jako $\mathcal{D} = (\mathbf{X}, \mathbf{y})$. Tj. cílem je na základě pozorovaných dat dojít k závěru, který model s kterými parametry a hyperparametry nejlépe vysvětluje pozorovaná data.

Na první úrovni hierarchie odhadneme pravděpodobnostní rozdělení parametrů modelu $\mathbf{w}$ pro dané (fixní) hyperparametry $\boldsymbol{\theta}$ a model $\mathcal{H}_i$. Zde používáme pojem posteriorního rozdělení, tj. ptáme se, jaké je pravděpodobnostní rozdělení parametrů modelu $\mathbf{w}$ vzhledem k pozorovaným datům $\mathbf{X}$, fixním hyperparametrům $\boldsymbol{\theta}$ a modelu $\mathcal{H}_i$. Předpokládejme, že chceme získat predikci pro výstupní data $\mathbf{y}$. Potom podle Bayesovy věty platí:

$$p(\mathbf{w}|\mathbf{y}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i) = \frac{p(\mathbf{y}|\mathbf{w}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)p(\mathbf{w}|\boldsymbol{\theta}, \mathcal{H}_i)}{p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)}, \quad (1.1)$$

kde $p(\mathbf{y}|\mathbf{w}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)$ je pravděpodobnost výstupních dat $\mathbf{y}$ podmíněná vstupními daty $\mathbf{X}$, parametry modelu $\mathbf{w}$, hyperparametry $\boldsymbol{\theta}$ a modelem $\mathcal{H}_i$ (tzv. *likelihood*), $p(\mathbf{w}|\boldsymbol{\theta}, \mathcal{H}_i)$ je apriorní rozdělení parametrů modelu $\mathbf{w}$ vzhledem k hyperparametrům $\boldsymbol{\theta}$ a modelu $\mathcal{H}_i$, a $p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)$ je tzv. *evidence* dat.

Na druhé úrovni hierarchie odhadneme pravděpodobnostní rozdělení hyperparametrů modelu $\boldsymbol{\theta}$ pro daný (fixní) model $\mathcal{H}_i$. Podobně jako na první úrovni používáme Bayesovu větu:

$$p(\boldsymbol{\theta}|\mathbf{y}, \mathbf{X}, \mathcal{H}_i) = \frac{p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)p(\boldsymbol{\theta}|\mathcal{H}_i)}{p(\mathbf{y}|\mathbf{X}, \mathcal{H}_i)}, \quad (1.2)$$

kde $p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)$ je pravděpodobnost výstupních dat $\mathbf{y}$ podmíněná vstupními daty $\mathbf{X}$, hyperparametry $\boldsymbol{\theta}$ a modelem $\mathcal{H}_i$ (tzv. *likelihood*), $p(\boldsymbol{\theta}|\mathcal{H}_i)$ je apriorní rozdělení hyperparametrů modelu $\boldsymbol{\theta}$ vzhledem k modelu $\mathcal{H}_i$, a $p(\mathbf{y}|\mathbf{X}, \mathcal{H}_i)$ je *evidence* dat.

Na třetí úrovni hierarchie odhadneme pravděpodobnostní rozdělení modelů $\mathcal{H}_i$. Opět používáme Bayesovu větu:

$$P(\mathcal{H}_i|\mathbf{y}, \mathbf{X}) = \frac{p(\mathbf{y}|\mathbf{X}, \mathcal{H}_i)P(\mathcal{H}_i)}{p(\mathbf{y}|\mathbf{X})}, \quad (1.3)$$

kde $p(\mathbf{y}|\mathbf{X}, \mathcal{H}_i)$ je pravděpodobnost výstupních dat $\mathbf{y}$ podmíněná vstupními daty $\mathbf{X}$ a modelem $\mathcal{H}_i$ (tzv. *marginal likelihood*), $P(\mathcal{H}_i)$ je apriorní rozdělení modelů $\mathcal{H}_i$, a $p(\mathbf{y}|\mathbf{X})$ je *evidence* dat.

Celý tento postup nám umožňuje systematicky vyhodnotit různé modely, jejich hyperparametry a parametry na základě pozorovaných dat a vybrat ten, který nejlépe vysvětluje data podle Bayesovského principu. Zde je dokonce vidět, proč jsme použili hierarchický přístup. Na každé úrovni hierarchie odhadujeme pravděpodobnostní rozdělení na základě předchozí úrovně, což nám umožňuje efektivně zachytit nejistotu na různých úrovních modelování. Postupujeme tedy od nejvyšší úrovně nejistoty, což je vůbec výběr modelu, přes střední úroveň, což je výběr hyperparametrů, až k nejnižší úrovni, což je odhad parametrů modelu, tj. náš problém se na každé úrovni zúžuje.

Avšak v praxi při výpočtech hustot pravděpodobností na jednotlivých úrovních hierarchie můžeme narazit na problém, že tento integrál neumíme spočítat analyticky, např. protože tato hustota není normalizovaná. Takovým integrálem je např. normalizační konstanta na první úrovni hierarchie:

$$p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i) = \int_{\mathcal{W}} p(\mathbf{y}|\mathbf{w}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)p(\mathbf{w}|\boldsymbol{\theta}, \mathcal{H}_i)d\mathbf{w}, \quad (1.4)$$

anebo na druhé úrovni hierarchie:

$$p(\mathbf{y}|\mathbf{X}, \mathcal{H}_i) = \int_{\Theta} p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)p(\boldsymbol{\theta}|\mathcal{H}_i)d\boldsymbol{\theta} \quad (1.5)$$

Tyto integrály je nutné aproximovat numericky pomocí takových metod jako Markov chain Monte Carlo (MCMC) nebo pokročilejším No-U-Turn Sampler (NUTS), pokud chceme odhadnout integrál z nenormalizovaných hustot pravděpodobnosti, jako je integrál (1.5), nebo např. Type II maximum likelihood (Type II – MLE), chceme-li odhadnout pouze bodovou hodnotu hyperparametrů modelu, což spočívá v maximalizaci marginal likelihood $p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)$ přes $\boldsymbol{\theta}$. Všechny tyto metody jsou však dost výpočetně náročné a může trvat značné množství času, než se podaří najít vhodný model pro daná data. Jedním z účelů této práce je ukázat, jak lze tyto pravděpodobnostní hustoty efektivně aproximovat pomocí nového postupu, a to pomocí architektury transformer.

1.3 GP regrese

Zde si ukážeme, jak aplikovat Bayesovský přístup výběru modelu na konkrétním příkladě, což bude prokládání dat pro nelineární regresi pomocí Gaussovských procesů (dále jen GP). Dále si ukážeme i základní metody pro hledání vhodného modelu, o kterých jsme se zmínili v předchozí sekci, a to jsou Type II – MLE a MCMC. Avšak předtím zdefinujeme základní pojmy GP regrese.

Představme si tedy, že máme před sebou následující problém. Někdo nám donesl sadu dat, mohou to být biomedicínská data, data z ekonomických trhů nebo dokonce i data z fyzikálního experimentu a chce po nás sestavit model, který by popisoval tato data a pomocí kterého bychom mohli také stanovit závěry nebo predikce pro budoucí výsledky na základě aktuálních. Je zcela přirozené, že chceme najít co nejpřesnější model, který by tato data popisoval. Jedním z přístupů, jak toho dosáhnout, je použití Gaussovských procesů (GP) pro regresi. Nejdříve ve zkratce zavedeme pojem GP regresi.

Zvykem je definovat Gaussovský proces pomocí vícerozměrného normálního rozdělení. Mějme tedy vícerozměrnou náhodnou veličinu $\mathbf{X} = (X_1, \dots, X_n)$. Říkáme, že $\mathbf{X}$ má vícerozměrné normální rozdělení s vektorem středních hodnot $\boldsymbol{\mu} = (\mu_1, \dots, \mu_n)$ a kovarianční maticí Σ , pokud její sdružená hustota pravděpodobnosti je dána vztahem:

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})\right), \quad (1.6)$$

kde $|\Sigma|$ je determinant matice Σ , vektor $\mathbf{x}$ je vektor realizací, značíme $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$.

Z toho vztahu plyne i definice Gaussovského procesu [4]:

Definice 1.1 (Gaussovský proces). Gaussovský proces je soubor náhodných veličin, libovolný konečný počet kterých má sdružené normální rozdělení.

Dalším klíčovým pojmem je tzv. kovarianční funkce (nebo jádro), která určuje vlastnosti GP. Ta se definuje jako funkce $k : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$, která pro každé dva vstupy $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^d$ vrací hodnotu $k(\mathbf{x}, \mathbf{x}')$, která udává míru podobnosti mezi těmito dvěma vstupy. Je to jistá modifikace kovarianční matice z klasického vícerozměrného normálního rozdělení, která vznikne po aplikaci tzv. *kernel triku*. Máme-li nějaký proces $f(\mathbf{x})$ a chceme ho zkoumat i pro jiný vstup $\mathbf{x}'$, ptáme se, jaká je kovariance mezi $f(\mathbf{x})$ a $f(\mathbf{x}')$. Tuto kovarianci lze odvodit v prostoru vah pomocí tzv. *feature mapy*, odvození ale vynecháme a odkážeme čtenáře na knihu [4]. Pro naše účely postačí, že kovarianční funkci definujeme přímo pomocí středních hodnot.

Gaussovský proces je totiž plně definován svou střední funkcí $m(\mathbf{x})$ a kovarianční funkcí $k(\mathbf{x}, \mathbf{x}')$. Střední funkci a kovarianční funkci tohoto procesu definujeme jako:

$$m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})], \quad k(\mathbf{x}, \mathbf{x}') = \mathbb{E}[(f(\mathbf{x}) - m(\mathbf{x}))(f(\mathbf{x}') - m(\mathbf{x}'))], \quad (1.7)$$

a následně říkáme, že proces $f(\mathbf{x})$ je Gaussovský proces a značíme $f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$.

Platí dále, že kovarianční funkce, které se také říká kernel (jádro), může mít různé formy. Mezi nejčastěji používané patří např. Radial Basis Function (RBF) kernel (někdy squared exponential kernel), který je definován jako:

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right), \quad (1.8)$$

kde σ_f^2 je rozptyl funkce a ℓ je tzv. korelační délka (*lengthscale*). Pak existuje řada dalších kernelů, jako je Matérn kernel, Periodic kernel nebo Linear kernel, každý s různými vlastnostmi a aplikacemi, ale pro naše účely postačí základní RBF kernel. RBF kernel má řadu výhod: je hladký, nekonečně diferencovatelný a má tendenci dobře aproximovat širokou škálu funkcí. Současně platí, že každý datový bod můžeme představit jako střed Gaussovského rozdělení. Odtud plyne i velmi intuitivní představa o tom, jak funguje GP regrese: každý datový bod přispívá k predikci nového bodu podle své vzdálenosti od něj, přičemž váha tohoto přispění je určena kovarianční funkcí (jádreem). Tím pádem, čím blíže je nějaký bod k jinému bodu, tím větší vliv na něj má, a to tak, že body blízko sebe mají hodnotu blízkou 1, vzdálené body mají hodnotu blízkou 0 a celé to funguje jako „měřítko podobnosti“ mezi body. Pokud bychom tedy chtěli předpovědět počasí, pomůže nám, že teploty naměřené v blízkých časech jsou si podobné. Naměříme-li v posledních hodinách teploty kolem -5 stupňů Celsia, bude i predikce pro příští hodinu ležet poblíž těchto hodnot.

1.4 Výběr modelu pro GP regresi

V sekci 1.2 jsme však ukázali, že hledat matematické modely, které přesně popisují data, není vůbec jednoduché nebo dokonce nemožné a je potřeba použít různé metody při hledání vhodného modelu.

Hlavní potíží je výpočet integrálů, které se objevovaly při sestavování Bayesovské hierarchie. Zde si ukážeme tři přístupy: nejprve obecný rámec variační inference, poté Type II – MLE jako úzce související bodovou alternativu a nakonec MCMC jako další metodu.

1.4.1 Variační inference

Variační inference (také Variační Bayes) je obecný přístup k aproximaci těžce vypočítatelných posteriorních rozdělání, které jsme uvedli v předchozí sekci. V celé této části se držíme výkladu z knihy [1]. Místo přímého výpočtu posteriorního rozdělání $p(\mathbf{w}|\mathbf{y}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)$, který dle (1.4) vyžaduje integraci přes celý prostor parametrů, hledáme jeho aproximaci z vhodné parametrické rodiny rozdělání.

Konkrétně zavádíme variační rozdělání $q(\mathbf{w})$ z rodiny $\mathcal{Q}$ (typicky např. normálních rozdělání se strukturovanou kovarianční maticí) a snažíme se ho co nejvíce přiblížit skutečnému posteriornímu rozdělání minimalizací KL divergence:

$$q^*(\mathbf{w}) = \arg \min_{q \in \mathcal{Q}} \text{KL}(q(\mathbf{w}) \| p(\mathbf{w}|\mathbf{y}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)). \quad (1.9)$$

Avšak přímá minimalizace KL divergence je opět náročná pro výpočet, neboť vyžaduje znalost normalizační konstanty (1.4). Namísto toho se maximalizuje tzv. *Evidence Lower Bound* (ELBO):

$$\mathcal{L}(q, \boldsymbol{\theta}) = \mathbb{E}_{q(\mathbf{w})}[\log p(\mathbf{y}|\mathbf{w}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)] - \text{KL}(q(\mathbf{w}) \| p(\mathbf{w}|\boldsymbol{\theta}, \mathcal{H}_i)). \quad (1.10)$$

Lze totiž ukázat, že:

$$\log p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i) = \mathcal{L}(q, \boldsymbol{\theta}) + \text{KL}(q(\mathbf{w}) \| p(\mathbf{w}|\mathbf{y}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)) \geq \mathcal{L}(q, \boldsymbol{\theta}), \quad (1.11)$$

kde nerovnost plyne z nezápornosti KL divergence. ELBO tedy tvoří dolní mez log-evidence a maximalizace $\mathcal{L}(q, \boldsymbol{\theta})$ přes q odpovídá minimalizaci KL divergence od skutečného posteriorního rozdělání, přičemž normalizační konstantu znát už nepotřebujeme.

Nevýhoda variační inference spočívá v tom, že výsledná aproximace je omezena volbou rodiny $\mathcal{Q}$. Příliš úzká rodina nemusí skutečné posteriorní rozdělání dobře zachytit.

1.4.2 Type II – MLE

Pokud bychom chtěli rychlejší a jednodušší řešení, a namísto hledání celého rozdělání nám stačí pouze bodový MLE odhad parametrů, použijeme tzv. Type II maximum likelihood (Type II – MLE), což je úzce související bodová alternativa k variační inferenci. Tato metoda spočívá v tom, že maximalizujeme tzv. *marginal likelihood*, což je pravděpodobnost dat daná modelem, integrovaná přes všechny možné hodnoty parametrů modelu. V případě GP regrese je *marginal likelihood* dána stejným vztahem jako normalizační konstanta na první úrovni hierarchie, tedy rovnicí (1.4), kde $\boldsymbol{\theta}$ jsou hyperparametry modelu, $\mathcal{H}$ je struktura modelu, $\mathbf{w}$ jsou parametry modelu, $\mathbf{X}$ jsou vstupní data a $\mathbf{y}$ jsou odpovídající cílové hodnoty. To znamená následující, vzpomeňme si na 2. úroveň hierarchie v Bayesovském přístupu, tj. na rovnici (1.2). Místo celého posteriorního rozdělání hyperparametrů hledáme jen jeho bodový odhad, a to maximalizací marginal likelihood:

$$\hat{\boldsymbol{\theta}} = \arg \max_{\boldsymbol{\theta} \in \Theta} p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i). \quad (1.12)$$

Za předpokladu rovnoměrného prioru $p(\boldsymbol{\theta} | \mathcal{H}_i)$ toto maximum odpovídá i maximu posteriorního rozdělání hyperparametrů.

Principem tohoto přístupu je, že na základě napozorovaných dat hledáme hyperparametry pro náš model, se kterými on popisuje naše data co nejlépe pro dané modely. Tato metoda ale má řadu nevýhod. Náš odhad totiž může skončit v lokálním maximu a nemusí vždy dosáhnout optimálních hodnot. To samozřejmě vynucuje nás zkoušet různé startovní body, zkoumat citlivost odhadu a patrně ladit parametry hledání. Tyto výpočty navíc nejsou triviální a vyžadují nemalé množství výpočetního výkonu a času.

Ze statistiky víme, že jedním ze způsobů, jak najít ML odhad, je použít logaritmickou transformaci a pak najít maximum této funkce. Ukážeme to na konkrétním příkladě a použijeme k tomu Gaussovský proces s RBF kernelem, který jsme definovali v sekci 1.3. Zkusme tedy rozepsat výraz $\log p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta})$ (člen $\mathcal{H}_i$ můžeme vynechat pro lepší čitelnost). Vzpomeňme si, že při definici Gaussovského procesu jsme použili vícerozměrnou Gaussovskou hustotu (1.6). Zlogaritmujeme tento výraz, přičemž předpokládáme nulovou střední funkci $m(\mathbf{x}) = 0$, kterou v celé práci používáme, a obdržíme:

$$\log p(\mathbf{y}) = -\frac{1}{2}\mathbf{y}^\top \boldsymbol{\Sigma}^{-1}\mathbf{y} - \frac{1}{2}\log |\boldsymbol{\Sigma}| - \frac{n}{2}\log 2\pi \quad (1.13)$$

Jediné, co nám tedy zbývá v případě Gaussovského procesu, je dosadit za kovarianční matici kernelovou matici $\mathbf{K}_y = \mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma_n^2 \mathbf{I}$, kde $\mathbf{K}(\mathbf{X}, \mathbf{X})_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ a σ_n^2 je rozptyl šumu, čímž obdržíme:

$$\log p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i) = -\frac{1}{2}\mathbf{y}^\top \mathbf{K}_y^{-1}\mathbf{y} - \frac{1}{2}\log |\mathbf{K}_y| - \frac{n}{2}\log 2\pi \quad (1.14)$$

Chceme-li najít extrém této funkce, musíme tento výraz derivovat podle hyperparametrů, čímž dostáváme [4]:

$$\begin{aligned} \frac{\partial}{\partial \theta_j} \log p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}) &= \frac{1}{2}\mathbf{y}^\top \mathbf{K}_y^{-1} \frac{\partial \mathbf{K}_y}{\partial \theta_j} \mathbf{K}_y^{-1} \mathbf{y} - \frac{1}{2} \text{tr} \left(\mathbf{K}_y^{-1} \frac{\partial \mathbf{K}_y}{\partial \theta_j} \right) \\ &= \frac{1}{2} \text{tr} \left(\left(\boldsymbol{\alpha} \boldsymbol{\alpha}^\top - \mathbf{K}_y^{-1} \right) \frac{\partial \mathbf{K}_y}{\partial \theta_j} \right), \end{aligned} \quad (1.15)$$

kde $\boldsymbol{\alpha} = \mathbf{K}_y^{-1}\mathbf{y}$. Zde je dokonce vidět, proč tento model není optimální z výpočetního hlediska. Je totiž známo, že najít inverzní matici je značně náročná operace. Pro standardní algoritmy platí, že jejich složitost je $O(n^3)$, kde n je velikost matice. V praxi ovšem n nabývá velkých hodnot. Avšak lze se vyhnout výpočtu přímé inverze, místo toho se může použít např. Choleského rozklad, který tento výpočet může usnadnit. Máme-li spočítanou inverzi, pak je složitost zbylých výpočtů $O(n^2)$.

1.4.3 Markov Chain Monte Carlo

Další metodou je Markov Chain Monte Carlo (MCMC). Její princip spočívá v tom, že integrál, který neumíme vyřešit analyticky, aproximujeme náhodným vzorkováním z příslušného rozdělení. Konkrétně už jsme si říkali, že integrály (1.4) a (1.5) nemusíme umět spočítat. Místo přímého výpočtu proto generujeme posloupnost vzorků, tzv. Markovův řetězec, zkonstruovanou tak, aby její rozdělení konvergovalo k cílovému posteriornímu rozdělení. Z vygenerovaných vzorků pak můžeme aproximovat samotné rozdělení i libovolné jeho statistiky. Existuje celá řada variant MCMC, např. *Metropolis-Hastings algoritmus*, *Gibbs sampling* nebo modernější *Hamiltonian Monte Carlo* [9]. Jejich podrobný rozbor však přesahuje rámec této práce, ucelený přehled poskytuje např. kniha [10]. Podrobněji se zastavíme jen u varianty *No U-Turn Sampler* (NUTS) [11], která staví na Hamiltonian Monte Carlo a při hledání posteriorních rozdělení patří k nejpoužívanějším. Hamiltonian Monte Carlo využívá gradient posteriorního rozdělení. Základní myšlenkou je pohyb v parametrickém prostoru.

Tento proces vnímáme jako fyzikální systém:

- $\boldsymbol{\theta}$ = poloha částice,

- zavádíme pomocnou proměnnou *momentum* $\mathbf{r}$,
- pohyb řídí Hamiltonovy rovnice s energií:

$$H(\boldsymbol{\theta}, \mathbf{r}) = -\log p(\boldsymbol{\theta}|\mathcal{D}) + \frac{1}{2}\mathbf{r}^T \mathbf{M}^{-1} \mathbf{r},$$

kde první člen je potenciální energie (negativní log-posterior) a druhý je kinetická energie, přičemž matice $\mathbf{M}$ (tzv. *mass matrix*) je volný parametr metody, který škáluje momentum a typicky se volí jako jednotková matice.

Rovněž si musíme uvědomit, že částice má tendenci se pohybovat po hladinách konstantní energie, a proto zkoumá prostor efektivněji než náhodné procházky, které používají předchozí algoritmy. Jednotlivé kroky metody Hamiltonian Monte Carlo shrnuje algoritmus 1.

Algoritmus 1 Hamiltonian Monte Carlo

- 1: Vzorkuj náhodný *momentum*: $\mathbf{r} \sim \mathcal{N}(\mathbf{0}, \mathbf{M})$
 - 2: Simuluj Hamiltonovy rovnice pomocí *leapfrog* integrace na L kroků délky ϵ
 - 3: Metropolis *accept/reject* krok (zachovává přesnou stacionaritu)
-

Hamiltonian Monte Carlo má však dva hyperparametry, které je těžké nastavit:

- délka kroku ϵ (*step size*)
- počet kroků L (*trajectory length*)

NUTS oba problémy řeší automaticky, každý ale jiným mechanismem. Délku kroku ϵ ladí během zahřívací fáze metodou duálního průměrování (*dual averaging*), délku trajektorie pak volí podle tzv. *U-turn kritéria*. To znamená, že pokračuje v simulaci, dokud trajektorie nezačne dělat tzv. *U-turn*, což znamená, že částice se po trajektorii otočí a začne se vracet zpět směrem k místu, odkud vyšla. Toto *U-turn kritérium* lze vyjádřit takto: směr pohybu začne být opačný než směr od startovní pozice, matematicky to znamená $(\boldsymbol{\theta}_{\text{forward}} - \boldsymbol{\theta}_{\text{backward}}) \cdot \mathbf{r}_{\text{forward}} < 0$, kde skalární součin vektoru mezi krajními body trajektorie a momentem je záporný, což signalizuje, že se pohybujeme zpět. Tento princip dává dobrý smysl. Jakmile se částice začne vracet zpět, přestává být efektivní pokračovat, protože to znamená, že jsme danou „energetickou hladinu“ už plně prozkoumali. Další kroky by jen ztrácely výpočetní čas tím, že by procházely oblastí, které už jsme navštívili. NUTS tedy automaticky zastaví simulaci v optimálním bodě, když jsme prozkoumali maximum užitečného prostoru, ale ještě před tím než začneme ztrácet efektivitu návratem zpět.

Společným rysem všech těchto přístupů je jejich vysoká výpočetní cena. V následující kapitole proto ukážeme alternativu, která ji amortizuje předtréninkem.

Kapitola 2

Prior–Data Fitted Networks

V této kapitole uvedeme samotnou architekturu Prior–Data Fitted Networks (dále jenom PFN). Začneme krátkou motivací, kde popíšeme hlavní ideu této Deep Learning architektury. Následně ukážeme mechanismy, které se schovávají uvnitř PFN, jinými slovy, jak PFN postupně zpracovává vstupní hodnoty, a to vše na příkladě Gaussovských procesů.

2.1 Úvod do Prior–Data Fitted Networks

Prior–Data Fitted Networks je Deep Learning (dále jenom DL) architektura, kterou v roce 2022 uvedli Samuel Müller a Noah Hollmann v článku [2]. Hlavní motivací pro vznik tohoto modelu je snaha posunout techniky Bayesovské statistiky na další úroveň spolu se strmým rozvojem v oblasti strojového učení. PFN je založeno na proslulé architektuře transformer [7], která je viníkem dnešního pokroku všemožných Deep Learning modelů, a o níž víme díky velkým jazykovým modelům, a není to náhoda, existuje totiž jisté spojení mezi těmito architekturami. Ačkoli samotnou architekturu PFN a její technické nuance budeme podrobně diskutovat až v následující sekci, zde jenom uvedeme její základní charakteristiky. PFN je tzv. *encoder-only* architektura (tj. neobsahuje *decoder* blok, ale pouze *encoder* blok transformera), na kterou je pak napojena softmax hlava pro vyhodnocování výsledků ve tvaru pravděpodobností. Na vstup PFN dostává sekvenci dat. V případě velkých jazykových modelů by se jednalo o slova nebo věty, umístěné v jistém pořadí. Tady je ta myšlenka shodná, ale namísto slov dostane síť neuspořádanou sekvenci dat, které získáme z prior rozdělení. Právě toto je důležité intuitivní pojetí, na kterém je založená celá ta architektura. Stejně jako po jazykovém modelu, který dostal sekvenci slov, budeme chtít, aby odhadl celkový kontext a jako výsledek např. i neznámé slovo, které by mohlo následovat, budeme i po PFN chtít, aby po předložení sekvence dat zkusilo uchopit celkový trend (neboli kontext) a předpovědět, co by mohlo následovat dále. Což je velmi relevantní, když zpracováváme časové řady a chceme řešit regresní problém, jak tomu je např. během práce s Gaussovskými procesy.

Celý princip fungování této metodiky lze popsat následujícím způsobem. Jak již jsme diskutovali dříve, účelem Bayesovského přístupu ve statistice je využití apriorní znalosti nebo informace o jistém jevu v přírodě. A tento princip se dobře prolíná s obecnou úlohou aproximace funkcí pomocí neuronových sítí, protože při jejich návrhu vždy musíme enkódovat apriorní znalosti o této funkci. To se dělá buď přímo skrz architekturu konkrétního modelu nebo za pomoci různých regularizací. Každopádně je to velmi netriviální úloha, kterou je třeba vždy řešit než začneme řešit kteroukoliv úlohu pomocí DL nástrojů. Jako jedno z možných řešení se nabízí právě tzv. Bayesovská inference, jejímž hlavním předpokladem je, že se jistý jev nebo proces vyskytuje v reálném světě a disponujeme informacemi o jeho chování. Tento předpoklad nám umožňuje definovat následující postup. Uvažujme jev nebo celý proces (to jest prior, pro jednoduchost si pod tím můžeme představit obyčejnou množinu dat), ze kterého

můžeme neomezeně vzorkovat simulované datové sady $\mathcal{D}$ reprezentující různé prediktivní úlohy (body). Bez újmy na obecnosti předpokládejme, že tyto datové sady obsahují vstupní příznaky $\mathbf{x} \in \mathbb{R}^d$ a k nim příslušné výstupní třídy $y \in \mathcal{Y}$. Během trénování sítě PFN náhodně vygenerujeme dávku (batch) obsahující K takových datových sad. S každou sadou v dávce pak naložíme podle algoritmu 2.

Algoritmus 2 Příprava jedné trénovací dávky pro PFN

- 1: **for** $i = 1, \dots, K$ **do**
 - 2: Zvol náhodně velikost kontextu N_i
 - 3: Rozděl vygenerovanou datovou sadu na kontextovou (trénovací) část $\mathcal{D}_{train}^{(i)} = (\mathbf{x}_j^{(i)}, y_j^{(i)})_{j=1}^{N_i}$ a testovací část obsahující (pro jednoduchost) jeden testovací vzorek $(\mathbf{x}_{test}^{(i)}, y_{test}^{(i)})$
 - 4: Předlož modelu data ve tvaru $D^{(i)} = \mathcal{D}_{train}^{(i)} \cup \{\mathbf{x}_{test}^{(i)}, y_{test}^{(i)}\}$
 - 5: **end for**
-

Následně natrénujeme náš model q_θ tak, že mu na vstup předložíme kontextovou sadu společně s testovacím příznakem a učíme ho predikovat správný výstup. K tomu minimalizujeme ztrátovou funkci Negative Log-Likelihood (NLL) přes celou dávku K vzorků ve tvaru:

$$L(\theta) = - \sum_{i=1}^K \log q_\theta(y_{test}^{(i)} | \mathbf{x}_{test}^{(i)}, \mathcal{D}_{train}^{(i)})$$

Optimalizací této funkce (např. pomocí gradientního sestupu) hledáme parametry $\hat{\theta} = \arg \min_{\theta} L(\theta)$, čímž se síť učí provádět inferenci in-context na základě poskytnutých dat.

Toto schéma nám umožňuje aproximovat posteriorní prediktivní rozdělení (PPD) pro libovolný problém, pro který jsme schopni definovat apriorní rozdělení (prior) a vzorkovat z něj data. Zde je namístě si ujasnit terminologii.

Posteriorní rozdělení (Posterior)

Posteriorní rozdělení popisuje naši nejistotu ohledně samotného generativního modelu p (nebo jeho parametrů) poté, co jsme pozorovali trénovací data $\mathcal{D}_n$. Matematicky se jedná o podmíněné rozdělení modelu vzhledem k těmto datům, které se značí jako $\Pi(p | \mathcal{D}_n)$.

Posteriorní prediktivní rozdělení (PPD - Posterior Predictive Distribution)

PPD naproti tomu popisuje nejistotu ohledně nových dat, která model dosud neviděl, jako je například predikce testovacího *labelu* y pro nový vstupní příznak x , na základě již pozorovaných dat $\mathcal{D}_n$. Vypočítá se jako průměr všech možných podmíněných rozdělení $p(y|x)$ vážených jejich posteriorní pravděpodobností. Matematicky je definováno pomocí integrálu:

$$\pi(y | \mathbf{x}, \mathcal{D}_n) = \int p(y | \mathbf{x}) d\Pi(p | \mathcal{D}_n).$$

Takže účelem PFN je aproximovat právě PPD.

V praxi to znamená, že model je trénován na široké rodině úloh vygenerovaných z tohoto prioru. Tím se PFN naučí implicitně provádět Bayesovskou inferenci nad daným prostorem funkcí.

V případě Gaussovských procesů to vypadá následovně: Jelikož známe přesný tvar rozdělení (např. Gaussovský proces s RBF kernelem), dokážeme z něj nekonečněkrát vzorkovat nová data. Navíc můžeme hyperparametrům RBF kernelu přiřadit vlastní rozdělení (tzv. hierarchický prior), čímž úlohu pro PFN zesložitíme. V této chvíli se model musí naučit generalizovat (tj. najít obecné řešení) přes celou rodinu funkcí pro všemožné hodnoty hyperparametrů. Potom stačí takto natrénovanému modelu předložit několik známých bodů (tzv. kontext) a on nám v jediném průchodu sítě vrátí rozdělení, které chceme predikovat a snaží se co nejvíce aproximovat původní funkci.

2.2 Praktická motivace k PFN

Abychom navázali na předchozí kapitulu, kde jsme uvedli dnes již standardní metody pro aproximaci PPD, ukážeme motivační příklady s porovnáním PFN s MCMC metodami a variační inferencí. Müller ve své práci tuto problematiku uvedl a na ní představil světu PFN architekturu [2]. Hlavní výhodou PFN je dle něj to, že na rozdíl od variační inference nebo metod Monte Carlo se učí aproximovat PPD přímo z datových vzorků. Jediným a velmi důležitým předpokladem ale je, abychom mohli rychle a výpočetně levně vzorkovat z apriorního rozdělení. Naproti tomu MCMC požaduje nenormované posteriorní rozdělení. Musíme umět spočítat jeho likelihood a také mít prior jako hustotu, ne jenom možnost vzorkovat. Pro variační inferenci potřebujeme znát sdružené rozdělení parametrů a dat, tato rozdělení jsou často těžce spočitatelná. Navíc často je optimalizace takových řešení výpočetně náročná (výpočet gradientů, ELBO optimalizace) a kvalita závisí na volbě variační rodiny rozdělení. Dalším úskalím je, že pro každý nový dataset je nutné spustit celý proces znovu, což jenom zhoršuje časovou a výpočetní náročnost při řešení jistého problému. Výhodou těchto klasických řešení je, že např. MCMC konverguje k pravému posterioru v limitě nekonečně mnoha vzorků. Type II – MLE (bodová alternativa k variační inferenci) konverguje k pravým hyperparametrům v limitě nekonečně dat za předpokladu korektně specifikovaného modelu. Navíc pro všechny modely platí, že systematická chyba takových aproximací, tzv. bias, jde k nule.

PFN obrací situaci. Model je předtrénován offline na obrovské mase syntetických datasetů z prioru a inference se pak provádí jediným dopředným průchodem (forward passem) sítí. Strukturální požadavek je výrazně slabší, stačí umět vzorkovat z apriorního rozdělení, není potřeba hustota ani její gradient. To otevírá třídu priorů, kterou klasické metody pojmut neumí, např. BNN s libovolnou architekturou nebo ručně psané generativní procedury. Inference je řádově rychlejší, protože model je předtrénovaný. Müller ukazuje cca 200× zrychlení proti Type II – MLE a přibližně 1 000× až 8 000× proti NUTS, navíc na rozdíl od NUTS neprovádí PFN před inferencí žádnou přípravnou fázi. Konečně, PFN není trénováno na rekonstrukci posterioru bod po bodu, ale přímo na predikční kvalitě skrz cross-entropy s PPD. Nevýhody jsou strukturální. Jak se ukáže dále, nemáme vždy zaručeno to, že bias bude klesat k nule. Naopak např. u architektury transformer bias nikdy nezmizí. Další nevýhodou je např. to, že pokud testovací dataset pochází z jiného rozdělení než trénovací prior, PFN může selhat nebo minimálně výsledky nebudou tak přesné. NUTS je oproti tomu robustní v tom smyslu, že jeho výstupem je vždy korektní posterior pro právě ten model, který byl explicitně specifikován, bez ohledu na původ dat. Nakonec je třeba mít na paměti, že PFN musí být předem natrénováno na konkrétním problému a s obrovským množstvím dat, např. pro aproximaci Gaussovského procesu na dostatečně bohaté rodině funkcí a hyperparametrů. Pokud by ale takové předtrénování udělal někdo za nás, používali bychom PFN jako hotový produkt, podobně jako dnešní velké jazykové modely, a pro libovolný problém bychom dostávali okamžitou aproximaci.

2.3 Vnitřní architektura PFN

V této sekci se pokusíme popsat procesy, které se dějí uvnitř PFN transformeru. Naším účelem není poskytnout kompletní popis architektury transformer obecně, za co odpovídají jednotlivé komponenty nebo parametry. Předpokládáme, že čtenář je již seznámen se základy architektury transformer. Pokud tak tomu není, doporučujeme nastudovat alespoň základní pojmy a principy této architektury. Potřebné zájemce jenom odkážeme na příslušnou kapitolu v knize [5]. Přesto pevně věříme, že našemu výkladu budou rozumět i laici v oblasti strojového učení, neboť zde nepotřebujeme pokročilé znalosti nebo složité pojmy a zkusíme vše popsat co nejjednodušeji. Pokud se takové pojmy ve výkladu objeví, slouží jenom pro odborníky v této oblasti a na porozumění hlavním principům nebudou mít velký vliv. Dále opět podotýkáme, že účelem práce není rozebírat konkrétní implementaci PFN a související s tím nuance.

Navíc PFN je relativně mladá architektura, která se neustále vylepšuje a aktualizuje, proto pro nás nemá smysl jít do takových detailů. Aktuální verzi zdrojového kódu PFN naleznete na GitHubu¹.

V předchozí sekci jsme již zmínili, že PFN je transformer, který se trénuje na obrovském množství syntetických datasetů, které vzorkujeme z prioru. Tímto způsobem se PFN naučí provádět Bayesovskou inferenci a po natrénování stačí jeden forward pass sítí a model vrátí kompletní PPD pro nové testovací body na vstupu. V této sekci budeme uvažovat, že naším priorem je Gaussovský proces s RBF kernelem. Dále chceme upozornit, že celý proces, který se odehrává uvnitř PFN od vstupu po výstup, popíšeme s použitím konkrétní konfigurace, kterou jsme běžně používali během zkoumání PFN:

1. Dimenze embeddingu $EMSIZE = 512$.
2. Počet *attention* hlav v každé vrstvě $NHEAD = 8$.
3. Dimenze vnitřních skrytých vrstev dopředných neuronových sítí $NHID = 1024$.
4. Počet vrstev $NLAYERS = 6$.

Jelikož se vše chystáme navíc ukázat pro případ 1D GP regrese, použijeme úplně klasické nastavení PFN, které uvádějí Müller a Hollmann v článku [2], konkrétně s těmito parametry:

- `features_per_group = 1`, nastaven na jedničku, protože řešíme 1D regresi.
- `attention_between_features = False`, slouží pro zpracování tabulkových dat. Pokud je zapnutý, jedná se o architekturu *TabPFN*, kterou v této práci neuvažujeme (viz Hollmann et al. [6]).

Nakonec uvažujme, že provádíme inferenci pro GP regresi s 20 trénovacími a 50 testovacími body. Teď už máme nachystané formální parametry a nastavení, což nám umožňuje se pustit na samotný popis. Celý proces si rozdělíme na několik fází.

2.3.1 Fáze 1 – Meta-Learning

Pro obecnost ukážeme, jak probíhá učení pro případ, kdy hyperparametry pro RBF kernel nejsou známy a pochází z nějakého rozdělení. Případ fixních hyperparametrů je pouze speciálním případem tohoto problému a všechno, co napíšeme dále, platí i pro tento speciální případ. Meta-Learning je paradigma strojového učení, které spočívá v tom, že namísto toho, abychom model naučili řešit jeden konkrétní fixní problém, naučíme ho řešit celou třídu podobných úkolů. Konkrétně v našem případě by to znamenalo, že model naučíme na základě dat z GP s RBF kernelem, kde mu ukážeme celé spektrum všemožných hyperparametrů, které se v RBF kernelu vyskytují. Potom stačí jenom předložit modelu těch 20 kontextových bodů a hned bude schopen aproximovat GP, ze kterého tyto body pocházejí. V předchozí sekci jsme již drobně popsali, jak funguje trénink PFN. Formálněji a mnohem podrobněji jej shrnuje algoritmus 3.

Tímto způsobem se dostáváme do situace, kdy model vidí miliony různých datasetů generovaných z GP prioru a učí se pravidla, dle kterých pak musí uhádnout, jak vypadá posteriorní predikce na základě předložených dat. Tohle je velmi důležitá myšlenka, PFN se neučí aproximovat konkrétní funkce, učí se inferenční pravidlo.

¹<https://github.com/automl/PFNs>

Algoritmus 3 Meta-trénink sítě PFN na syntetických GP datech

Require: Model q_θ , apriorní rozdělení hyperparametrů Π , velikost kroku η , velikost dávky m

- 1: **while** není dosaženo konvergence **do**
- 2: **for** $j = 1, \dots, m$ **do**
- 3: Vzorkuj hyperparametry GP: $(\ell_j, \sigma_j, \sigma_{n,j}) \sim \Pi$
- 4: Vzorkuj vstupní body $x_1, \dots, x_N$ rovnoměrně z $[0, 1]$
- 5: Vzorkuj výstupy z GP s daným kernelem: $(y_1, \dots, y_N) \sim \mathcal{GP}(0, k_{\ell_j, \sigma_j}) + \mathcal{N}(0, \sigma_{n,j}^2 I)$
- 6: Rozděl na kontextovou sadu $\mathcal{D}_j = \{(x_i, y_i)\}_{i=1}^n$ a testovací body $\{(x_i^*, y_i^*)\}_{i=1}^t$
- 7: **end for**
- 8: Předej q_θ kontexty $\mathcal{D}_j$ a testovací vstupy x_i^* ; model vrátí prediktivní rozdělení přes biny
- 9: Spočítej ztrátu (NLL):

$$\mathcal{L}(\theta) = -\frac{1}{m} \sum_{j=1}^m \sum_{i=1}^t \log q_\theta(y_i^* | x_i^*, \mathcal{D}_j)$$

- 10: Aktualizuj: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$
 - 11: **end while**
 - 12: **return** $\hat{\theta}$
-

2.3.2 Fáze 2 – Vstup do modelu

Během inference model dostane tři tenzory:

1. Souřadnice trénovacích bodů `x_train`.
2. Naměřené hodnoty na trénovacích bodech `y_train`.
3. Souřadnice testovacích bodů `x_test`.

Jak už plyne z logiky věci, `x_train` a `y_train` jsou dva různé objekty, první říká, kde jsme měřili (souřadnice na ose x), a druhý říká, co jsme v těchto místech naměřili. Tak dohromady tvoří datové body (x_i, y_i) . Souřadnice `x_test` nám říká, kde chceme ty predikce, přitom model by měl vrátit pravděpodobnostní rozdělení pro příslušné y -hodnoty, ale k tomu se dostaneme později v našem výkladu. Následně se `x_train` a `x_test` spojí do jedné sekvence a model s nimi pracuje jako s jedním objektem. Přičemž modelu současně explicitně řekneme, že prvních 20 souřadnic jsou trénovací hodnoty a zbytek jsou testovací (50 testovacích souřadnic), jak jsme si to definovali na začátku sekce. Pro y -hodnoty je ta situace podobná, vytvoříme jednu sekvenci pro `y_train` a `y_test` hodnoty, až na to, že místo test hodnot vložíme NaN hodnoty, protože právě ty musí model odhadnout a nechceme je do modelu vkládat v žádné podobě. Pro zájemce uvedeme, jak to pak vypadá v kódu. V metodě `forward()` se `x_train` a `x_test` spojí pomocí funkce konkaténace:

```
x = torch.cat((x_train, x_test), dim=1) # (1, 70, 1)
```

Jedná se o tenzor, kde první dimenze je počet datasetů, které zpracováváme naráz, druhá dimenze je počet bodů v sekvenci a třetí dimenze je počet příznaků, který jsme na začátku této sekce nastavili na 1. Pro y -hodnoty analogicky:

```
y = torch.cat((y_train, NaN * torch.zeros(1, 50, 1)), dim=1) # (1, 70, 1)
```

Zde je explicitně vidět, že test hodnoty se nahradí za NaN. Dále si myslíme, že je namístě říct jedno upřesnění, aby nedošlo k nedorozumění. To, že modelu v y -hodnotách předkládáme NaN neznamena, že pak

budeme chtít tyto hodnoty nahradit za odhady někde v průběhu zpracovávání, je to čistě technická záležitost. Zůstanou NaN po celou dobu. Výsledky obdržíme v jiné podobě, opět, budeme o tom diskutovat dále.

Dále následuje tzv. *Feature grouping*, o parametru, který určuje počet příznaků jsme také zmiňovali na začátku sekce. Připomínáme, že pro náš jednoduchý případ 1D regrese je nastaven na 1. Tento mechanismus slouží hlavně pro případ, kdy chceme zpracovávat tabulkové hodnoty, anebo pracovat s vícerozměrnými daty, ale to se týká hlavně verze modelu *TabPFN*. V tabulkových datech s 12 sloupci (věk, plat, výška, ...) model potřebuje tyto sloupce nějak zpracovat. Parametr `features_per_group` určuje, kolik sloupců se seskupí do jednoho *tokenu*. Pro náš 1D GP je grouping triviální: máme 1 příznak, tedy 1 grupu. Tenzor x se tak přeformátuje z $(1, 70, 1)$ na $(1, 70, 1, 1)$.

Dalším krokem je převod dat na vstup na tzv. *embedding*, tj. do řeči, které rozumí transformer. Ve skutečnosti se jedná o vytváření nového prostoru, ve kterém pak PFN bude operovat s daty, a kterému rozumí víc, než kdyby musel pracovat pouze se sekvencí dat.

X-encoder:

Zde to je jednoduchá lineární vrstva, která vezme jednu konkrétní číselnou hodnotu x —souřadnice a vynásobí ji maticí vah 1×512 (parametr velikosti embeddingu jsme nastavili nahoře). Výsledkem je 512—dimenzionální vektor (*embedding*). Tímto způsobem jsme zakódovali polohu bodu v tom novém prostoru, ze kterého model umí jednoduše číst a operovat v něm. Výstupem je `embedded_x` tvaru $(1, 70, 1, 512)$

Y-encoder:

Pro y —hodnoty je ta situace zajímavější, jelikož v předchozím kroku, jsme si řekli, že pro testovací hodnoty modelu předložíme NaN hodnoty. A teď otázkou je, jak tyto hodnoty zakódovat do *embeddingu*. K tomu slouží metoda `NanHandlingEncoderStep`, která nejdříve transformuje skalární y —hodnotu na vektor délky 2:

1. Pro trénovací pozice (y existuje): $[y, -1]$, tedy skutečná hodnota a indikátor „data jsou k dispozici“.
2. Pro testovací pozice ($y = \text{NaN}$): $[0, +1]$, tedy nulová hodnota a indikátor „data chybí“.

Pak lineární vrstva opět převede dvourozměrný vektor do 512—dimenzionálního prostoru. Výstupem je `embedded_y` tvaru $(1, 70, 512)$, tady je o jeden parametr méně, protože příznaky mezi skupinami se nevztahují na y —hodnoty.

V této chvíli máme dva *embeddingy*, cílem ale je, mít jeden hromadný *embedding*, který v sobě nese informace o souřadnici x a odpovídající jí hodnotě y . Zde opět existují dva způsoby implementace v PFN: pro `attention_between_features=False` a `attention_between_features=True`, kde poslední verze je opět určena pro práci s vícedimenzionálními daty. Pro tu první verzi je postup velmi jednoduchý, `embedded_x` a `embedded_y` jednoduše sečteme. Výsledkem je jeden *token* pro jednu konkrétní pozici, tj. tento *token* kombinuje informaci o poloze x a hodnotě y v jednom vektoru. Sčítání tady funguje, protože oba *embeddingy* jsou již stejného tvaru a jsou ve stejném 512—dimenzionálním prostoru a model bude umět z tohoto součtu dekodovat obě informační položky díky předchozímu postupu. V kódu to pak formálně vypadá následovně:

```
1 embedded_input = embedded_x + embedded_y.unsqueeze(2)
2 # (1, 70, 1, 512) + (1, 70, 1, 512) -> (1, 70, 1, 512)
```

Druhý režim zde již popisovat nebudeme, zájemce odkážeme na zdrojové kódy na GitHub, které jsme uvedli na začátku kapitoly.

2.3.3 Fáze 3 – Transformer vrstvy

Tenzor `embedded_input` pak musí projít 6 transformer vrstvami (jejich počet jsme též definovali na začátku sekce). Jakmile `embeddingy` vstupují do transformer vrstev je zvykem jim říkat *token*. To znamená, že objektům `embedded_x` a `embedded_y` bychom říkali `x_token` a `y_token`. Ačkoliv jsme tyto dva `embeddingy` sloučili do jednoho, pořád musíme na ně nahlížet jako na samostatné objekty, protože naším cílem je nakonec získat `y_token`, který v sobě bude obsahovat predikce pro testovací hodnoty. Sloučení se tedy provádí pouze pro technické potřeby architektury. Vraťme se ale zpátky k věci. Každá vrstva v naší konfiguraci provádí dvě hlavní operace:

1. *Self-attention* přes datové body (neplést s *attention between features*)
2. Feedforward MLP

Self-attention:

Tenzor se transponuje na (1, 1, 70, 512), aby sekvence 70 bodů byla na dimenzi $\text{dim}=2$ (standardní konvence pro *attention*). Multi-head *attention* s 8 hlavami pak počítá:

Pro každou hlavu h (z 8):

$$\mathbf{Q}^{(h)} = x \cdot \mathbf{W}_Q^{(h)}, \quad \mathbf{K}^{(h)} = x \cdot \mathbf{W}_K^{(h)}, \quad \mathbf{V}^{(h)} = x \cdot \mathbf{W}_V^{(h)}$$

kde $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{512 \times 64}$ a $\mathbf{W}_V \in \mathbb{R}^{512 \times 64}$ (protože $512/8 = 64$).

Klíčový detail: $\mathbf{K}$ a $\mathbf{V}$ se počítají pouze z trénovacích bodů. V kódu:

```
1 attention_src_x = x[:, :single_eval_pos].transpose(1, 2)
2 # first 20 positions
```

Attention matice je pak:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q} \cdot \mathbf{K}^T}{\sqrt{64}}\right)$$

Pro trénovací body (pozice 0–19) je $\mathbf{Q}$ i $\mathbf{K}$ ze stejných 20 bodů, takže matice je 20×20 , trénovací body se dívají na sebe navzájem.

Pro testovací body (pozice 20–69) $\mathbf{Q}$ pochází z testovacích bodů, ale $\mathbf{K}$ pochází z trénovacích bodů. Matice je 50×20 , testovací body se dívají na trénovací body.

Tohle je přímá analogie ke GP predikci. Ve formuli $\mu_* = k(x_*, \mathbf{X})\mathbf{K}^{-1}\mathbf{y}$ se testovací body taky „dívají“ na trénovací body přes kernel $k(x_*, \mathbf{X})$. Rozdíl je, že GP používá explicitní RBF kernel, zatímco PFN se naučilo vlastní implicitní „kernel“ zakódovaný ve váhách $\mathbf{W}_Q$ a $\mathbf{W}_K$.

Výstup *attention*:

$$\text{Output}^{(h)} = \mathbf{A} \cdot \mathbf{V}^{(h)}$$

Všech 8 výstupů (každý $\in \mathbb{R}^{64}$) se zřetězí a projde výstupní projekcí $\mathbf{W}_O$:

$$\text{MultiHead} = \text{Concat}(\text{Output}^{(1)}, \dots, \text{Output}^{(8)}) \cdot \mathbf{W}_O$$

Po *attention* následuje reziduální spojení (vstup + výstup *attention*) a LayerNorm, tedy normalizace, která stabilizuje trénink. Reziduální spojení je technika, která přidává původní vstupní reprezentaci (třeba `embedding` datového bodu) k výstupu nelineární vrstvy (například MLP) těsně předtím, než tento výsledek pošle dál do sítě. Reziduální spojení zajišťuje, že model nezačíná optimalizaci chaotickým hádáním, nýbrž bezpečným průtokem dat, ke kterému postupně přidává potřebnou komplexitu. Co se týče LayerNorm, ten normalizuje `embedding` na nulový průměr a jednotkový rozptyl přes posledních d dimenzí (v našem případě 512). Pro vektor x délky 512:

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sigma + \epsilon},$$

kde μ a σ jsou průměr a směrodatná odchylka přes 512 komponent vektoru a $\epsilon = 10^{-5}$ je malá konstanta pro numerickou stabilitu.

MLP – Multi-Layer Perceptron:

MLP je dvouvrstvá dopředná síť:

vstup (512) $\rightarrow$ Linear(512 $\rightarrow$ 1024) $\rightarrow$ GELU $\rightarrow$ Linear(1024 $\rightarrow$ 512) $\rightarrow$ výstup (512)

Vezme 512-dimenzionální vektor, rozšíří ho do 1024 dimenzí přes lineární vrstvu, aplikuje nelineární aktivaci GELU (hladká varianta ReLU), a zase zúží zpět na 512.

GELU (*Gaussian Error Linear Unit*) je funkce

$$\text{GELU}(x) = x \cdot \Phi(x),$$

kde Φ je distribuční funkce standardního normálního rozdělení. Pro velká kladná x se chová jako identita, pro velká záporná x vrací přibližně nulu.

Proč nakonec musíme přidávat MLP je pro zkušeného čtenáře v oblasti strojového učení poněkud triviální otázka, vysvětlíme i pro ostatní čtenáře. *Attention* je totiž lineární operace (ačkoliv se to nezdá), počítá pouze vážené průměry. MLP do tohoto procesu přidává nelinearitu, bez něj by transformer mohl dělat jenom lineární transformace s embeddingy. Avšak ve strojovém učení je to nedostačující, chceme zachytit i složitější transformace a operace, čímž zvyšujeme generalizační schopnosti modelu. A právě díky této vlastnosti může PFN aproximovat např. inverzi matice $\mathbf{K}^{-1}$ v GP. Váhy druhé lineární vrstvy jsou inicializovány na nulu. To znamená, že na začátku tréninku MLP vrací nulový vektor, a tak se tato vrstva na začátku chová jako identita. To se však mění v průběhu tréninku. Model začíná od jednoduchého stavu a postupně se učí komplexnější transformace. Po MLP opět následuje reziduální spojení a normalizace.

Toto se opakuje tolik, kolik vrstev obsahuje model, v našem případě 6 krát. Po každé vrstvě se embedding každého bodu obohacuje o informaci z ostatních bodů (přes *attention*) a transformuje (přes MLP). Po 6 vrstvách embedding testovacího bodu obsahuje dostatečnou informaci k tomu, aby *decoder* z něj vyrobil prediktivní rozdělení.

Po průchodu 6 vrstvami dostáváme opět nový tenzor. Ten obsahuje zpracované reprezentace jak trénovacích, tak i testovacích bodů. Naším úkolem teď by bylo extrahovat testovací hodnoty y (neboli y -tokeny, jak jsme diskutovali výše), respektive jejich predikce, které model postupně vyrobil. Zbytek je pro účely inference nepodstatný. Platí totiž, že celá architektura PFN je postavena tak, že se právě v y -tokenu postupně akumulují informace přes všechny vrstvy transformeru přes *attention* z x -tokenů a z kontextu trénovacích bodů. Na konci celého procesu y -token pro testovací pozici obsahuje 512-dimenzionální vektor, který implicitně kóduje celé posteriorní prediktivní rozdělení (PPD) pro daný trénovací bod. Uveďme opět pro zájemce, jak tento proces vypadá uvnitř, což by mělo pomoci k pochopení, co se v tomto kroku odehrálo:

```
1 # Split into train and test part
2 test_encoder_out = encoder_out[:, current_context_len:, :]
3 # (1, 50, 1, 512) -- test positions (20-69)
4
5 train_encoder_out = encoder_out[:, :current_context_len, :]
6 # (1, 20, 1, 512) -- train positions (0-19)
```

Proměnná `current_context_len` je rovna `single_eval_pos = 20`.

V dalším kroku se extrahuje y -token, tedy poslední token v dimenzi feature groups:

```
1 test_encoder_out = test_encoder_out[:, :, -1, :]
2 # (1, 50, 512) -- only y-token for test positions
```

2.3.4 Fáze 4 – Decoder, převod embeddingu na logity.

Jsme si již řekli, že PFN je *encoder-only* architektura, tj. *decoder* blok, který je součástí obecné architektury transformer, v něm chybí. Decoder blok ale je napojen na hlavu, která v sobě obsahuje opět MLP vrstvu a softmax vrstvu. Tyto součástky však budeme potřebovat. Na konci předchozí fáze jsme získali tenzor `test_encoder_out` o tvaru (1, 50, 512), který v sobě obsahuje pro každý z 50 testovacích bodů jeden 512–dimenzionální embedding. Ted' potřebujeme z tohoto vektoru vytvořit prediktivní rozdělení. To se opět udělá pomocí MLP (jednoduchá dopředná síť), která je doplněná o jednu novinku. Celkově schéma vypadá následovně:

vstup (512) → Linear(512 → 1024) → GELU → Linear(1024 → numBars) → výstup (numBars)

Jednotlivé prvky v schématu, který jsme uvedli na začátku sekce, plní následující funkci:

1. `Linear(512 -> 1024)`: Vezme 512-dimenzionální embedding a promítne ho do 1024 dimenzí. To je lineární transformace, tedy násobení maticí vah $W_1 \in \mathbb{R}^{512 \times 1024}$.
2. `GELU()`: Aplikuje nelineární aktivaci. Bez ní by dvě lineární vrstvy za sebou byly ekvivalentní jedné lineární vrstvě (součin matic je matice). GELU umožňuje *decoderu* zachytit nelineární vztahy mezi embeddingem a výstupním rozdělením.
3. `Linear(1024 -> numBars)`: Zúží z 1024 dimenzí na numBars (řekněme 1000). Výstupem je 1000 čísel, tzv. *logity*, a to jedno pro každý bin.

Je vidět, že tady se objevuje nová proměnná numBars, jedná se totiž o analogii binu v histogramech, který se zavádí pro tzv. *BarDistribution*.

BarDistribution

BarDistribution je po–částech konstantní pravděpodobnostní rozdělení. V článku [2] se mu říká *Riemann distribution*, jedná se o diskretizaci spojitého rozdělení. Avšak je důležité odlišit tento koncept od klasického histogramu. Histogram odhaduje hustotu z dat. Hlavním principem je, že počítáme, kolik naměřených hodnot padlo do každého binu, a to určuje výšku jednotlivých sloupců. BarDistribution je naopak výstup modelu, tj. my neděláme měření a nepočítáme, kolik hodnot padlo do toho či oného binu. Právě samotný model určuje výšku sloupců dle nějakých svých pravidel. Neuronová síť na výstupu vrátí n čísel, každé číslo říká, jak vysoký má být jeden sloupec. To jest rozdělení určuje síť, to umožňuje pak modelovat pestrou škálu různých rozdělení a nejsme omezeni jenom normálním, exponenciálním rozdělením apod. V implementaci PFN se binům říká *bucket*. V implementaci je BarDistribution definován pomocí hranic (parametr `borders`), tím definujeme interval, na kterém je tato funkce definována. Může to vypadat například takto:

`borders = [-3.0, -2.994, ..., 0.0, ..., 2.994, 3.0]` Mezi sousedními hranicemi leží jeden bin. Bin č. i pokrývá interval $[b_i, b_{i+1})$ a má šířku $w_i = b_{i+1} - b_i$. V naší konfiguraci jsou `borders` typicky rovnoměrně rozložené, takže všechny biny mají stejnou šířku. Implementace ale podporuje i nerovnoměrné dělení (hustší biny v zajímavých oblastech, řidší na krajích).

Logit v tomto případě je jenom surové číslo z poslední lineární vrstvy našeho *decoderu*. Může to být jakékoliv reálné číslo. Důležité ale je, že nemají žádné jednotky, nesčítají se na jedničku a v této etapě nemají žádnou přímou pravděpodobnostní interpretaci. Logit je pouze signál pro určitý bin, do kterého spadne, jak moc si model myslí, že skutečná hodnota padne do daného intervalu. Čím vyšší je logit, tím více si je model jistý. Ale pro lepší interpretovatelnost musí tyto hodnoty nejdříve projít softmax vrstvou. Uvažujeme-li, že máme 1000 binů, softmax převede 1000 logitů na 1000 pravděpodobností:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{1000} e^{z_j}}$$

26

Logicky pak součet všech pravděpodobností je přesně 1 a vyšší logit implikuje vyšší pravděpodobnost. Hustotu pravděpodobnosti potom definujeme jako $f_i = \frac{p_i}{w_i}$, kde w_i je šířka binu. Širší bin musí mít nižší hustotu při stejné pravděpodobnosti, aby toto rozdělení dávalo smysl jako aproximace spojitého rozdělení. Při rovnoměrných binech je tento rozdíl jen konstantní škálování. Evidentně pak můžeme dopočítat i všechny potřebné deskriptivní charakteristiky jako střední hodnotu, rozptyl atd. Jak jsme již minili dříve, neklademe na BarDistribution žádná omezení co se týče tvaru výsledného rozdělení. Avšak v kontextu GP inference výstup typicky vypadá přibližně jako Gaussovský zvon. To proto, že GP posteriorní prediktivní rozdělení je přesně normální rozdělení. Pokud PFN dobře aproximuje GP posterior, biny budou mít tvar zvonu. Ale model to nikdo nenutí, naučil se to sám, protože loss funkce, se kterou byl model trénován, ho odměňuje za to, že jeho rozdělení co nejlépe odpovídá skutečnému posteriornímu rozdělení.

Poznámka o prokletí dimenze

Uveďme jednu zajímavost, která se vyskytuje u BarDistribution. U tohoto konceptu se totiž neobjevuje tzv. prokletí dimenze (anglicky *curse of dimensionality*), na které jsou běžné histogramy náchylné. Připomeňme, kdy toto prokletí nastává. Objevuje se u diskretizačních metod tehdy, když binujeme prostor, jehož dimenze neustále roste. Počet binů potřebný k udržení dané přesnosti pak roste exponenciálně, konkrétně jako $O(m^d)$ pro m binů na osu a dimenzi d . Přesně tak je tomu například u histogramového odhadu hustoty, kde se může diskretizovat vysokodimenzionální vstupní prostor. BarDistribution ale diskretizuje výhradně výstupní veličinu y , která je skalární, a to nezávisle na dimenzi vstupu x . Zdůrazněme, že to není ve sporu s tím, že model vrací celé rozdělení. Výstupem modelu je sice mnoho čísel (výšky všech binů), ale toto rozdělení popisuje jedinou skalární veličinu, binuje se tedy jen jedna osa. I když bychom PFN trénovali na regresi s dimenzí $d = 5$ nebo $d = 100$, prediktivní rozdělení $q_\theta(y | x, \mathcal{D})$ je vždy jednorozměrné rozdělení nad $\mathbb{R}$. Počet binů potřebný k aproximaci této jednorozměrné hustoty s danou přesností tedy vůbec nezávisí na dimenzi vstupu. Veškerou práci s vysokodimenzionálním vstupem totiž odvádí transformer ve svých spojitých reprezentacích, do binování už vysoká dimenze nevstupuje.

Dále poskytneme krátký komentář, proč během trénování PFN volíme pro loss funkci právě Negative Log-Likelihood (NLL) tvar. NLL je ztrátová funkce, která měří kvalitu predikovaného rozdělení. Její výpočet pro jeden testovací bod shrnuje algoritmus 4.

Algoritmus 4 Výpočet NLL ztráty přes BarDistribution pro jeden testovací bod

- 1: Vyprodukuji modelem logity $z_1, \dots, z_{1000}$
 - 2: Převeď logity softmaxem na pravděpodobnosti $p_1, \dots, p_{1000}$
 - 3: Převeď pravděpodobnosti na hustoty $f_i = p_i/w_i$, kde w_i je šířka binu
 - 4: Najdi bin k , do kterého padne skutečné y_{test}
 - 5: Spočítej loss pro tento bod: $\mathcal{L} = -\log(f_k) = -\log(p_k) + \log(w_k)$
-

Člen $\log(w_k)$ je konstanta (borders se nemění), takže gradient teče jen přes $-\log(p_k)$. Minimalizace NLL je ekvivalentní maximalizaci likelihood, to znamená, že model se učí přiřadit co nejvyšší hustotu tam, kde skutečné y leží.

Otázkou je, proč bychom nemohli volit např. tvar Mean Square Error (MSE) jako loss funkci. MSE loss by měřil jen $(y_{\text{pred}} - y_{\text{true}})^2$, tj. chybu bodového odhadu. NLL přes BarDistribution měří kvalitu celého rozdělení. Model se učí nejen správný střed, ale i správnou šířku a tvar nejistoty. To je přesně to, co potřebujeme pro aproximaci GP posterioru, chceme celou PPD, ne pouze průměr. Navíc, jak se ukáže dále [3], minimalizace NLL vede k optimálnímu řešení, které je Bayesovské posteriorní prediktivní rozdělení, tedy přesně to, co GP analyticky počítá.

Na konci sekce zkusíme shrnout kompletní postup. Na úplném začátku jsme modelu předložili sadu trénovacích a testovacích bodů, přitom ale model nedostal y_{test} a účelem bylo ho naučit tyto hodnoty predikovat. PFN postupně zpracovalo a přetransformovalo skrz svoje vrstvy $\mathbf{x}_{\text{train}}$, $\mathbf{y}_{\text{train}}$ a

`x_test`. To znamená, že model zpracoval mu předložené informace jako celek naráz. Během toho se PFN naučilo pravidlo, jak se chovají různé GP posteriory pro různé parametry. Na konci jsme dostali pravděpodobnostní rozdělení pomocí převodu na logity a použití BarDistribution. Výsledek jsme vždy porovnali se skutečným GP a jako metriku, jak moc dobře PFN odhaduje skutečnost, jsme použili NLL, který bychom chtěli minimalizovat. Po dostatečném tréninku jsme pak dostali model, který na základě předložených dat ze skutečného GP prioru, které nikdy neviděl, je schopen odhadnout, odkud pochází a jak vypadá funkce, ze které pochází.

2.4 Teoretické statistické vlastnosti PFN

V této sekci uvedeme dosud známé statistické vlastnosti PFN, které publikoval Nagler v článku [3]. V okamžiku, kdy již máme natrénovaný model, dává smysl prozkoumat jeho statistické vlastnosti a ptát se, proč a jak fungují jeho prediktivní a aproximační schopnosti. Müller ve své práci ukázal empirická pozorování, která ale nepodpořil teoretickým zdůvodněním. To se avšak podařilo Naglerovi, který dokázal vybudovat teoretický základ pro PFN a řádně odůvodnit, proč a jak architektura PFN dává dobré výsledky při Bayesovské inferenci. Nagler ve své práci používá určitý formalismus pro zavedení celého problému, na kterém pak buduje samotnou teorii. My tento formalismus v této práci zachováváme. Z jeho článku přebíráme i všechny věty této sekce, zdroj proto u jednotlivých vět už opakovat nebudeme.

2.4.1 Statistický model

Uvažujme klasifikační úlohu, která se skládá ze tříd $Y \in \mathcal{Y}$ a příznaků $X \in \mathcal{X} \subseteq \mathbb{R}^d$. Předpokládejme dále, že máme iid data $D_n = (Y_i, X_i)_{i=1}^n$ pocházející z nějakého teoretického, avšak neznámého rozdělení p_0 . Naším cílem je predikovat toto podmíněné rozdělení na základě dostupných dat:

$$p_0(y | x) = P(Y = y | X = x).$$

Z hlediska Bayesovského neparametrického odhadování můžeme na p_0 nahlížet jako na realizaci náhodného parametru $p \in \mathcal{P}$, kde $\mathcal{P}$ je prostor podmíněných pravděpodobnostních rozdělení nad $\mathcal{Y} \times \mathcal{X}$. Tento parametr má rozdělení Π , kterému budeme říkat prior. Prior bude vyjadřovat naše přesvědčení o tom, jaké modely p jsou pravděpodobnější pro daný problém ještě předtím než uvidíme data. Formálně celý proces, ze kterého pocházejí data lze zapsat takto:

1. Vygeneruj $p \sim \Pi$.
2. Vygeneruj iid vzorky $D_n = (Y_i, X_i)_{i=1}^n$ a jeden dodatečný pár (Y, X) z modelu p .

Tento mechanismus definuje sdružené rozdělení nad $(D_n \cup (Y, X), p)$ pro každé n . Skutečné rozdělení p_0 je ta realizace p , ze které skutečně pocházejí naše data, zpravidla je p_0 neznámá.

Pro každé n poskytuje výše popsany statistický model dobře definované sdružené rozdělení n -tice $(D_n \cup (Y, X), p)$. Na základě tohoto modelu lze $p_0(y | x)$ aproximovat pomocí posterior predictive distribution (PPD):

$$\pi(y | x, D_n) = P(Y = y | X = x, D_n),$$

kterou jsme již zmiňovali v předchozí sekci, uved' me její definici ještě jednou i zde a se zařazením do kontextu. Tím vzniká celá rodina PPD indexovaná n . Pokud prior Π faktorizuje nezávisle pro marginální složky $p(y | x)$ a $p(x)$, tj. rozdělení příznaků a rozdělení tříd jsou v prioru nezávislé, pak lze PPD dále zapsat jako:

$$\pi(y | x, D_n) = \int p(y | x) d\Pi(p | D_n), \quad (2.1)$$

kde $\Pi(\cdot | D_n)$ je tzv. *posterior*, což je podmíněné rozdělení p dané pozorovanými daty D_n , získané Bayesovým pravidlem. Tento integrál říká v zásadě relativně jednoduchou věc: průměrujeme $p(y | x)$ přes všechna možná p vážená jejich posteriorní pravděpodobnostmi, pak modely p , které jsou konzistentní s D_n dostanou větší váhu. Podmínka faktorizace prioru je spíše technická. Platí-li podmínka, pak když jsme viděli data D_n , říkají nám něco o $p(y | x)$, ale x -ová souřadnice testovacího bodu sama o sobě nic neříká o $p(y | x)$. Pokud by totiž ta podmínka neplatila, potom by i testovací souřadnice x byla informativní pro $p(y | x)$, což pro náš případ nedává smysl. PPD $\pi(y | x, D_n)$ je tedy posteriorní střední hodnota podmíněných rozdělení $p(y | x)$, vážená tím, jak jsou jednotlivé modely p konzistentní s D_n .

PFN je numerická aproximace celé rodiny PPD $\{\pi(\cdot | \cdot, D_n), n \in \mathbb{N}\}$. Základním teoretickým poznatkem je, že PPD sama o sobě je pro každé n optimálním prediktorem v následujícím smyslu. Definujeme třídu všech podmíněných pravděpodobnostních funkcí takto:

$$\mathcal{Q} = \left\{ q : (\mathcal{Y} \times \mathcal{X})^{n+1} \rightarrow [0, 1] \mid \sum_{y \in \mathcal{Y}} q(y | \cdot, \cdot) = 1 \right\}.$$

Každé $q \in \mathcal{Q}$ je tedy funkce, která dostane n trénovacích párů (Y_i, X_i) a jeden testovací vstup $\mathbf{X}$, a vrátí rozdělení přes $\mathcal{Y}$.

Věta 2.4.1. π z rovnice 2.1 splňuje:

$$\pi = \arg \max_{q \in \mathcal{Q}} \mathbb{E}_{\Pi}[\log q(Y | X, D_n)], \quad (2.2)$$

kde $\mathbb{E}_{\Pi}$ je střední hodnota přes $(Y, X) \cup D_n$ generované dle mechanismu popsaného v sekci 2.4.1.

Jinými slovy PPD π maximalizuje očekávanou podmíněnou log-likelihood přes všechny možné prediktory z $\mathcal{Q}$. To znamená, že to je nejlepší možný prediktor za daného prioru Π .

Dále navíc maximalizaci výrazu $\mathbb{E}_{\Pi}[\log q(Y | X, D_n)]$ lze ekvivalentně chápat jako minimalizaci očekávané KL divergence

$$\mathbb{E} \left[\text{KL} \left(q(\cdot | X, D_n) \parallel \pi(\cdot | X, D_n) \right) \right].$$

PPD π je tedy KL-optimálním prediktorem v $\mathcal{Q}$.

Abychom PPD aproximovali v praxi, natrénujeme model q_{θ} , kde $\theta \in \Theta$ jsou parametry (např. váhy neuronové sítě). To znamená, že pro každou hodnotu θ existuje celá rodina funkcí:

$$\{q_{\theta} : (\mathcal{Y} \times \mathcal{X})^{n+1} \rightarrow [0, 1], n \in \mathbb{N}\},$$

ale závislost na n v notaci explicitně neuvádíme. KL-optimální parametry jsou definovány jako:

$$\theta^* = \arg \max_{\theta \in \Theta} \mathbb{E}_{\Pi_N} \mathbb{E}_{\Pi}[\log q_{\theta}(Y | X, D_N)], \quad (2.3)$$

kde Π_N je tzv. *size prior*, tj. rozdělení nad velikostí trénovací množiny N . Střední hodnota přes N zajišťuje, že q_{θ^*} umí napodobit celou rodinu PPD pro různá n , ne pouze její n -tý prvek. Pro model q_{θ} zpravidla neexistuje θ takové, že $q_{\theta} = \pi$ přesně. V tom případě rovnice 2.3 definuje KL-optimální aproximaci π ve třídě $\{q_{\theta} : \theta \in \Theta\}$, tj. nejlepší aproximaci, které je daná architektura schopna.

Dále lze ukázat následující teoretickou vlastnost. Střední hodnota v rovnici 2.3 se v praxi aproximuje průměrováním přes m iid datasetů, např. pomocí metody Monte Carlo. Konkrétně generujeme datasety $(Y_j, X_j) \cup D^{(j)}$ velikosti $N_j + 1$ s $N_j \sim \Pi_N$ dle postupu 2. Empirická verze optimalizačního problému pak je:

$$\hat{\theta} = \arg \max_{\theta \in \Theta} \sum_{j=1}^m \log q_{\theta}(Y_j | X_j, D^{(j)}). \quad (2.4)$$

Přesnost $\hat{\theta}$ jako aproximace θ^* závisí na počtu Monte Carlo vzorků m . Ale důležité je, že se dá ukázat $\hat{\theta} = \theta^* + O_p(m^{-1/2})$. Z toho plyne, že konvergence $\hat{\theta} \rightarrow \theta^*$ je v praxi zaručena.

Nagler [3] ukazuje, že pokud je trénovací loss NLL (negative log-likelihood, viz algoritmus 4) a model má dostatečnou kapacitu, optimální řešení konverguje k Bayesovskému posteriornímu prediktivnímu rozdělení. To znamená, že správně natrénovaný PFN je teoreticky ekvivalentní analytickému GP posterioru.

2.4.2 Od PPD k PFN

V této sekci si ukážeme podrobněji jak spolu souvisí pojmy PPD a PFN, řekneme si zajímavé vlastnosti ohledně učení PPD, které spojují tyto dva nástroje, a jako důsledek popíšeme i vlastnosti učení samotného PFN. Uvažujme definici PPD 2.1. Posterior $\Pi(\cdot | D_n)$ je plně určen priorem Π . Jde o standardní Bayesovo pravidlo aplikované na nekonečnědimenzionální parametr p . Pokud jsou data D_n generována z p_0 , doufáme, že se posterior $\Pi(p | D_n)$ s rostoucím n soustředí okolo p_0 , a tím pádem $\pi(y | x, D_n) \rightarrow p_0(y | x)$. Avšak v tomto okamžiku čelíme velmi netriviálnímu problému, a to jak zvolit správný prior, aby to konvergovalo. Obecně existují celé články a knížky o tom, jak se vypořádat s tímto problémem, který se vyskytuje v praxi tu a tam. Platí totiž, že je třeba, aby Π měl dostatečně velký support a tím pokrýval dostatečně bohatou třídu funkcí. To nám pak totiž umožní efektivní inferenci. Např. pro případ GP s RBF kernelem bychom chtěli pokrýt celý prostor všemožných tvarů té funkce, tj. aby model uměl najít správné hyperparametry v RBF kernelu. Tím pádem, když PPD dostane sadu bodů, dokáže nám říct, ze které konkrétní konfigurace GP s RBF kernelem pochází. Ale ani velký support nemusí stačit. Prior může sice p_0 obsahovat ve svém supportu, ale dávat příliš velkou hmotnost nepříznivým oblastem prostoru $\mathcal{P}$, čímž může konvergence posteriorů probíhat velmi pomalu nebo vůbec. Avšak Nagler ve svém článku ukazuje, že PPD se umí učit i v situaci, kdy p_0 leží mimo support prioru $\mathcal{P} = \{p : \Pi(p) > 0\}$. To platí, za určitých podmínek.

Věta 2.4.2. Uvažujme prior Π . Necht' dále platí:

1. Existuje jedinečné $p^* \in \mathcal{P}$ takové, že

$$p^* = \arg \min_{p \in \mathcal{P}} \text{KL}(p | p_0), \quad \text{KL}(p^* | p_0) < \infty.$$

2. Pro každé $\alpha \in (0, 1/2)$ existují množiny $B_1, \dots, B_{J(\alpha)}$ takové, že

$$\mathcal{P} \subseteq \bigcup_{j=1}^{J(\alpha)} B_j, \quad \sup_{p, p' \in B_j} H(p, p') \leq 4(\alpha^2/2)^{1/\alpha}, \quad \sum_{j=1}^{J(\alpha)} \Pi(B_j)^\alpha < \infty,$$

kde $H(p, p')$ je Hellingerova vzdálenost mezi p a p' .

Pak existuje $p^* \in \mathcal{P}$ takové, že

$$\pi(y | x, D_n) \xrightarrow{n \rightarrow \infty} p^*(y | x) \quad \text{s. j.,}$$

pro P_0 s. j. (y, x) . Navíc p^* je KL-optimální aproximace p_0 v $\mathcal{P}$:

$$p^* = \arg \min_{p \in \mathcal{P}} \text{KL}(p | p_0).$$

První podmínka v této větě požaduje, aby v supportu prioru existovala jednoznačná KL-optimální aproximace p^* pravého modelu p_0 a KL divergence p^* vůči p_0 je konečná. Druhá podmínka je technickým požadavkem na to, aby prior nepřiděloval příliš velkou hmotnost oblastem daleko od p^* , prior nesmí být příliš rozptýlený v nežádoucích oblastech supportu. Sama o sobě věta tvrdí, jak se PPD učí. Konkrétně, že se učí tím lépe, čím víc dat máme, a snaží se naučit co nejlepší aproximaci p_0 dosažitelnou v $\mathcal{P}$. Pokud $p_0 \in \mathcal{P}$ (prior obsahuje pravý model): $p^* = p_0$ a PPD konverguje přímo k p_0 . Pokud $p_0 \notin \mathcal{P}$ (prior pravý model neobsahuje): PPD stále konverguje, ale k nejbližšímu $p^* \in \mathcal{P}$ v KL smyslu. Logicky čím větší je $\mathcal{P}$, tím víc se přibližuje p^* ke skutečnému p_0 .

Toto tvrzení nám tedy úplně řeší problém hledání správného prioru, na který jsme upozorňovali na začátku sekce. My totiž nepotřebujeme perfektní prior, nýbrž aby byl „dostatečně rozumný“ ve smyslu předpokladů věty 2.4.2, a objem dat postupně ten náš odhad prioru opraví. Avšak PPD je ideální teoretický případ. Pokud bychom chtěli tyto vlastnosti použít např. u PFN, je zde několik nuancí. PFN je totiž pouze aproximací PPD, jak ukážeme dále, trénovanou na omezených datasetech, a proto tvrzení 2.4.2 nelze přímo aplikovat.

Podívejme se podrobněji na to, jak PFN aproximuje PPD. Již jsme si definovali vztah 2.4 a teď na něj budeme nahlížet jako na optimalizační problém. Na konečný výsledek $\hat{\theta}$ mají vliv čtyři faktory:

1. datový prior Π ,
2. prior velikostí vzorků Π_N ,
3. model q_θ ,
4. počet vzorků m .

Jelikož PFN je předtrénovaný model, třídu modelů $\{q_\theta : \theta \in \Theta\}$ lze považovat za fixní vůči m . Přesnost $\hat{\theta}$ jako aproximace θ^* pak plyne ze standardních výsledků empirické minimalizace vztahu: $\hat{\theta} = \theta^* + O_p(m^{-1/2})$. Tím máme hned vyřešený jeden z faktorů. Ostatní tři jsou mnohem zajímavější.

Pokud Π obsahuje pouze jednoduché modely, bude i optimální PFN q_{θ^*} produkovat jednoduché funkce (y, x) . Naopak, jednoduchá architektura $\{q_\theta : \theta \in \Theta\}$ nemůže těžit ze složitého prioru Π . Aby PFN dobře fungoval na různorodých úlohách, musí mít dostatečnou kapacitu jak architektura q_θ jako samotná, tak i prior Π .

Prior Π_N velikostí vzorků lze chápat jako regularizátor. Při tréninku (viz algoritmus 3) se vzorkují datasety $D^{(j)}$ s náhodnou velikostí $N_j \sim \Pi_N$. Definujme KL-optimální parametr pro danou velikost vzorku:

$$\theta_n^* = \arg \max_{\theta} \mathbb{E}_{\Pi}[\log q_{\theta}(Y | X, D_n)].$$

PPD $\pi(y | x, D_n)$, kterou se snažíme aproximovat, se mění s n . Jako funkce od (y, x) roste její složitost s n . Pro $n = 1$ je PPD blízká průměrnému modelu v prioru a typicky téměř konstantní. Pro velká n je stále víc a víc komplexní. Analogicky θ_n^* preferuje složitější modely s rostoucím n . Optimalizací přes náhodné velikosti N_j výsledný θ^* průměruje $\theta_{N_j}^*$. Rozdělení Π_N tak určuje, které velikosti datasetů jsou více signifikantní. To lze chápat jako regularizaci komplexity modelu. Malé N_j nutí model k jednoduchým predikcím, velké N_j k složitým.

2.4.3 Učení PFN In-Context

V předchozí sekci jsme ukázali podstatnou vlastnost PPD, a to, že se se zvětšujícím se datasetem PPD přibližuje ke skutečnému rozdělení. Následně jsme ukázali, že PFN je pouhou aproximací PPD, jejíž kvalita závisí na určitých faktorech, a nelze s jistotou tvrdit, že pro něj platí stejné vlastnosti jako pro PPD a nelze na něj přímo aplikovat větu 2.4.2. Ukazuje se, že PFN se pořád může chovat jako

PPD a zachovat její některé klíčové vlastnosti, jako např. již zmíněné tvrzení. Avšak v praxi to není tak jednoduché a jednoznačné. PFN se dokáže přiblížit k $p^*(y | x)$, a to i při relativně malém množství dat ($n \leq 1000$). Navíc se zachovává i to, že ani nemusíme pro tuto vlastnost najít perfektní prior. Přesto ale prozatím nemáme žádné teoretické důvody nebo předpoklady, aby toto byla pravda. V této sekci se pokusíme je odvodit.

V tomto okamžiku je výhodné přepnout se z Bayesovského pohledu na frekventistický. Pro libovolnou velikost datasetu n při inferenci nahlížíme na předtrénovaný model $q_\theta(y | x, \cdot)$ jako na nenatrénovaný frekventistický prediktor pro $p_0(y | x)$, což je funkce $(\mathcal{Y} \times \mathcal{X})^n \rightarrow \mathcal{P}_{\mathcal{Y}|\mathcal{X}}$, která mapuje dataset D_n na podmíněné rozdělení. Parametry θ jsou pak jen hyperparametry tohoto prediktoru, vyladěné před tréninkem. Prior Π a Π_N jsou jednoduše rozdělení nad úlohami, na kterých chceme, aby prediktor fungoval dobře. Dále rozepíšeme chybu predikce na bias a rozptyl:

$$q_\theta(y | x, D_n) - p_0(y | x) = \underbrace{q_\theta(y | x, D_n) - \mathbb{E}_{D_n \sim p_0^n}[q_\theta(y | x, D_n)]}_{\text{rozptyl}} + \underbrace{\mathbb{E}_{D_n \sim p_0^n}[q_\theta(y | x, D_n)] - p_0(y | x)}_{\text{bias}}. \quad (2.5)$$

Již jsme řekli, že empiricky chyba predikce klesá spolu se zvětšujícím se n . Takže zkusme najít které strukturální aspekty PFN by to mohly vysvětlit.

Obecně ze struktury transformerů plyne, že jsou invariantní vůči permutacím vstupních dat (tj. symetrické), což je mimochodem jednou z výhod pro architekturu PFN: dataset D_n je ze své podstaty množina, ne sekvence, takže permutační invariance je přesně ta vlastnost, kterou chceme pro i.i.d. vzorky.

Věta 2.4.3. Necht' $f : (\mathcal{Y} \times \mathcal{X})^n \rightarrow \mathcal{P}_{\mathcal{Y}|\mathcal{X}}$ je libovolný prediktor. Pak existuje jeho symetrizovaná verze $\tilde{f}$ taková, že pro každé pravděpodobnostní míry P :

$$\mathbb{E}_{D_n \sim P^n}[\tilde{f}(D_n)] = \mathbb{E}_{D_n \sim P^n}[f(D_n)], \quad \text{Var}_{D_n \sim P^n}[\tilde{f}(D_n)] \leq \text{Var}_{D_n \sim P^n}[f(D_n)]. \quad (2.6)$$

Tato věta říká, že takové symetrické prediktory jsou optimální z hlediska MSE. Toto je první opodstatnění faktu, proč chyba predikce pro PFN stále klesá.

Jsou tady i další strukturální vlastnosti prediktoru q_θ , které mají vliv na kvalitu predikce. Platí, že s větším D_n klesá vliv jednotlivých vzorků na výslednou predikci. Formálně předpokládáme, že existují $\alpha > 0$ a $L < \infty$ takové, že pro dostatečně velká n a s. j. datasety D_n, D'_n lišící se v jediném vzorku:

$$|q_\theta(y | x, D_n) - q_\theta(y | x, D'_n)| \leq Ln^{-\alpha}. \quad (2.7)$$

To nám umožňuje vyslovit tvrzení, které ukazuje, proč rozptyl během učení postupně mizí.

Věta 2.4.4. Necht' platí předpoklad 2.7, pak

$$|q_\theta(y | x, D_n) - \mathbb{E}[q_\theta(y | x, D_n)]| \lesssim n^{1/2-\alpha} \quad (2.8)$$

s vysokou pravděpodobností. Navíc pro $\alpha > 1/2$ platí $\lim_{n \rightarrow \infty} n^{1/2-\alpha} = 0$ a

$$q_\theta(y | x, D_n) - \mathbb{E}[q_\theta(y | x, D_n)] \xrightarrow{n \rightarrow \infty} 0 \quad \text{s. j.} \quad (2.9)$$

Jinými slovy rozptyl postupně mizí s počtem dat n . Tak dostáváme, že hlavním pramenem chyby při inferenci je bias, kterého se už jen tak nezbavíme.

Ze vztahu 2.5 je vidět, že bias je určen chováním poslušností $\mathbb{E}[q_\theta(y | x, D_n)]$. Je logické, že bychom mohli předpokládat (nebo očekávat), že platí i tento vztah:

$$\mathbb{E}[q_\theta(y | x, D_n)] \xrightarrow{n \rightarrow \infty} \bar{q}_\theta(y | x),$$

kde $\bar{q}_\theta(\cdot | \cdot)$ je jakýsi zprůměrovaný prediktor, jehož konkrétní architekturu neznáme. Ale co můžeme říct s jistotou, tak že chování biasu hodně závisí na oné architektuře, jak ukazuje Nagler ve svém článku [3], může růst, klesat, anebo být celou dobu konstantní. Co je avšak v našich silách, je vyslovit nutnou podmínku pro mizení biasu. Má-li prediktor $q_\theta(\cdot | \cdot, D_n)$ mizející bias na dostatečně bohaté třídě funkcí, pak nutně musí být lokální. Lokální zde znamená, že např. k $q_\theta(y | x, D_n)$ by měly přispívat (asymptoticky) pouze vzorky $(Y_i, X_i) \in D_n$ s X_i blízko x . Tyto poznatky dává dohromady následující věta.

Věta 2.4.5. Necht' $\mathcal{P}$ je třída rozdělení taková, že pro každé $p \in \mathcal{P}$:

$$\mathbb{E}_{D_n \sim p^n} [q_\theta(y | x, D_n)] \xrightarrow{n \rightarrow \infty} p(y | x). \quad (2.10)$$

Pokud platí 2.7, pak existuje posloupnost $\epsilon_n \rightarrow 0$ taková, že pro každé $\tilde{p} \in \mathcal{P}$ platí:

$$|q_\theta(y | x, D_n) - q_\theta(y | x, \tilde{D}_n)| \xrightarrow{n \rightarrow \infty} 0, \quad \text{s. j.} \quad (2.11)$$

kde $D_n = (Y_i, X_i)_{i=1}^n$ a $\tilde{D}_n = (Y'_i, X_i)_{i=1}^n$ s

$$Y'_i = \begin{cases} Y_i & \text{pokud } \|X_i - x\| \leq \epsilon_n, \\ \sim \tilde{p}(\cdot | X_i) & \text{pokud } \|X_i - x\| > \epsilon_n. \end{cases}$$

To znamená, že množina $\tilde{D}_n$ je poskládaná tak, že místo vzdálených od x bodů obsahuje náhodný šum, ale podstatné je, že nakonec ten šum (odlehle body) vůbec nemá vliv na výslednou predikci. To ovšem platí pro dostatečně bohatou množinu $\mathcal{P}$. Výsledek postrádá smysl, je-li $\mathcal{P}$ příliš malá. I pro bohatou $\mathcal{P}$ jde jen o nutnou, nikoliv postačující podmínku, konstantní prediktor $q_\theta = 1$ je lokální, ale jeho bias se s n nemění. Důležitý je ale důsledek pro bias-variance tradeoff: pokud bias mizí pro bohatou $\mathcal{P}$, prediktor může efektivně využít jen $\epsilon_n n = o(n)$ vzorků, takže podmínka 2.7 pravděpodobně neplatí pro $\alpha = 1$. Dále se ukáže, že lokalizace je kamenem úrazu pro transformer architekturu PFN. *Attention váhy* $a_j^{(h)}$ jsou výstupem SoftMax, tedy jsou vždy kladné pro všechny vzorky j , nikdy přesně nulové. Transformer tedy vždy váží nenulově i vzdálené vzorky. Přepis labelů vzdálených vzorků proto vždy predikci ovlivní, a lokalita ve smyslu věty 2.4.5 není splněna. Transformer tak garantuje mizení rozptylu, ale nikoli mizení biasu. Tato pozorování popíšeme podrobněji dále.

2.4.4 PFN transformer

Jelikož tato práce je věnovaná pohledu na PFN skrz architekturu transformer [7], dává smysl podívat se na statistické vlastnosti právě této architektury. Uvažujme tedy opět pro jednoduchost transformer s jednou vrstvou. Dataset $D_n = \{V_i\}_{i=1}^n$, kde $V_i = (Y_i, X_i) \in \{0, 1\} \times \mathbb{R}^d$, a testovací vektor $v = (0, x)$. Sít' je definována následujícími operacemi. Nejprve *attention* část:

$$a_j^{(h)} = \text{SoftMax} \left(v^\top W_q^{(h)} V_1, \dots, v^\top W_q^{(h)} V_n \right)_j,$$

$$u' = \sum_{h=1}^H \sum_{j=1}^n a_j^{(h)} W_v^{(h)} V_j.$$

Na ni navazuje dopředná síť s normalizacemi a výstupní vrstvou:

$$\begin{aligned} u &= \text{LayerNorm}(v + u'; \gamma), \\ z' &= W_{r,2} \text{ReLU}(W_{r,1}u; \gamma), \\ z &= \text{LayerNorm}(u + z'; \gamma), \\ q_\theta(\cdot \mid x, D_n) &= \text{SoftMax}(W_o z), \end{aligned}$$

kde matice vah jsou $W_q^{(h)}, W_v^{(h)} \in \mathbb{R}^{(d+1) \times (d+1)}$, $W_{r,1}, W_{r,2}^\top \in \mathbb{R}^{m \times (d+1)}$, $W_o \in \mathbb{R}^{|\mathcal{Y}| \times (d+1)}$. V tomto případě parametr θ , který jsme všude uváděli v kontextu PFN v předchozích sekcích, sbírá všechny tyto matice. To znamená, že $\theta = \{W_q^{(1)}, \dots, W_q^{(H)}, W_v^{(1)}, \dots, W_v^{(H)}, W_{r,1}, W_{r,2}, W_o, \gamma\}$ Operace SoftMax, LayerNorm a ReLU jsou definovány jako:

$$\text{SoftMax}(v) = \frac{\exp(v)}{\sum_j \exp(v_j)}, \quad \text{LayerNorm}(v; \gamma) = \gamma_1 \frac{v - \text{avg}(v)}{\|v - \text{avg}(v)\|_2 + |\gamma_2|} + \gamma_3, \quad \text{ReLU}(v) = \max(0, v),$$

kde $\exp$ a $\max$ působí po složkách. První dvě rovnice definují *attention mechanismus* s H hlavami [7]. Ze součtu $a_1^{(h)} + \dots + a_n^{(h)} = 1$ plyne, že *attention váhy* tvoří pravděpodobnostní rozdělení přes trénovací vzorky. Myšlenka je, že v každé hlavě *attention váhy* $a_j^{(h)}$ zdůrazňují konkrétní vzorky $V_j \in D_n$, konkrétně ty, které jsou „podobné“ testovacímu vektoru v ve smyslu měřeném skalárním součinem $v^\top W_q^{(h)} V_j$. Každá *attention hlava* dovoluje jinou definici podobnosti prostřednictvím matice $W_q^{(h)}$. Intuitivně to funguje takto: transformer si při predikci pro x vyhledá v trénovacích datech vzorky, které považuje za relevantní, a jejich labely agreguje. Různé hlavy mohou sledovat různé aspekty podobnosti, jedna hlava třeba srovnává hodnoty X_i s x , jiná sleduje jiný rys vstupních vektorů.

Nás by také zajímalo, jaké vlastnosti má rozptyl a bias pro tuto architekturu, protože v předchozí sekci jsme poznamenávali, že konkrétní vlastnosti těchto charakteristik se mění v závislosti na volbě konkrétní architektury.

Věta 2.4.6. Pro $\mathcal{X} = \{x : \|x\|_2 \leq K\}$ a omezené matice vah ($\|W_q^{(h)}\|_2, \|W_v^{(h)}\|_2, \|W_{r,1}\|_2, \|W_{r,2}\|_2, \|W_o\|_2 < \infty$) platí

$$|q_\theta(y \mid x, D_n) - \mathbb{E}[q_\theta(y \mid x, D_n)]| \lesssim n^{-1/2} \quad (2.12)$$

s vysokou pravděpodobností.

Velmi podobnou větu jsme již viděli v předchozí sekci. Dokonce tvrdí skoro stejné: rozptyl mizí nezávisle na parametru θ . Jde o strukturální vlastnost architektury, nikoliv výsledek tréninku. Důvodem je, že *attention mechanismus* nutně přiděluje každému vzorku váhu řádu $1/n$, takže vliv libovolného jednotlivého vzorku klesá s n . V případě biasu jsme již říkali, že ta situace je horší. Bias v případě transformeru totiž silně závisí na volbě θ . Označme $\bar{q}_\theta(y \mid x)$ limitní hodnotu predikce pro $n \rightarrow \infty$.

Věta 2.4.7. Za předpokladů platnosti věty 2.4.6 plyne

$$\mathbb{E}[q_\theta(\cdot \mid x, D_n)] \xrightarrow{n \rightarrow \infty} \bar{q}_\theta(\cdot \mid x), \quad (2.13)$$

kde $\bar{q}_\theta(\cdot \mid x)$ je definováno stejně jako $q_\theta(\cdot \mid x, D_n)$, ale s u' nahrazeným výrazem $\bar{u}' = \sum_{h=1}^H W_v^{(h)} \mathbb{E}_{V \sim g_h}[V]$,

kde $g_h(s) = \frac{\exp(v^\top W_q^{(h)} s)}{\mathbb{E}_{V \sim p_0}[\exp(v^\top W_q^{(h)} V)]} p_0(s)$.

To znamená, že v definici transformeru výše jsme nahradili výraz u' tím novým $\hat{u}'$. Navíc v definici $\hat{u}'$ se objevuje tzv. *exponentially tilted function* (česky exponenciálně vychýlená funkce). Jde o standardní transformaci z teorie pravděpodobnosti. Původní rozdělení $p_0(s)$ se přenásobí exponenciálním faktorem $\exp(v^\top W_q^{(h)} s)$ a znormuje, čímž se jeho pravděpodobnostní hmota přesune k hodnotám s s velkým skalárním součinem $v^\top W_q^{(h)} s$. V tomto kontextu se objevuje proto, že přesně takto se chovají *attention váhy* v limitě nekonečně mnoha vzorků, je tedy asymptotickou variantou *attention mechanismu*. Tato funkce $g_h(s)$ zvyšuje pravděpodobnost těm hodnotám s , které se nejvíce podobají vektoru v ve smyslu $v^\top W_q^{(h)} s$, a snižuje pro zbylé body. Jinými slovy ten složitý vzoreček výše ukazuje teoretickou situaci, co by se stalo, kdyby měl transformer k dispozici nekonečně mnoho vzorků. Už nerozděluje váhy mezi jednotlivými datovými body, ale vezme celé spojitě rozdělení dat a „vychýlí“ je celé směrem k té informaci, na kterou zrovna potřebuje upřít pozornost. Více formálně to znamená, že každá *attention hlava* h tak hodnotí určitý aspekt neznámého rozdělení p_0 (charakterizovaný maticí $W_q^{(h)}$). Pokud jsou matice $(W_q^{(h)})_{h=1}^H$ dobře nastaveny, tyto jednotlivé pohledy rozlišují různé hodnoty příznaků.

Avšak g_h lokalizuje prediktor pouze do jisté míry. Zvyšuje váhu vzorků podobných v , ale ne ve smyslu věty 2.4.5. Vzorky vzdálené od x stále nenulově přispívají, protože SoftMax nikdy nevytváří přesně nulové váhy. Přepsání labelů vzdálených vzorků proto vždy predikci změní, a tak lokalita podle 2.4.5 není splněna. Proto plyne, že bias transformeru obecně nemůže úplně zmizet. Limitní bias $\bar{q}_\theta(y | x) - p_0(y | x)$ přesto může být malý, pokud síť za *attention vrstvou* dokáže ze součtu aspektových souhrnů $W_v^{(h)} \mathbb{E}_{V \sim g_h}[V]$ zkonstruovat dobrou aproximaci p_0 . Relevance jednotlivých aspektů závisí na pravém rozdělení p_0 . Méně relevantní aspekty mohou přispívat méně. Na malých vzorcích ($n = 1$) jsou všechny *attention váhy* $a_j^{(h)} = 1$, takže všechny aspekty přispívají stejnou měrou. To naznačuje, že bias může klesat v rozsahu velikostí, na které byl θ natrénován, nikoliv však za jejich hranicí, kde není důvod očekávat pokles biasu. Klíčovou roli hraje přítomnost více *attention hlav* ($H > 1$), spolupráce více hlav může efektivně napodobit mechanismus průměrování modelů. Dalším klíčovým nástrojem je větší počet *attention vrstev*. Navíc empirické testy ukazují, že na výslednou kvalitu predikce (a zmenšování biasu) hraje neposlední roli samotný trénink a jeho nastavení. Všechny tyto manipulační součástky tak dávají velký prostor pro manévr a různé jejich kombinace či nastavení mohou vést k velmi signifikantnímu poklesu biasu.

Dále lze různě upravovat strukturu PFN během inference či tréninku pro zlepšení lokality. Naším účelem je tedy nějakým způsobem donutit transformer se dívat na jednotlivé body co nejvíc lokálně a neuvažovat pro bod x příliš vzdálené hodnoty. Jedním z přímočarých způsobů je *post-hoc lokalizace*, kterou navrhuje Nagler. Pro predikci labelu v testovacím bodě x :

1. Sestrojí se redukováná trénovací množina $D_n(x)$ odstraněním všech vzorků kromě k_n nejbližších sousedů bodu x z D_n .
2. Predikuje se label maximalizující $q_\theta(\cdot | x, D_n(x))$.

Omezení na okolí x je ekvivalentní „roztažení“ cílové funkce $p(y | \cdot)$ v okolí x , lokálně hladká funkce se chová přibližně jako konstantní. Konstantní funkce jsou pro q_θ snazší k aproximaci. Lokalizace tedy zlepšuje bias za cenu mírného zvýšení rozptylu.

Kapitola 3

Experimentální průzkum vlastností PFN

V této kapitole provedeme řadu experimentů, ve kterých budeme zkoumat a ověřovat různé hypotézy o tom, jak funguje PFN. Nagler se ve své práci snaží matematicky popsat principy PFN. Uvažuje však model s jednou vrstvou a to, co PFN transformer ve výsledku dělá a jak fungují jeho vnitřní mechanismy, popisuje jen částečně. Proto se pokusíme tyto otevřené otázky prozkoumat, když ne matematickým aparátem, tak alespoň empirickými pozorováními. Všechny experimenty provedeme jen pro architekturu typu transformer. Zajímat nás bude jak vnitřní struktura PFN, tak i některé jeho vlastnosti jako nástroje pro Bayesovskou inferenci. Rovněž prozkoumáme mechanismy, kterými PFN vyhodnocuje vstupní data, a ukážeme, čím se tyto mechanismy liší od regrese pomocí Gaussovských procesů. Veškerý kód všech experimentů, natrénované modely i návod k jejich spuštění jsou k dispozici ve veřejném repozitáři¹.

3.1 Vnitřní struktura PFN transformeru a jeho základní principy

Nejdříve uvedeme základní analýzu PFN transformeru. Zaměříme se zde na to, co přesně dělá se vstupními daty. Podíváme se, zda se snaží napodobit stejné mechanismy jako GP, anebo zda dělá něco jiného. Experimenty jsme prováděli na dvou typech modelů. Oba sdílejí stejnou architekturu transformeru: 6 vrstev, 8 *attention* hlav, dimenze embeddingu 512, dimenze skrytých vrstev dopředné sítě 1024 a výstupní `FullSupportBarDistribution` s 1000 biny. Architektura je nastavena s parametry `features_per_group=1` a `attention_between_features=False`, tedy přesně tak, jak jsme ji popsali v předchozí kapitole pro případ 1D regrese. Oba modely byly trénovány na syntetických datech vygenerovaných z Gaussovského procesu s RBF kernelem, vstupní prostor $x \in [0, 1]$. Liší se však tím, z jakého prioru nad hyperparametry trénovací data pocházela. S tím souvisí i délka tréninku, pro nefixní hyperparametry totiž model potřebuje více času na učení.

Model se fixními hyperparametry:

Tento model byl trénován na GP s pevně nastavenými hyperparametry RBF kernelu: $\ell = 0,3$, `outputscale = 1,0`, $\sigma^2 = 10^{-4}$. Model se tedy učil aproximovat posterior jednoho konkrétního GP a trénink konvergoval za 100 epoch.

Model s náhodně vzorkovanými hyperparametry:

Model byl trénován na GP, jehož hyperparametry byly při každé dávce vzorkovány nezávisle z předem daného rozdělení: $\ell \sim \mathcal{U}(0,05, 1,0)$, `outputscale` $\sim \text{LogNormal}(0, 0,5)$, $\sigma^2 \sim 10^{\mathcal{U}(-3, -1)}$. Tento model se tedy musí naučit generalizovat přes celou rodinu GP funkcí s různými korelačními délkami a amplitudami. Jde tedy o plnohodnotný *meta-learning* nad rozdělením hyperparametrů. Právě proto vy-

¹<https://github.com/gulierus/Transformers-for-Bayesian-statistics>

žaduje podstatně delší trénink. Model viděl nespočet různých tvarů funkcí a musel se naučit rozpoznávat hyperparametry přímo z kontextových dat. Trénink trval 500 epoch.

Výsledky obou modelů porovnáváme v průběhu jednotlivých experimentů s referenčním analytickým GP posteriorem, který pro dané hyperparametry a kontextová data vrací přesné řešení. Začneme ovšem případem fixních hyperparametrů. Pro model to je značné zjednodušení, přesně ví, co se má naučit. Jak se ukáže dále, naučí se to velmi efektivně. Na tomto jednoduchém příkladě ukážeme hlavní principy inference této metody, abychom následně mohli přejít na těžší příklady. To znamená, že model s náhodně vzorkovanými hyperparametry budeme zkoumat v další sekci.

3.1.1 Struktura attention matic

Začneme tím, že si vykreslíme jednotlivé vrstvy. Podíváme se tak dovnitř transformeru na to, jak jeho vrstvami prochází tok dat. *Attention matice* je přirozené okno do toho, jak transformer interně organizuje informaci. Připomeňme, že v naší konfiguraci má model 6 vrstev a v každé vrstvě 8 *attention hlav*, přičemž každá hlava může sledovat jiný aspekt vstupních dat. Vizualizaci provedeme pro náš fixní model na sekvenci délky 120 bodů: 20 trénovacích bodů a 100 testovacích bodů. Výsledkem je *attention matice* tvaru 120×120 , přičemž každý prvek a_{ij} říká, jak moc se bod i (jako *query*) dívá na bod j (jako *key*). Každá vrstva přitom neobsahuje jen jednu takovou matici, ale 8, tj. jednu pro každou *attention hlavu*, protože každá hlava má vlastní naučené váhy. Abychom nemuseli sledovat všech 8 matic zvlášť (pro 6 vrstev bychom obdrželi 48 matic), sečteme je po prvcích a vydělíme osmi, čímž z nich uděláme jedinou průměrnou matici pro každou vrstvu. Prvek a_{ij} v ní pak vyjadřuje, jak moc se bod i dívá na bod j v průměru přes všech 8 hlav.

Protože víme, které pozice odpovídají trénovacím bodům (0–19) a které testovacím (20–119), lze matici přirozeně rozdělit na čtyři kvadranty:

	Key: Train	Key: Test
Query: Train	Train→Train	Train→Test
Query: Test	Test→Train	Test→Test

Kvadrant *Test→Train* je pro nás klíčový, ten nám totiž říká, jak moc testovací body „čerpají“ informaci z trénovacích dat při budování své predikce. Z pohledu GP inference bychom očekávali, že právě tento kvadrant bude dominantní. Predikce v novém bodě x^* totiž závisí na pozorovaných párech (X, y) , nikoli naopak.

Na obrázku 3.1 je vidět, jak se struktura *attention matic* vyvíjí přes vrstvy:

Vrstva 0:

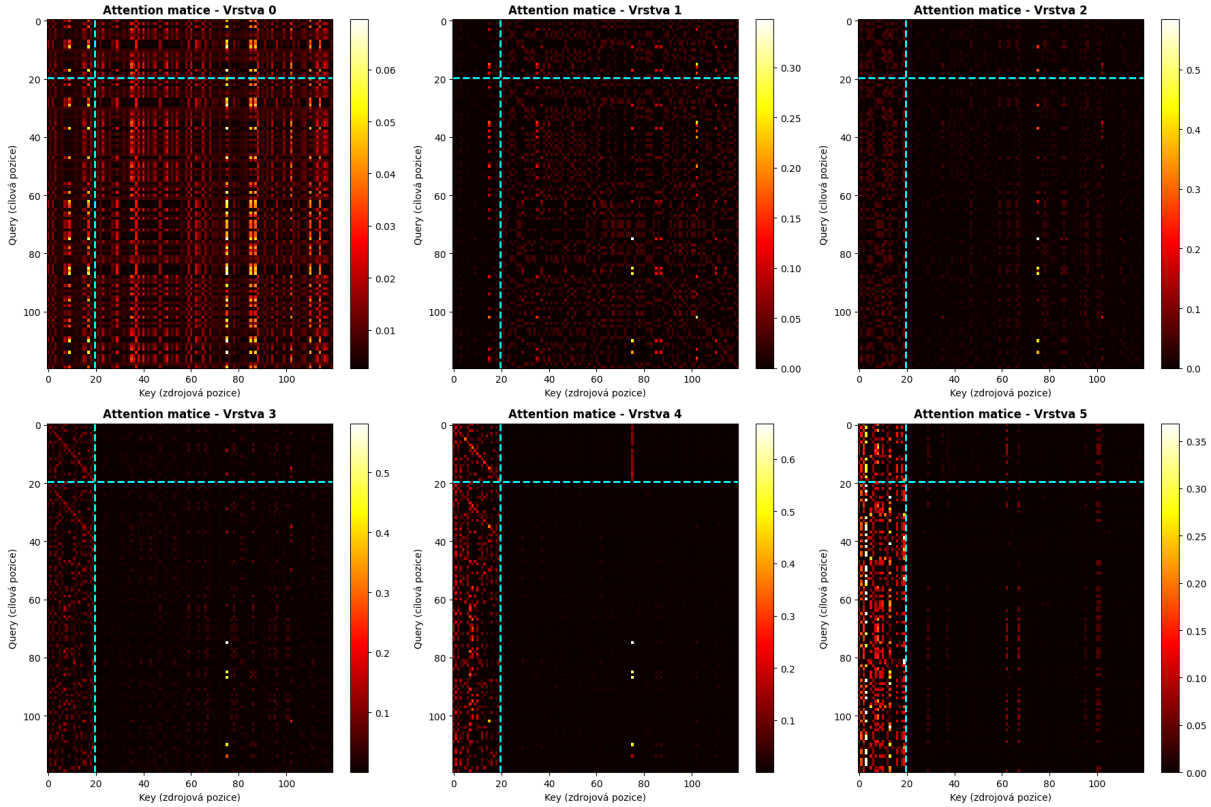
Vyazuje relativně rovnoměrně rozložené váhy. Žádný kvadrant výrazně nedominuje a váhy jsou rozloženy plošně přes všechny pozice. Tuto vrstvu můžeme interpretovat jako explorativní fázi (neboli poznávací), tzn. model sbírá globální informaci o rozložení všech bodů v sekvenci a podrobněji jejich vzájemné vazby nezkoumá.

Vrstvy 1–4:

Attention postupně řídne, váhy se soustředí na méně pozic a zbytek klesá k nule. Lze zde pozorovat tvorbu určitých struktur ve vzorech. Kvadrant *Test→Train* začíná nabývat na intenzitě, zatímco *Train→Test* slábne. Tyto prostřední vrstvy zřejmě slouží k internímu zpracování informace, tj. postupnému sestavování reprezentace, ze které bude moci poslední vrstva provést samotnou inferenci.

Vrstva 5:

Tato poslední vrstva již vykazuje velmi zřetelný vzor. Kvadrant *Test→Train* je jasně dominantní a strukturovaný, zatímco kvadrant *Train→Test* je skoro nulový. Tato asymetrie je přesně to, co bychom čekali od GP inference, testovací body jsou podmíněny trénovacími daty, nikoli naopak.



Obrázek 3.1: *Attention matice* pro všech 6 vrstev fixního modelu (průměr přes 8 hlav) pro jednu konkrétní GP realizaci. Osa x : *key* pozice (na které body se díváme), osa y : *query* pozice (které body se dívají). Cyánová čára odděluje trénovací (0–19) a testovací (20–119) pozice. Světlejší barva odpovídá vyšší *attention váze*. Přesné hodnoty vah závisí na konkrétních vstupních datech, kvalitativní vzor (vrstva 0 distribuovaná → vrstva 5 s dominantním *Test*→*Train* blokem) je však stabilní.

Podstatné je, že tuto asymetrii model neměl explicitně zakódovanou, a tudíž vyplynula přirozeně z optimalizace na syntetických GP datech. Kdybychom totiž model trénovali na datech bez této struktury, *attention mechanismus* by zůstal symetrický. Samotná existence asymetrie v poslední vrstvě je tak prvním experimentálním dokladem, že PFN provádí Bayesovskou inferenci, nikoli pouhý *pattern matching*, neboli učení příkladů nazpaměť.

Na maticích na obrázku 3.1 si lze všimnout ještě jednoho nápadného jevu, konkrétně svislých pruhů. Některé *key* pozice (sloupce) dostávají vysoké *attention* od téměř všech *query* pozic, nezávisle na tom, kde daný *query* bod leží. Ve vrstvě 0 jsou nejostřejší, v hlubších vrstvách obvykle přetrvává jeden reziduální pruh (v naší realizaci nápadně kolem *key* pozice 75). Jde o tzv. *attention sink*, mechanismus dobře popsany u velkých jazykových modelů [8]. Vzniká ze dvou důvodů. Za prvé, softmax nutí každý řádek matice, aby se sečetl na jedničku. Každý *query* bod tedy musí své *attention* někam rozdělit, i když pro něj žádný klíč není skutečně relevantní. Typicky se to stává v raných vrstvách, než se vybudují informativní reprezentace, nebo u bodů daleko od dat. Přebytek *attention* se pak přerozdělí na několik stále stejných pozic, které vytvoří svislé pruhy. Za druhé, body s netypickou hodnotou (extrémní Y_j nebo okrajové X_j) dostanou po zakódování větší embedding, a tím i vyšší skóre od všech ostatních bodů. Proto přitahují *attention* bez ohledu na to, odkud se *query* dívá.

Pro naši otázku je podstatné, že tyto pruhy jsou tzv. *query*-nezávislé. To znamená, že stejný *key* bod přitahuje vysoké *attention* od všech *query* pozic stejně a bez ohledu na to, který bod zrovna predikujeme.

RBF estimátor by je nikdy nevytvořil, protože jeho váhy jsou lokální a závisejí na poloze *query* bodu x^* . Svislý pruh naopak znamená „všechny *query* se dívají na stejný bod bez ohledu na svou polohu“. Část *attention rozpočtu* tedy jde na *normalizační sink*, nikoli na *kernelové porovnávání*. Dá se tedy říct, že existence pruhů je dalším nepřímým dokladem, že PFN *attention* není prostý *kernel smoothing*.

3.1.2 Matematický rámec pro srovnávací experimenty

Než přistoupíme k samotným experimentům, je užitečné shrnout matematický aparát, který budeme v dalších sekcích potřebovat. Jde konkrétně o tři formule, jejichž vzájemný vztah je jádrem naší analýzy.

Posteriorní střední hodnota GP:

Po pozorování kontextových párů (X, y) je posteriorní střední hodnota Gaussovského procesu v novém bodě x^* dána vztahem [4]:

$$\mu(x^*) = k(x^*, \mathbf{X})[\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I}]^{-1} \mathbf{y},$$

kde $\mathbf{K}(\mathbf{X}, \mathbf{X})_{ij} = k(x_i, x_j)$ je kernelová matice a σ^2 je rozptyl šumu. Klíčovou roli hraje inverze $[\mathbf{K} + \sigma^2 \mathbf{I}]^{-1}$, která dekoreluje blízké trénovací body. Body, které si jsou podobné, si vzájemně „konkurují“ a jejich kumulativní vliv je potlačen.

Nadaraya-Watsonův estimátor:

Nejjednodušší alternativou, jak odhadnout GP, je Nadaraya-Watsonův (NW) estimátor [1], tj. normalizovaný vážený průměr trénovacích hodnot:

$$\hat{y}_{\text{NW}}(x^*) = \frac{\sum_i k(x^*, x_i) y_i}{\sum_i k(x^*, x_i)}.$$

NW se od GP liší právě absencí inverze kernelové matice. Tento model je příliš velkým zjednodušením, pokud bychom ho chtěli použít pro odhad GP. NW totiž implicitně předpokládá, že trénovací body jsou vzájemně nekorelované. Blízké body proto dostávají neúměrně velkou kumulativní váhu a predikce jsou pak příliš hladké. Současně je ale velmi přirozenou alternativou, jak GP odhadovat. Právě proto dále budeme zkoumat, zda se transformer nenaučil jen tuto jednoduchou cestu.

Attention mechanismus transformeru:

Tento mechanismus jsme již zmiňovali v sekci 2.3, kde jsme popisovali vnitřní strukturu PFN transformeru. *Attention* v PFN počítá pro každý testovací bod x^* váhovanou kombinaci hodnot z trénovacích bodů:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}, \quad \mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V.$$

Výsledná kernelová matice závisí na datech:

$$\text{Kernel}_{\text{attn}}(\mathbf{X}) = \text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top}{\sqrt{d_k}}\right).$$

Na rozdíl od fixního RBF kernelu se tato matice plně adaptuje na konkrétní vstupní sekvenci. Matice vah $\mathbf{W}_Q, \mathbf{W}_K$ jsou naučeny z dat a mohou kódovat libovolnou strategii podobnosti.

Klíčová otázka, kterou budeme v následujících experimentech testovat, pak zní: odpovídají *attention váhy* PFN normalizovaným RBF vahám, nebo se model naučil odlišnou strategii? A pokud odlišnou, jak moc se jeho výstup blíží skutečnému GP posterioru oproti triviálnímu NW estimátoru?

3.1.3 Srovnání s RBF kernelem a Nadaraya-Watsonovým estimátorem

V této sekci se zaměříme na to, zda se PFN transformer snaží napodobit chování RBF kernelu, a proto GP tak dobře aproximuje, anebo zda dělá něco jiného. Konkrétně pro každý testovací bod x^* porovnáme *attention váhy* a_i z poslední vrstvy (průměr přes 8 hlav) s normalizovanými RBF vahami:

$$w_i = \frac{k(x^*, x_i)}{\sum_j k(x^*, x_j)}.$$

Experiment provedeme pro fixní model na $n_{\text{ctx}} = 20$ trénovacích bodech a 100 testovacích bodech, průměrem přes 5 nezávislých *seedů*.

Každá metrika je průměrována přes 5 nezávislých *seedů*, přičemž pro každý *seed* je vygenerována jedna GP sekvence délky 100 (prvních 20 bodů jako kontext, všech 100 jako testovací body). MSE je počítáno jako střední kvadratická odchylka mezi střední hodnotou predikce PFN a analytickým GP posteriorem přes všech 100 testovacích bodů. Korelace *attention* vs. RBF je pro každý testovací bod x^* spočítána jako Pearsonova korelace mezi *attention vahami* poslední vrstvy (průměr přes 8 hlav) a normalizovanými RBF vahami $\exp(-d^2/2\ell^2)$, zprůměrovaná přes všechny testovací body. Jako referenční metodu zahrnujeme rovněž Nadaraya-Watsonův estimátor: MSE(NW, GP) měří jeho odchylku od analytického GP posterioru a slouží jako dolní mez kvality nejjednoduššího *kernel smoothingu*. MSE(PFN, NW) pak kvantifikuje, o kolik se predikce PFN od NW liší. Je-li současně $\text{MSE}(\text{PFN}, \text{GP}) \approx 0$, tedy PFN sedí na GP posteriorem, a přitom $\text{MSE}(\text{PFN}, \text{NW}) \approx \text{MSE}(\text{NW}, \text{GP})$, znamená to, že PFN a NW predikují zcela odlišné hodnoty, a tedy že PFN neprovádí prostý *kernel smoothing*. Naměřené hodnoty jsou shrnuty v tabulce 3.1.

Metrika	Fixní model
Průměrná korelace Attn vs RBF	$0,374 \pm 0,270$
MSE(PFN, GP)	$0,000167 \pm 0,000096$
MSE(NW, GP)	$0,2099 \pm 0,1462$
MSE(PFN, NW)	$0,2092 \pm 0,1439$
Poměr MSE(NW)/MSE(PFN)	$\sim 1\,256\times$

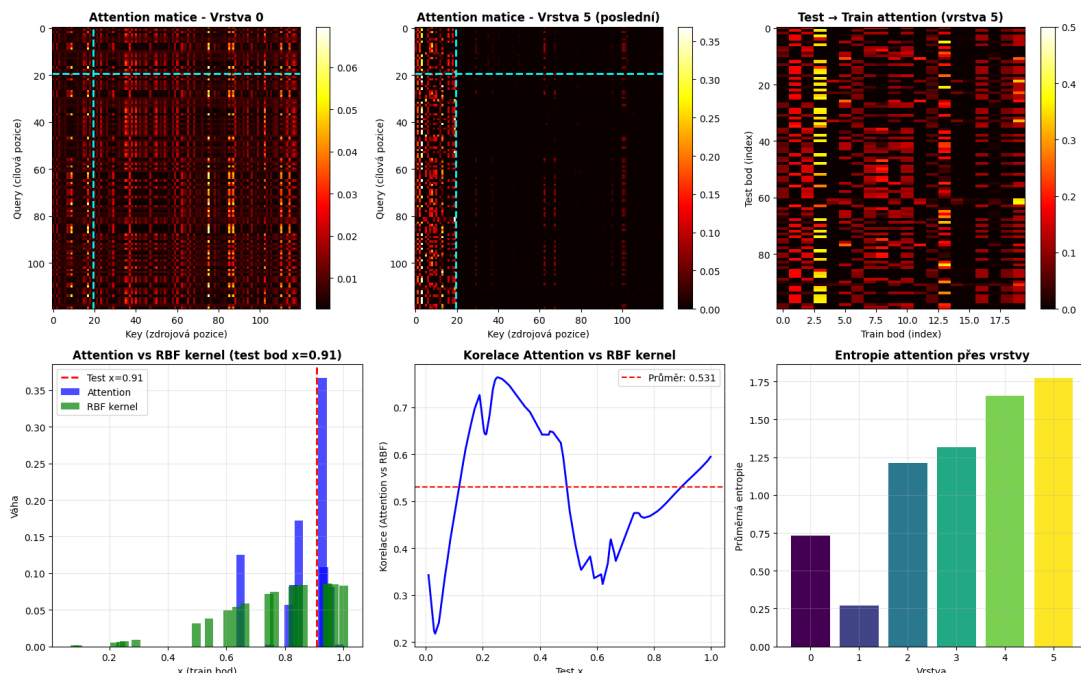
Tabulka 3.1: Výsledky korelační analýzy *attention vah* vs. RBF kernelu a srovnání přesnosti predikcí. Průměr přes 5 *seedů*.

Diskuze:

Z tabulky 3.1 vyplývají dvě zásadní pozorování. Za prvé, korelace *attention vah* s RBF kernelem je nízká ($0,374 \pm 0,270$) a má vysoký rozptyl. Pouze pro některé realizace je ta korelace větší než 0,5, a to stále nepovažujeme za dostatečné, abychom mohli říct, že kopírujeme chování RBF. Tímto usuzujeme, že model neprovádí prostý RBF *kernel smoothing*. Také je zjevné, že *attention mechanismus* se naučil jinou strategii. Jak je vidět na obrázku 3.2, *attention váhy* jsou výrazně ostřejší než RBF, soustředí ují téměř veškerou váhu na nejbližší bod, zatímco RBF ji distribuuje plynule.

Za druhé, $\text{MSE}(\text{PFN}, \text{NW}) \approx \text{MSE}(\text{NW}, \text{GP}) \approx 0,21$, vzdálenost mezi PFN a NW je přibližně stejně velká jako vzdálenost NW od GP. To znamená, že PFN sedí blízko GP posterioru, zatímco NW stojí daleko od obou, jak názorně ukazuje obrázek 3.3. Obě tyto metody predikují zcela odlišné hodnoty. Dále je vidět, že je PFN 1 256× přesnější než NW (poměr $\text{MSE}(\text{NW}, \text{GP})/\text{MSE}(\text{PFN}, \text{GP}) = 0,2099/0,000167$).

Nás by ale mohlo zajímat, proč je *attention* ostřejší než RBF, tj. proč *attention váhy* nejsou rozloženy tak hladce a plynule jako RBF váhy na obrázku 3.2. *Attention hlavy* totiž nepotřebují počítat čistou kernelovou podobnost $k(x^*, x_i)$. Potřebují vypočítat to, co v kombinaci s ostatními hlavami a dopřednou sítí (*feed-forward network*, kterou budeme v celé práci označovat zkratkou FFN) dá nejlepší aproximaci



Obrázek 3.2: Detailní rozbor *attention* fixního modelu (průměr přes 8 hlav). Nahoře: *attention matice* vrstvy 0 a poslední vrstvy 5 a výřez bloku *Test*→*Train* poslední vrstvy. Dole vlevo: *attention váhy* pro vybraný testovací bod v porovnání s normalizovanými RBF vahami, *attention* je výrazně ostřejší, koncentruje téměř veškerou váhu na nejbližší bod, zatímco RBF distribuuje váhu plynule. Dole uprostřed: korelace *attention* s RBF jako funkce polohy testovacího bodu. Dole vpravo: průměrná entropie *attention vah* po vrstvách.

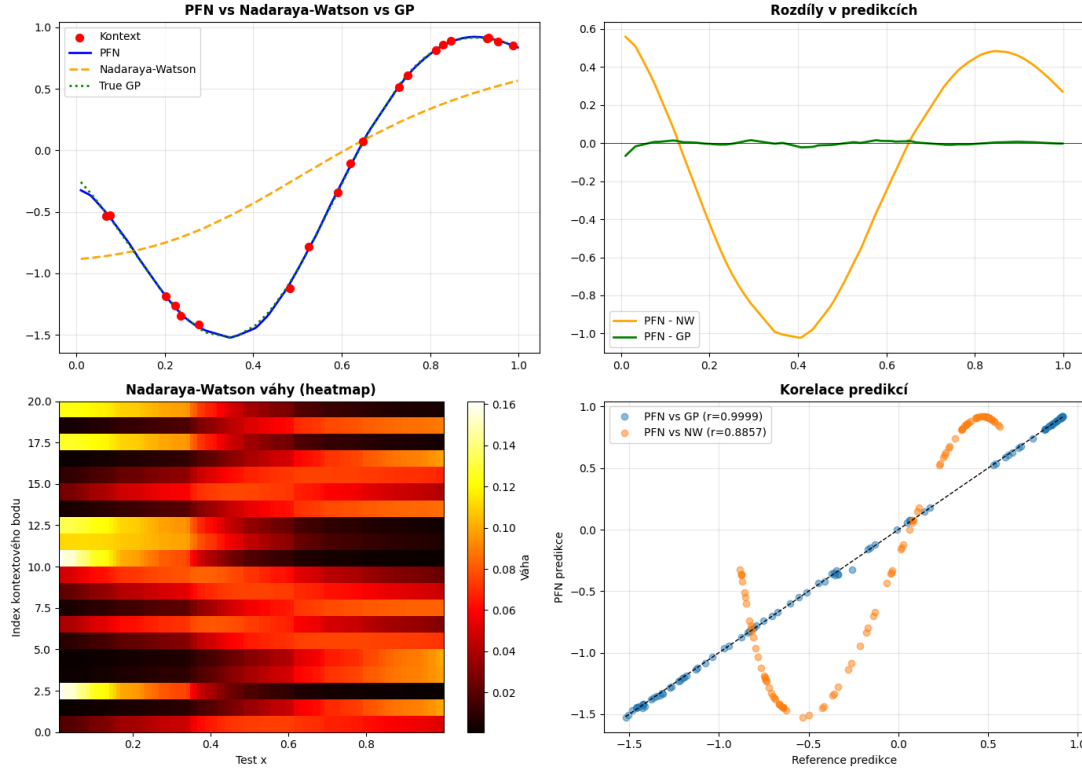
GP posterioru. A pro ten může být výhodnější jiná strategie, kde se bod dívá na sousedy s jinou silou. Dokonce se ukazuje, že se může dívat na vzdálenější body víc než na své blízké sousedy. Na konci pak FFN snáze provede korekci odpovídající efektu $\mathbf{K}^{-1}$.

Metodologická poznámka k této analýze:

Nízká korelace *attention vah* s RBF kernelem je platný a hodnotný závěr o chování jedné komponenty sítě. Existují však důvody, proč z ní nelze přímo usuzovat na celkové chování modelu.

Za prvé, *attention* není výstupní mechanismus, po něm následuje FFN, reziduální spojení a v případě vícevrstvého modelu dalších pět bloků. I kdyby *attention váhy* v poslední vrstvě přesně odpovídaly normalizovaným RBF vahám, výstup PFN by se od NW stále lišil vlivem korekcí ve FFN.

Za druhé, GP na rozdíl od RBF neváží trénovací body jen podle jejich podobnosti s x^* , ale přes inverzi $\mathbf{K}^{-1}$ zohledňuje i to, jak jsou body korelované mezi sebou (blízké body si „konkurují“, vzdálené mohou dostat i zápornou váhu). Nízká korelace *attention* s čistým RBF je tedy očekávaná, i perfektní GP by ji při tomto srovnání vykazoval.



Obrázek 3.3: Srovnání predikcí PFN, Nadaraya-Watsonova estimátoru a analytického GP posterioru ($n_{\text{ctx}} = 20$). Nahoře vlevo: predikce všech tří metod na jedné realizaci. Nahoře vpravo: rozdíly predikcí PFN–NW a PFN–GP. Dole vlevo: NW váhy jako teplotní mapa (kontextový bod $\times$ testovací poloha). Dole vpravo: korelace predikcí PFN s GP a s NW. NW systematicky přehlazuje, zatímco PFN sleduje GP posterior těsně, přesné hodnoty MSE shrnuje tabulka 3.1.

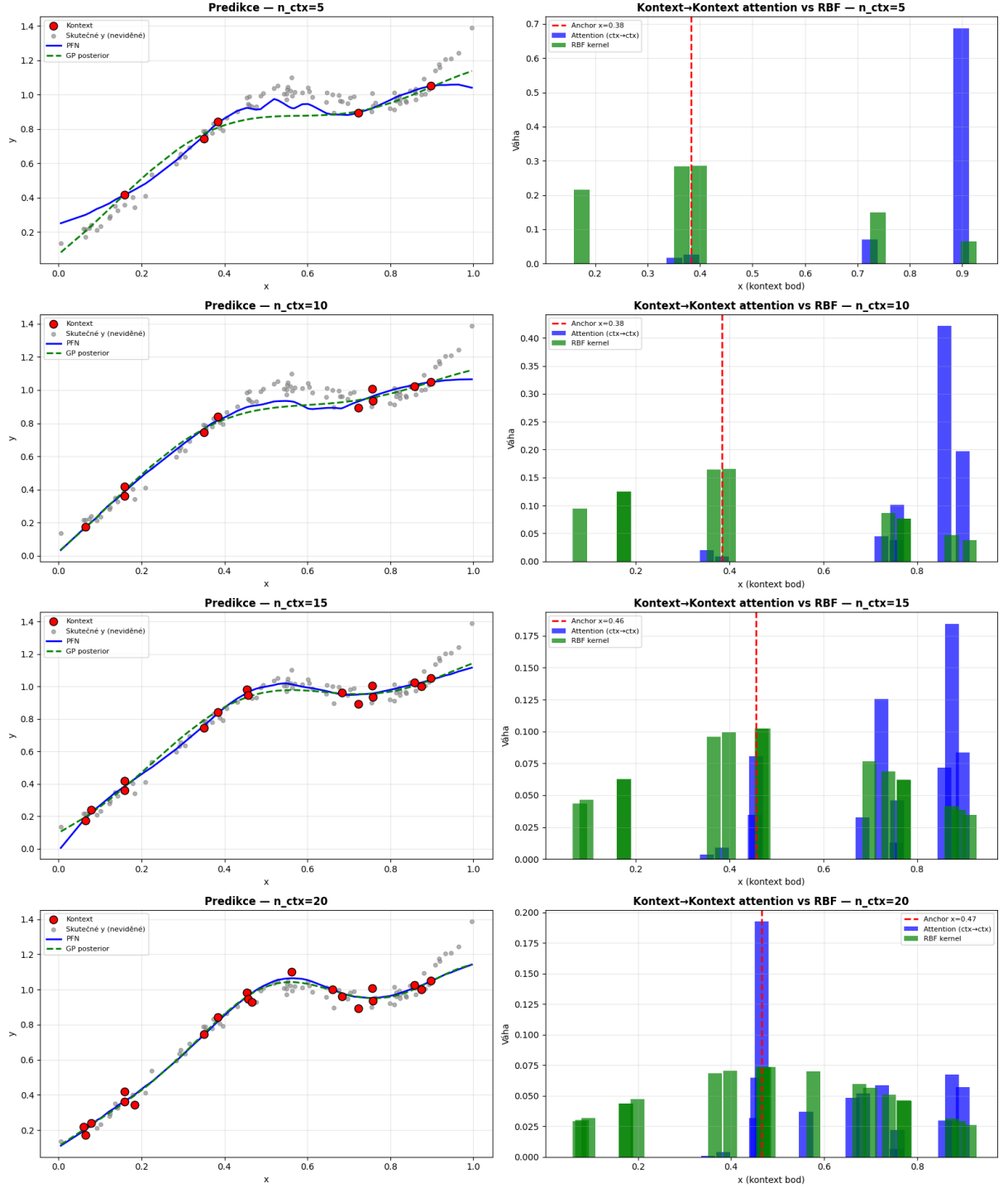
3.1.4 Přesnost predikcí na skutečné GP realizaci

Předchozí experiment porovnával PFN se syntetickým GP posteriorem, tj. modelu jsme předložili trénovací body a srovnávali jsme jeho predikci s analytickým řešením. V následující části přistoupíme k přísnějšímu testu. Oba modely (PFN i *GP oracle*) budou predikovat skutečné šumové hodnoty GP realizace, kterou neviděly. Cílem je zjistit, jak se přesnost PFN vyvíjí s velikostí kontextu $n_{\text{ctx}} \in \{5, 10, 15, 20\}$.

Nejprve vygenerujeme kompletní GP realizaci délky 100 bodů, tedy jednu konkrétní náhodnou funkci vzorkovanou z GP, u níž pro každý z 100 vstupů x známe jeho skutečnou hodnotu y s šumem. Z těchto 100 bodů náhodně vybereme n_{ctx} jako trénovací kontext, tj. dvojice (x, y) , které oběma metodám ukážeme jako pozorovaná data. Zbývajících $100 - n_{\text{ctx}}$ bodů necháme skryté a slouží jako testovací cíl, jejich x předložíme modelu k predikci, ale jejich pravé y -hodnoty mu neprozradíme.

Obě metody nyní pro každý testovací bod vrátí svou predikci $\hat{y}(x)$. PFN ji spočítá jedním dopředným průchodem z kontextu. *GP oracle* je referenční, „ideální“ Gaussovský proces, který má oproti PFN výhodu, že zná pravé hyperparametry, ze kterých byla realizace vygenerována ($\ell = 0,3$, $\sigma^2 = 10^{-4}$), a z kontextu spočítá přesný analytický posterior podle vzorce $\mu(x^*) = k(x^*, \mathbf{X})[\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I}]^{-1} \mathbf{y}$. Slouží tedy jako horní hranice toho, čeho lze na daných datech dosáhnout, se kterou PFN porovnáваме.

Kvalitu predikce měříme pomocí MSE zprůměrované přes všechny testovací body dané realizace. Aby výsledek nezávisel na jedné konkrétní realizaci, celý postup zopakujeme a metriky nakonec zprůměrujeme přes 5 *seedů* $\times$ 20 realizací = 100 instancí pro každou hodnotu n_{ctx} .



Obrázek 3.4: Vizualizace predikcí (levý sloupec) a *attention* vs RBF (pravý sloupec) pro $n \in \{5, 10, 15, 20\}$. Červené body: kontextová data dostupná modelu, šedé body: skutečné neviděné hodnoty, modrá: PFN, zelená přerušovaná: GP posterior se správným ℓ . Všechny řádky vychází ze stejné GP realizace, liší se pouze velikostí kontextu. Od $n = 15$ obě křivky téměř splývají.

n_{ctx}	PFN	GP oracle
5	$0,0408 \pm 0,0698$	$0,0340 \pm 0,0635$
10	$0,0123 \pm 0,0349$	$0,0112 \pm 0,0360$
15	$0,0064 \pm 0,0271$	$0,0028 \pm 0,0036$
20	$0,0021 \pm 0,0025$	$0,0017 \pm 0,0011$

Tabulka 3.2: MSE vůči skutečným hodnotám GP realizace jako funkce velikosti kontextu. Průměr přes 100 instancí ($5 \text{ seedů} \times 20 \text{ realizací}$).

Diskuze:

Z tabulky 3.2 a obrázku 3.4 vyplývají tři pozorování. Za prvé, *GP oracle* konzistentně dosahuje nižšího nebo srovnatelného MSE oproti PFN ve všech hodnotách n_{ctx} . To je očekávané, *oracle* zná přesné hyperparametry a provádí exaktní Bayesovskou inferenci, zatímco PFN je pouze aproximace.

Za druhé, rozdíl se zvětšuje u středních hodnot n_{ctx} . Při $n = 5$ je převaha *GP oracle* malá ($0,034$ vs $0,041$), při $n = 15$ již GP dosahuje $0,003$ oproti $0,006$ u PFN. Příčinou je, že s více kontextovými body může *GP oracle* využít přesnou inverzi kernelové matice $\mathbf{K}^{-1}$, zatímco PFN tuto operaci pouze aproximuje. Při malém kontextu je $\mathbf{K}^{-1}$ málo informativní a obě metody jsou si blízko.

Za třetí, od $n = 20$ se obě metody přibližují, jejich MSE ($0,0021$ vs $0,0017$) jsou prakticky rovnocenné. PFN tedy s dostatečným kontextem konverguje k chování *GP oracle*, aniž by explicitně invertovalo kernelovou matici. Toto je jedním z klíčových empirických potvrzení toho, že PFN provádí plnou GP inferenci namísto pouhého *kernel smoothingu*.

3.1.5 Konvergence střední hodnoty a kalibrace rozptylu

V tomto experimentu ověřujeme, zda PFN správně zachytí nejen střední hodnotu predikce, ale také tvar profilu nejistoty (tzv. *uncertainty*), tedy kde si je model jistější a kde méně. Trénovací data jsou omezena na oblast $x \in [0,3, 0,7]$ ($n_{\text{ctx}} = 20$), testovací body pokrývají celý interval $[0, 1]$ ($n_{\text{test}} = 100$). Výsledek experimentu jsme opět získali průměrováním přes $5 \text{ realizací} \times 5 \text{ seedů} = 25$ instancí celkem. Metriky kalibrace: $\text{Corr}(\hat{\sigma}_{\text{PFN}}, \sigma_{\text{GP}})$ (korelace směrodatných odchylek) a $\text{MSE}(\hat{\sigma}_{\text{PFN}}, \sigma_{\text{GP}})$ (absolutní chyba kalibrace). Dále uvádíme *ratio far/near*, což je poměr průměrné směrodatné odchylky (*std*) v oblasti extrapolace ($x \in [0, 0,2] \cup [0,8, 1]$) ke směrodatné odchylce v centru dat ($x \in [0,4, 0,6]$), který měří, jak výrazně model zvyšuje nejistotu mimo oblast trénovacích dat. Naměřené hodnoty shrnuje tabulka 3.3.

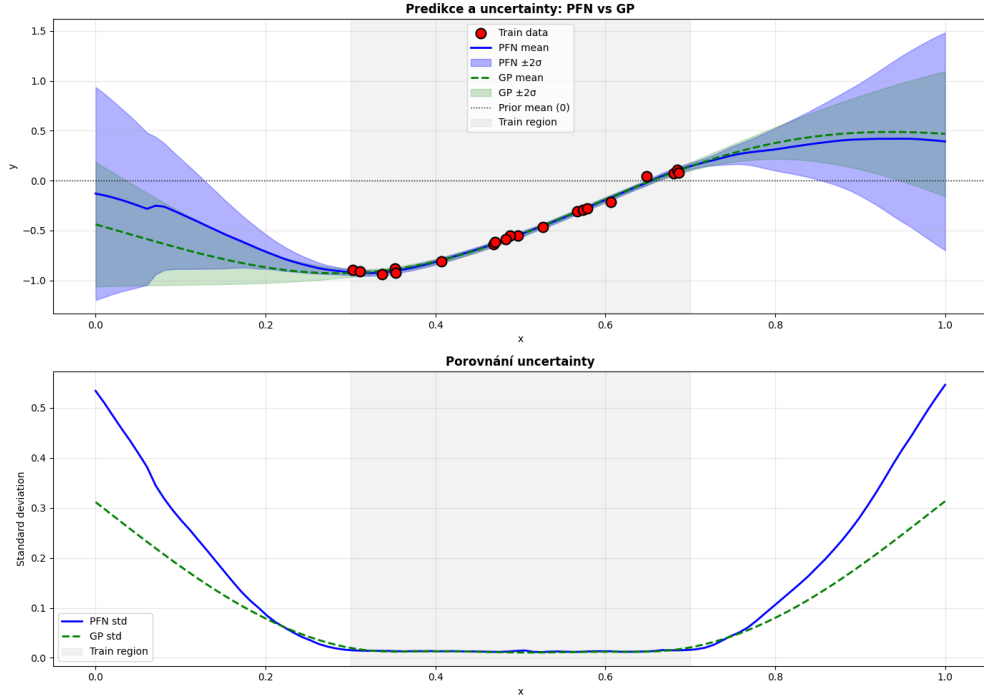
Střední hodnota PFN sleduje GP posterior s vysokou přesností v oblasti s daty, jak ukazuje obrázek 3.5. Mimo trénovací sadu ($x < 0,3$, $x > 0,7$) PFN správně konverguje k prioru (hodnota 0), stejně jako GP. Avšak poznamenejme, že pro realizace s vysokou amplitudou ($|m_{\text{train}}| > 1,5$) dochází k neúplné konvergenci, GP se v takových případech chová stejně.

Metrika	Hodnota
$\text{Corr}(\text{PFN std}, \text{GP std})$	$0,9812 \pm 0,0252$
$\text{MSE}(\text{PFN std}, \text{GP std})$	$0,00376 \pm 0,00911$
Ratio far/near – PFN	$21,3 \times \pm 5,2 \times$
Ratio far/near – GP	$25,9 \times \pm 3,8 \times$

Tabulka 3.3: Experiment 1: kalibrace rozptylu pro fixní model (průměr přes 25 instancí).

Diskuze:

Podle tabulky 3.3 korelace $r = 0,98$ potvrzuje, že PFN správně identifikuje tvar profilu *uncertainty*, tedy kde je nejistota větší a kde menší. $\text{MSE}(\hat{\sigma}) = 0,0038$ signalizuje, že absolutní kalibrace je



Obrázek 3.5: Experiment 1 – fixní model: srovnání střední hodnoty a rozptylu PFN (modrá) a GP (zelená). Trénovací body v $x \in [0,3, 0,7]$, predikce v celém $[0, 1]$. PFN správně rozpoznává region extrapolace zvýšeným rozptylem.

přesná. *Ratio far/near* u PFN (21,3×) dosahuje přibližně 82 % dynamického rozsahu GP (25,9×). Z toho usuzujeme, že PFN mírně podhodnocuje míru zvýšení *uncertainty* v oblasti extrapolace, avšak můžeme tvrdit, že charakter chování je zachován. Toto je zásadní vlastnost. Model bez explicitní reprezentace GP posterioru přesto generuje *uncertainty*, která odpovídá Bayesovskému výpočtu.

3.1.6 Shrnutí

Tato sekce zkoumala fixní model, tedy PFN natrénované na jediném GP prioru se známými hyperparametry. Ukázali jsme, že *attention matice* mají stabilní vnitřní strukturu, rané vrstvy distribuují váhu široce, zatímco v poslední vrstvě dominuje blok *Test*→*Train* (obrázek 3.1).

Následně jsme vyvrátili hypotézu, že se model chová jako prostý *kernel smoother*. Korelace *attention vah* s RBF kernelem je nízká a predikce PFN se od Nadaraya-Watsonova estimátoru řádově liší, poměr chyb dosahuje 1 256× ve prospěch PFN (tabulka 3.1). PFN tedy sedí těsně u analytického GP posterioru, zatímco NW stojí daleko od obou.

Stejný závěr platí i mimo syntetické srovnání. Při predikci skutečných šumových hodnot GP realizace se PFN s rostoucím kontextem přibližuje *GP oraclu* a od $n = 20$ jsou obě metody prakticky rovnocenné (tabulka 3.2). Model také správně reprodukuje tvar profilu nejistoty a nárůst nejistoty v oblasti extrapolace jen mírně podhodnocuje (tabulka 3.3).

Fixní model tedy dostatečně věrně aproximuje plnou GP inferenci, a to jak ve střední hodnotě, tak v rozptylu. Zůstává otázka, co se stane, když modelu hyperparametry předem nesdělíme. Tomu se věnuje následující sekce.

3.2 PFN nad rodinou Gaussovských procesů: náhodné hyperparametry

V předchozí sekci jsme zkoumali model, který jsme natrénovali na *fixních* hyperparametrech, a který se snažil aproximovat posterior jednoho konkrétního Gaussovského procesu s pevně zvoleným $\ell = 0,3$, $\sigma^2 = 10^{-4}$. Šlo o výrazné zjednodušení. Model přesně věděl, jaký tvar funkcí má predikovat, a jediné, co se musel naučit, bylo efektivně invertovat kernelovou matici jednoho jediného kernelu. V této sekci přejdeme k obtížnějšímu a zajímavějšímu případu. Ted' model natrénujeme na celé rodině Gaussovských procesů, jejíž hyperparametry jsou při každé dávce vzorkovány náhodně z předem daného rozdělení.

Tento posun mění úlohu zásadním způsobem. Zatímco fixní model aproximuje jeden GP posterior, náhodný model se musí naučit něco podstatně obecnějšího, a to rozpoznat hyperparametry přímo z kontextových dat a teprve poté provést odpovídající inferenci. Z jednoho dopředného průchodu se tak stává nejen aproximace inverze kernelové matice, ale i implicitní identifikace korelační délky, amplitudy a šumu. V důsledku model musí provést i amortizovanou Bayesovskou marginalizaci přes nejistotu v těchto hyperparametrech. Právě tato vlastnost otevírá řadu otázek, které u fixního modelu nemají smysl:

- Jak přesně model identifikuje ℓ z kontextu?
- Kolik bodů k tomu potřebuje?
- A jak dobře si poradí s celým rozpětím korelačních délek, na kterém byl trénován?

3.2.1 Trénink nad rozdělením hyperparametrů

Architektura modelu je identická s fixním případem: 6 vrstev, 8 *attention* hlav, dimenze embeddingu 512, dimenze skryté vrstvy dopředné sítě 1024, výstupní `FullSupportBarDistribution` s 1000 biny, nastavení `features_per_group=1`. Liší se pouze trénovací režim a délka tréninku oproti fixnímu modelu, který běžel 100 epoch, zatímco tento náš náhodný model běžel 500 epoch. Zatímco fixní model viděl při každé dávce data z téhož GP, náhodný model dostával při každé dávce data z GP s nezávisle vzorkovanými hyperparametry:

$$\ell \sim \mathcal{U}(0,05, 1,0), \quad \text{outputscale} \sim \text{LogNormal}(0, 0,5), \quad \sigma^2 \sim 10^{\mathcal{U}(-3, -1)}.$$

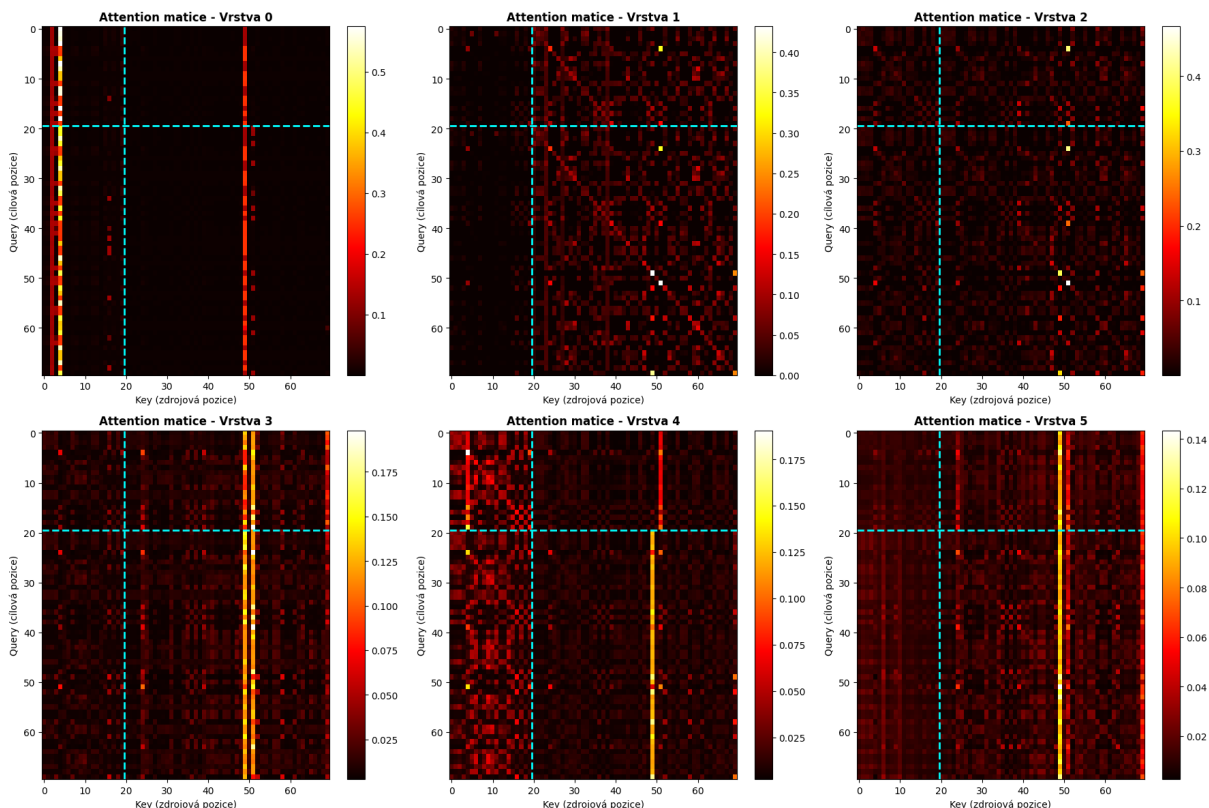
Vstupní prostor zůstává $x \in [0, 1]$, kernel je opět RBF. Šum tak během tréninku pokrývá rozmezí $\sigma^2 \in [10^{-3}, 10^{-1}]$. V experimentech níže, kde náhodný model testujeme samostatně (kalibrace rozptylu, přesnost napříč ℓ , identifikace ℓ), přitom volíme fixní šum $\sigma^2 = 10^{-3}$ na spodní hranici tohoto rozsahu, aby testovací data zůstala uvnitř trénovací distribuce.

Jak jsme již řekli v úvodu této sekce, tato drobná změna trénovacího protokolu má velký důsledek. Model totiž musí generalizovat přes nespočetně mnoho tvarů funkcí, a to od rychle oscilujících realizací s krátkou korelační délkou po hladké, téměř lineární průběhy. Nemůže si proto „zapamatovat“ jeden kernel, musí se ho naučit najít z dat. Právě proto vyžaduje podstatně delší trénink oproti fixnímu modelu.

Dále postupujeme ve dvou krocích. Nejprve ověříme, zda si náhodný model zachovává stejné vnitřní mechanismy, které jsme identifikovali u fixního modelu (*attention asymetrie*, ne-RBF strategie, kalibrace rozptylu), a za jakou cenu. Poté se zaměříme na schopnost, kterou fixní model nemá a mít nemůže, konkrétně na identifikaci hyperparametrů přímo z kontextových dat.

Přenositelnost vnitřních mechanismů.

Náhodný model řeší těžší úlohu než fixní. Přirozená otázka tedy zní, zda si zachovává stejné kvalitativní mechanismy, které jsme odhalili v předchozí sekci, nebo zda ho nutnost generalizace vede k odlišné vnitřní organizaci. V této části projdeme tři klíčové experimenty z předchozí sekce: strukturu *attention* matic, srovnání *attention* s RBF kernelem a kalibraci rozptylu, a v každém z nich porovnáme oba modely přímo.



Obrázek 3.6: *Attention matice* pro všech 6 vrstev náhodného modelu (průměr přes 8 hlav). Cyánová čára odděluje trénovací a testovací pozice. Vzor je kvalitativně identický s fixním modelem. Od distribuované vrstvy 0 po dominantní blok *Test*→*Train* v poslední vrstvě. Pouze méně vyostřený, protože část *attention rozpočtu* jde na identifikaci hyperparametrů.

3.2.2 Struktura attention matic

Stejně jako u fixního modelu vykreslíme *attention matice* pro všech 6 vrstev (průměr přes 8 hlav) a sledujeme jejich vývoj napříč hloubkou sítě. Připomínáme kvadrantovou interpretaci: matici lze rozdělit podle toho, zda *query* i *key* pozice patří trénovacím, nebo testovacím bodům, přičemž pro nás klíčový je kvadrant *Test*→*Train*, který říká, jak testovací body čerpají informaci z kontextu.

Výsledek je na obrázku 3.6. Kvalitativní vzor se podobá fixnímu modelu. Vrstva 0 je pořád spíše poznávací, model se dívá na celou situaci a zkoumá její zákonitosti. Prostřední vrstvy postupně řídí a specializují se. Poslední vrstva vykazuje jasně dominantní kvadrant *Test*→*Train*, zatímco *Train*→*Test* je téměř nulový. Model tímto způsobem sám z dat objevil základní princip GP inference: testovací body jsou podmíněny trénovacími, ale ne naopak.

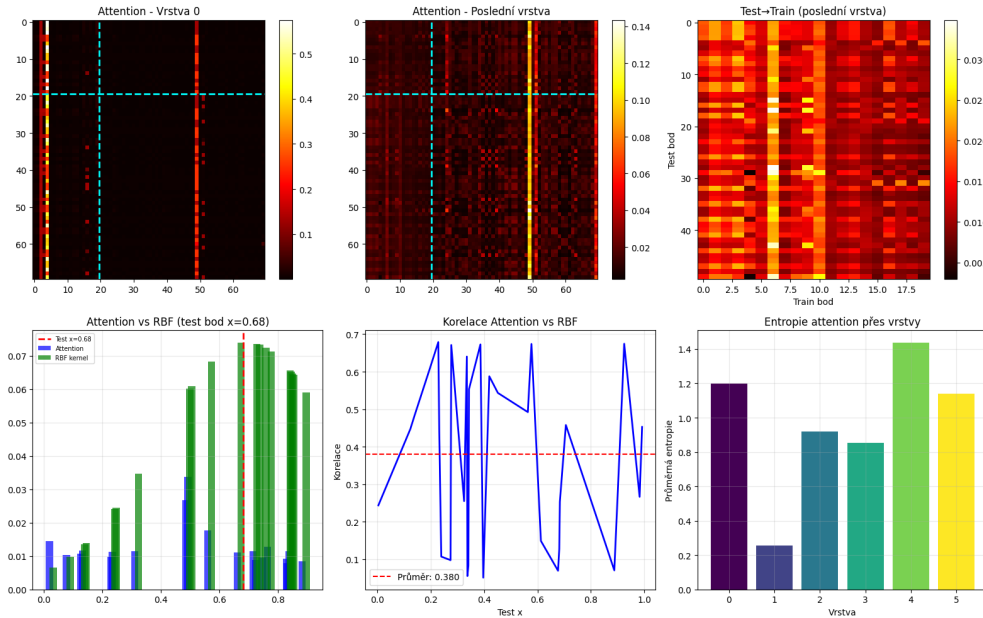
Jediný rozdíl je kvantitativní, vzory se u náhodného modelu zdají zašuměné a méně čisté než u fixního. To dává dobrý smysl. Fixní model se v poslední vrstvě soustředí uje na inverzi jediného, předem známého kernelu, a jeho *attention* se proto může vyladit velmi ostře. Náhodný model naproti tomu musí ve stejných vahách zakódovat i identifikaci hyperparametrů, proto část kapacity *attention* jde na hledání korelační délky z kontextu, nikoli jen na samotné vážení bodů. Výsledná matice je proto „rozmazanější“, aniž by však ztratila základní strukturu.

3.2.3 Attention vs RBF kernel

V předchozí sekci jsme ukázali, že *attention* fixního modelu *nekoreluje* s normalizovaným RBF kernelem. Model se naučil ostřejší a nelokální strategii, nikoli prostý kernel-smoothing. Zopakujeme stejný experiment i pro náhodný model. Pro každý testovací bod x^* porovnáme *attention váhy* poslední vrstvy s normalizovanými RBF vahami $w_i = k(x^*, x_i) / \sum_j k(x^*, x_j)$.

Metrika	Fixní model	Náhodný model
Průměrná korelace Attn vs RBF	$0,374 \pm 0,270$	$0,316 \pm 0,311$

Tabulka 3.4: Korelace *attention vah* poslední vrstvy s normalizovaným RBF kernelem. Průměr přes 5 *seedů*. U náhodného modelu je korelace ještě nižší než u fixního.



Obrázek 3.7: Detailní rozbor *attention* náhodného modelu (průměr přes 8 hlav). Nahoře: *attention matice* vrstvy 0 a poslední vrstvy a výřez bloku *Test→Train*. Dole vlevo: *attention váhy* (modrá) pro vybraný testovací bod v porovnání s normalizovanými RBF vahami (zelená). Dole uprostřed: korelace *attention* s RBF jako funkce polohy testovacího bodu. Dole vpravo: průměrná entropie *attention vah* po vrstvách. Stejně jako u fixního modelu je *attention* výrazně ostřejší a s RBF nekoreluje (korelace $0,316 \pm 0,311$).

Diskuze:

Výsledky obou modelů shrnuje následující tabulka 3.4. Korelace náhodného modelu ($0,316 \pm 0,311$) je dokonce *nižší* než u fixního ($0,374 \pm 0,270$). Na první pohled by se to mohlo jevit jako zhoršení, ale je tomu naopak. Jak jsme již podrobně rozebrali v předchozí sekci, čistý RBF kernel $k(x^*, x_i)$ není správnou referenční hodnotou, tou jsou GP váhy $w_i^{\text{GP}} = \sum_j k(x^*, x_j) (\mathbf{K}^{-1})_{ji}$, které jsou nelokální a mohou být i záporné. Nižší korelace tedy neznamená, že se model vzdaluje od GP. Spíše znamená, že se vzdaluje od prosté *kernel-averaging* strategie a nachází lepší a více efektivní reprezentaci. U náhodného modelu je tento efekt výraznější, protože model se navíc naučil, že fixní RBF s jedním konkrétním ℓ vůbec není univerzálně správný. Korelační délku teprve odhaduje z dat, takže se na žádný pevný kernel nemůže spoléhat. Obrázek 3.7 potvrzuje podobný kvalitativní obraz jako u fixního modelu: *attention* je ostré a koncentruje váhu na nejbližší body, zatímco RBF ji rozprostírá více plynule.

3.2.4 Kalibrace rozptylu

Zatímco střední hodnota vypovídá o kvalitě bodové predikce, rozptyl vypovídá o tom, zda model správně kvantifikuje svou nejistotu. Zopakujeme tedy experiment kalibrace z předchozí sekce i pro náhodný model. Trénovací data jsou omezena na oblast $x \in [0,3, 0,7]$ ($n_{\text{ctx}} = 20$), testovací body pokrývají celý interval $[0, 1]$, a měříme, jak moc směrodatná odchylka PFN odpovídá GP. Model je přitom testován na datech z GP ($\ell = 0,3$), tedy na jediné korelační délce, i když byl trénován na celém rozpětí.

Zde opět budeme zkoumat tzv. profil nejistoty, tedy to, jak se předpovídaná směrodatná odchylka modelu mění napříč vstupním prostorem. U dobře kalibrovaného modelu by měla být nejmenší tam, kde leží trénovací data (uvnitř $[0,3, 0,7]$), a plynule růst směrem k okrajům intervalu, kam už žádná data nedosahují. Tam, kde model „neví“, musí dávat najevo větší nejistotu. Sledujeme přitom dvě odlišné věci: zda má profil správný tvar (tzn. zda nejistota roste na správných místech) a zda má i správnou velikost (tzn. roste dostatečně, nebo si model svou nejistotou příliš věří). Kalibraci proto posuzujeme třemi metrikami. První je korelace mezi směrodatnou odchylkou předpovídanou modelem a směrodatnou odchylkou GP. Ta hodnotí právě tvar profilu, a hodnota blízká jedné znamená, že model dobře pozná, kde je nejistota větší a kde menší. Druhá je střední kvadratická odchylka obou těchto profilů, která naopak hodnotí velikost, tedy jak přesně se předpovídaná směrodatná odchylka shoduje s GP i v absolutních číslech. Třetí metrika, *ratio far/near*, popisuje dynamický rozsah nejistoty. Je to poměr průměrné směrodatné odchylky v oblasti daleko od dat, tedy v oblasti extrapolace u okrajů intervalu, ke směrodatné odchylce v centru dat, kde leží trénovací body. Číslo tak říká, o kolikrát je model daleko od dat víc nejistý než uprostřed nich. Hodnota pro GP je přitom analytická a slouží jako ideál. Je v obou sloupcích stejná, protože referenční GP je v obou experimentech stejný. Hodnota pro PFN pak ukazuje, jakou část tohoto ideálního rozsahu model skutečně reprodukuje.

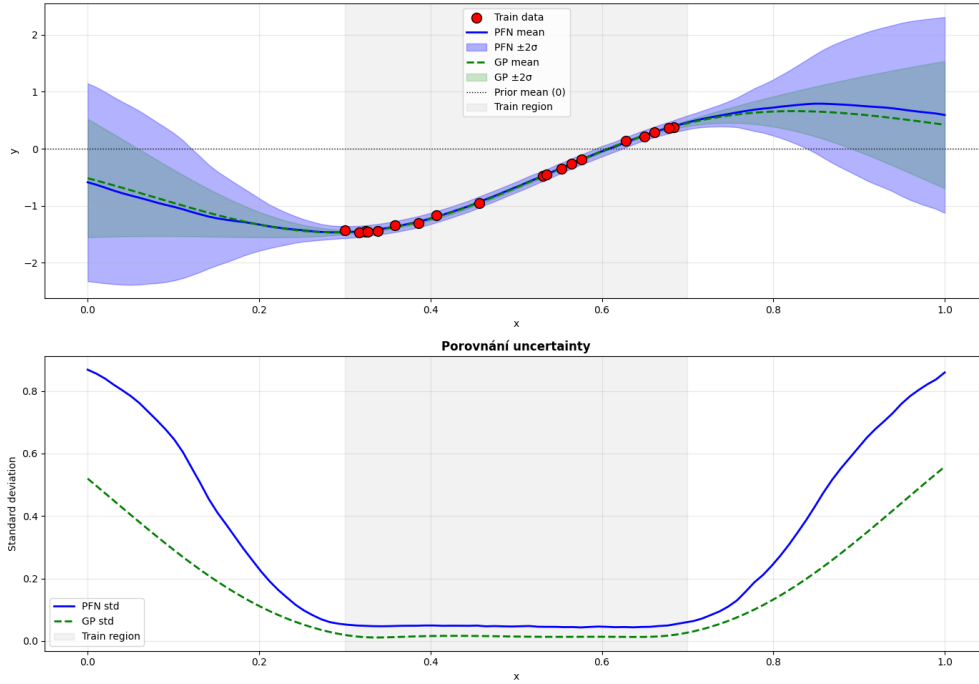
Metrika	Fixní model	Náhodný model
Corr(PFN std, GP std)	$0,9812 \pm 0,0252$	$0,9734 \pm 0,0292$
MSE(PFN std, GP std)	$0,00376 \pm 0,00911$	$0,03478 \pm 0,03461$
Ratio far/near – PFN	$21,3 \times \pm 5,2 \times$	$7,49 \times \pm 2,20 \times$
Ratio far/near – GP	$25,9 \times \pm 3,8 \times$	$25,9 \times \pm 3,8 \times$

Tabulka 3.5: Kalibrace rozptylu obou modelů (průměr přes 25 instancí). *Ratio far/near* je poměr směrodatné odchylky v oblasti extrapolace ke směrodatné odchylce v centru dat. Náhodný model správně poznává, kde je nejistota větší a kde menší (vysoká korelace), ale její velikost odhaduje výrazně hůř, v okrajích ji zvyšuje méně, než by měl.

Diskuze:

Tyto tři metriky pro oba modely srovnává tabulka 3.5. Zde nacházíme první výraznou nevýhodu náhodného modelu. Korelace profilu nejistoty zůstává vysoká ($\text{Corr} = 0,973$ oproti $0,981$ u fixního modelu). Obě sítě tedy správně rozpoznávají kde je nejistota větší a kde je menší. Absolutní kalibrace (tj. skutečná velikost předpovídané nejistoty, nikoli jen to, kde je větší a kde menší) je však u náhodného modelu podstatně horší. MSE směrodatné odchylky vzroste zhruba desetinásobně ($0,0348$ vs $0,0038$) a *ratio far/near* klesne z $21,3 \times$ na pouhých $7,49 \times$, tj. z 82% na zhruba 29% dynamického rozsahu GP. Jinými slovy, náhodný model zvyšuje nejistotu v oblastech extrapolace znatelně méně výrazně.

Tento výsledek se může zdát na první pohled paradoxní. Model trénovaný na větším množství epoch (500) má horší kalibraci rozptylu než model trénovaný na menším množství epoch (100). Vysvětlení se schovává právě v širších trénovacích podmínkách. Náhodný model se učil kalibrovat nejistotu pro celé rozpětí $\ell \in [0,05, 1,0]$, přičemž profil rozptylu se s korelační délkou dramaticky mění, a to takovým způsobem, že krátká korelační délka znamená rychlý nárůst nejistoty mimo data, dlouhá naopak volnější.



Obrázek 3.8: Náhodný model: srovnání střední hodnoty a rozptylu PFN (modrá) a GP (zelená). Trénovací body v $x \in [0,3, 0,7]$, predikce v celém $[0, 1]$. Model správně zvyšuje nejistotu v oblasti extrapolace, ale nárůst je mírnější než u GP. Širší predikce v centru dat odrážejí trénink na distribuci hyperparametrů.

Model tedy nemůže být současně dokonale kalibrován pro všechny ℓ , jeho odhad rozptylu je jistým průměrem přes možné korelační délky, což profil nutně vyhlazuje. Testujeme-li ho na jednom konkrétním $\ell = 0,3$, projeví se toto vyhlazení jako širší predikce v centru a mírnější nárůst na okrajích (obrázek 3.8). Fixní model tímto kompromisem netrpí, protože zná jediné správné ℓ . Tímto usuzujeme, že jde tedy o cenu za generalizaci, nikoli o vadu tréninku.

Nové schopnosti: identifikace hyperparametrů.

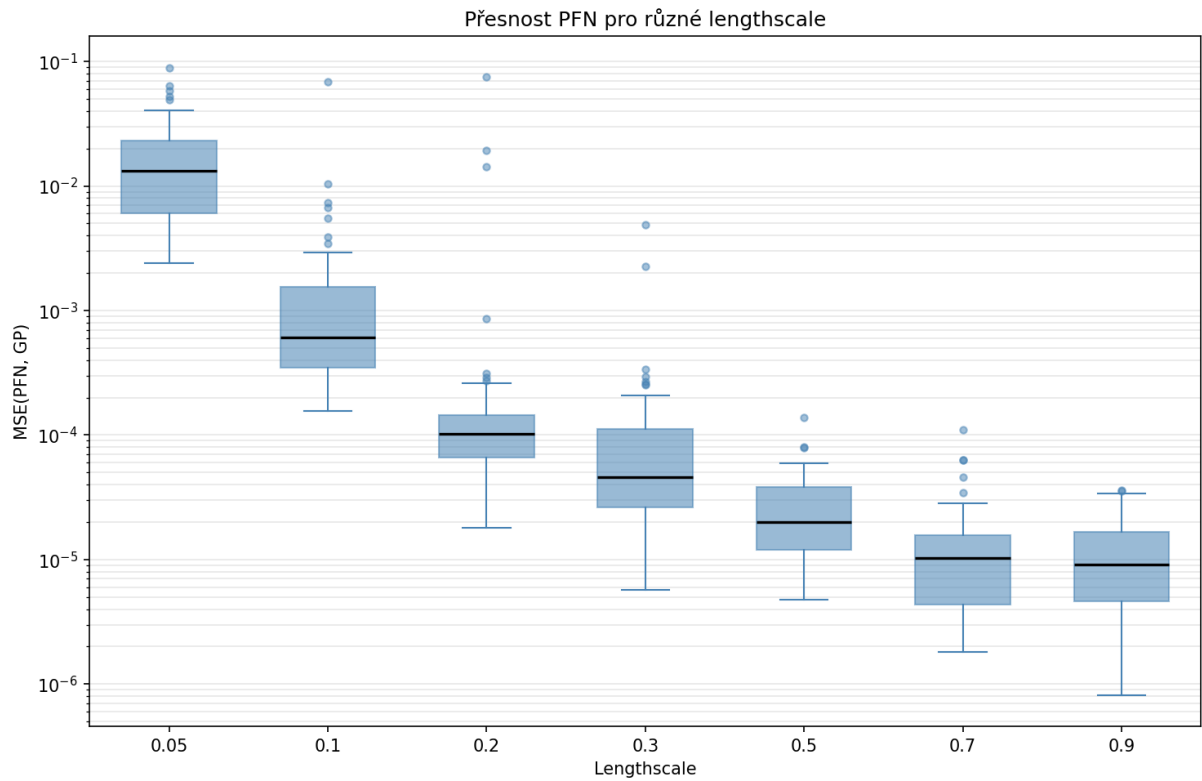
Postupně se dostáváme k jádru toho, proč náhodný model vůbec studujeme a čem je pro nás mnohem zajímavější. Experimenty této sekce nemají u fixního modelu obdobu, testují totiž přesně tu schopnost, kterou fixní model postrádá, a tou je identifikovat hyperparametry Gaussovského procesu z kontextových dat a přizpůsobit jim predikci.

3.2.5 Přesnost napříč korelačními délkami

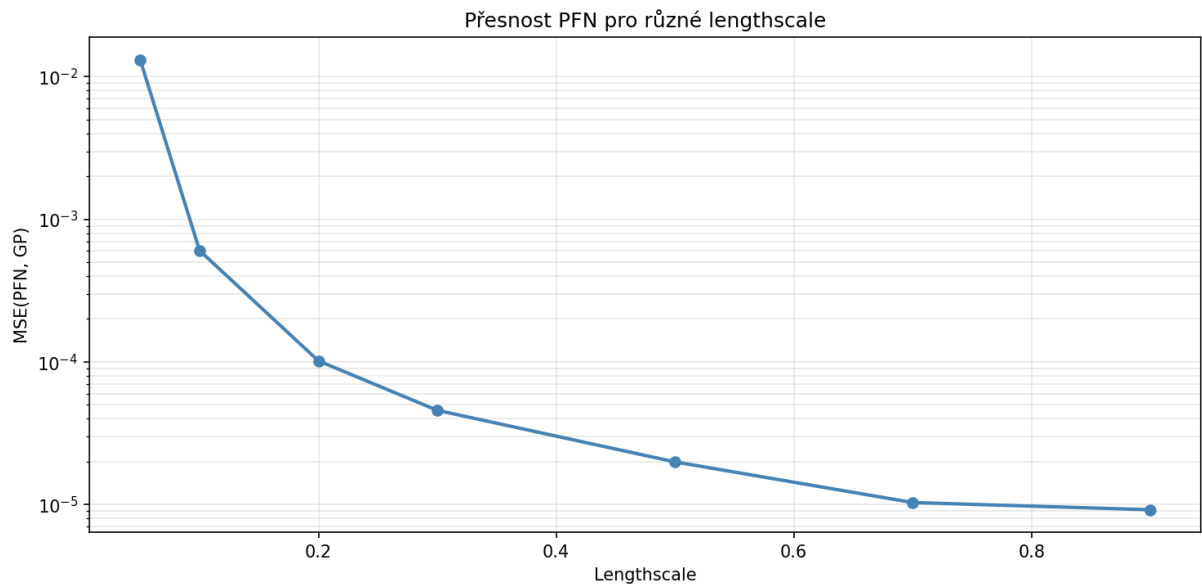
Nejprve se budeme ptát, jak moc dobře model zvládá různé korelační délky. Pro každou hodnotu $\ell \in \{0,05, 0,1, 0,2, 0,3, 0,5, 0,7, 0,9\}$ vygenerujeme GP realizace, předložíme modelu $n_{\text{ctx}} = 30$ kontextových bodů a měříme MSE mezi jeho střední hodnotou a analytickým GP posteriorem, který zná správnou korelační délku ℓ . Výsledky (medián, průměr a směrodatná odchylka přes 50 instancí) shrnuje tabulka 3.6. Rozdělení chyby přes instance i její medián napříč korelačními délkami zachycuje obrázek 3.9.

Diskuze:

Z tabulky 3.6 plyne jasný trend. Medián chyby klesá s rostoucí korelační délkou monotónně, a to o řády. Nejobtížnější jsou tedy pro model rychle oscilující funkce s krátkou korelační délkou. Vysvětlení souvisí s hustotou vzorkování. Připomeňme z části 3.2.1, že krátká korelační délka odpovídá rychle



(a) Krabicový graf MSE(PFN, GP) jako funkce ℓ (rozdělení přes 50 instancí, svislá osa logaritmická). Krabice značí medián a kvartily, samostatné body jsou odlehle instance. Pro krátkou korelační délku $\ell = 0,05$ výrazně roste chyba i její rozptyl.



(b) Medián MSE(PFN, GP) jako funkce ℓ (svislá osa logaritmická). Medián klesá monotónně s rostoucím ℓ , o více než tři řády z $\ell = 0,05$ na $\ell = 0,9$.

Obrázek 3.9: Náhodný model: přesnost napříč korelačními délkami.

ℓ	Medián MSE	Průměr MSE	Std MSE
0,05	0,01317	0,01918	0,01767
0,10	0,000604	0,002773	0,009616
0,20	0,000102	0,002304	0,010984
0,30	$4,57 \times 10^{-5}$	$2,18 \times 10^{-4}$	$7,40 \times 10^{-4}$
0,50	$1,99 \times 10^{-5}$	$2,86 \times 10^{-5}$	$2,42 \times 10^{-5}$
0,70	$1,03 \times 10^{-5}$	$1,52 \times 10^{-5}$	$1,90 \times 10^{-5}$
0,90	$9,18 \times 10^{-6}$	$1,23 \times 10^{-5}$	$9,99 \times 10^{-6}$

Tabulka 3.6: MSE(PFN, GP) náhodného modelu jako funkce korelační délky ℓ ($n_{\text{ctx}} = 30$, průměr přes 50 instancí). Medián klesá monotónně s rostoucím ℓ o více než tři řády z $\ell = 0,05$ na $\ell = 0,9$.

oscilujícím funkcím. Ty se znatelně mění už na vzdálenostech řádu ℓ , a právě na těchto vzdálenostech ji musí kontext zachytit. Funkce s $\ell = 0,05$ proto vyžaduje hustou vzorkovací mřížku. Kontext o 30 bodech na intervalu $[0, 1]$ je však řídký pro takové délky. Průměrný rozestup sousedních bodů je s ní srovnatelný, a protože body nejsou rozmístěny pravidelně, vznikají mezi nimi i mezery delší než ℓ . V takové mezeře se funkce stihne změnit, aniž o tom data cokoli prozradí. Mezi body tedy zůstává příliš mnoho neznámé informace. Model si proto nemůže být jistý, jak rychle funkce osciluje, a jeho predikce se od GP posterioru se správným ℓ vzdaluje.

Poznamenejme ještě rozdíl mezi mediánem a průměrem chyby. Pro malá ℓ leží průměr výrazně nad mediánem, což může prozrazovat existenci ojedinělých instancí, na kterých model selhává podstatně hůř než na ostatních. Tyto odlehle instance jsou dobře vidět na krabicovém grafu na obrázku 3.9a, kde se pro malá ℓ objevuje nad krabicí řada samostatných bodů. Pro $\ell \geq 0,5$ se naopak rozdělení chyby stahuje těsně k mediánu. V hladkém režimu je tedy model spolehlivý a téměř bez odlehklých instancí. Selhání při krátkých ℓ si vysvětlujeme tím, že PFN rychlé oscilace „přehlazuje“, protože je z řídkého kontextu nedokáže rozlišit.

3.2.6 Identifikace korelační délky z kontextu

Předchozí experiment ukázal, že přesnost roste s ℓ , ale ještě přímo nedokázal, že model korelační délku skutečně umí identifikovat. K tomu slouží následující test. Data pocházejí z GP se známým ℓ . Modelu předložíme $n_{\text{ctx}} \in \{3, 5, 10, 20, 40, 60\}$ bodů a jeho predikci porovnáme se dvěma referenčními posteriory: se správným GP (zná právě ℓ) a se špatným GP (má výrazně odlišné ℓ_{wrong}). Klíčová je následující úvaha: pokud PFN identifikovalo korelační délku z dat, jeho predikce bude blízko správnému GP a daleko od špatného, tedy poměr $\text{MSE}(\text{PFN}, \text{GP}_{\text{wrong}})/\text{MSE}(\text{PFN}, \text{GP}_{\text{correct}})$ bude velký. Tento poměr budeme dále značit zkráceně *wrong/correct*. Pokud ji neidentifikoval, obě chyby budou podobné a poměr bude blízký jedné. Provedeme tento test pro dvě pravé korelační délky, krátkou ($\ell = 0,1$) a střední ($\ell = 0,2$). Špatné ℓ_{wrong} přitom volíme zřetelně jiné, aby byl kontrast dobře vidět, konkrétně 0,7 pro první případ a 0,05 pro druhý. Výsledky jsou v tabulkách 3.7 a 3.8.

Diskuze:

Výsledek je jednoznačný a představuje jeden z nejsilnějších důkazů, že náhodný model skutečně provádí identifikaci hyperparametrů. Při $n_{\text{ctx}} = 3$ zůstává poměr *wrong/correct* malý (např. $14,9 \times$ pro $\ell = 0,1$, tabulka 3.7). S tak malým kontextem model nedokáže korelační délku spolehlivě určit a musí hledat kompromis mezi možnými ℓ , takže zůstává vzdálený od obou referenčních GP. Od $n_{\text{ctx}} = 20$ však poměr prudce roste, a to o řády (při $n_{\text{ctx}} = 60$ dosahuje $1\,691 \times$). Zásadní je, že chyba vůči správnému GP klesá s rostoucím kontextem k prakticky nulovým hodnotám, zatímco chyba vůči špatnému GP zůstává vysoká a od $n_{\text{ctx}} = 20$ se téměř nemění (při $n_{\text{ctx}} = 60$ je $0,000134$ oproti $0,2266$). To znamená, že od

n_{ctx}	MSE(PFN, GP _{correct})	MSE(PFN, GP _{wrong})	Poměr
3	$0,1239 \pm 0,1047$	$1,846 \pm 4,437$	14,9×
5	$0,1043 \pm 0,1226$	$1,403 \pm 6,431$	13,4×
10	$0,0483 \pm 0,0579$	$0,812 \pm 2,033$	16,8×
20	$0,00855 \pm 0,01269$	$0,2665 \pm 0,1725$	31,2×
40	$0,000629 \pm 0,00149$	$0,2799 \pm 0,1707$	444,7×
60	$0,000134 \pm 0,000111$	$0,2266 \pm 0,1447$	1 691×

Tabulka 3.7: Identifikace korelační délky pro $\ell = 0,1$ (proti $\ell_{\text{wrong}} = 0,7$). Průměr přes 50 instancí. Poměr *wrong/correct* prudce roste s n_{ctx} .

n_{ctx}	MSE(PFN, GP _{correct})	MSE(PFN, GP _{wrong})	Poměr
3	$0,0610 \pm 0,0563$	$0,5499 \pm 0,9273$	9,0×
5	$0,0613 \pm 0,1150$	$0,4576 \pm 2,020$	7,5×
10	$0,0200 \pm 0,0415$	$0,2319 \pm 0,4873$	11,6×
20	$0,00170 \pm 0,00409$	$0,04528 \pm 0,03577$	26,6×
40	$8,43 \times 10^{-5} \pm 1,27 \times 10^{-4}$	$0,04148 \pm 0,04236$	491,8×
60	$3,78 \times 10^{-5} \pm 3,13 \times 10^{-5}$	$0,03028 \pm 0,02328$	801,4×

Tabulka 3.8: Identifikace korelační délky pro $\ell = 0,2$ (proti $\ell_{\text{wrong}} = 0,05$). Průměr přes 50 instancí.

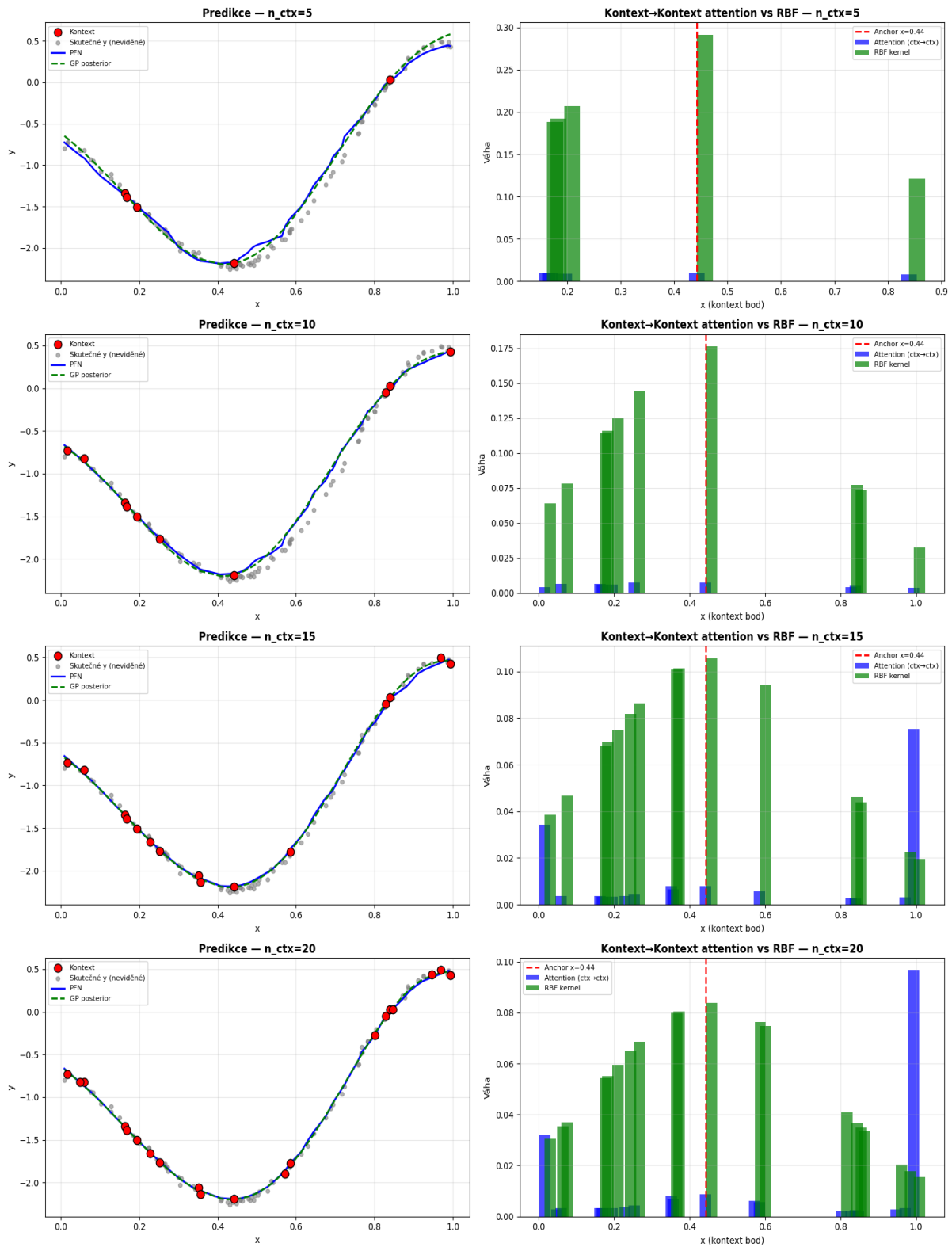
$n_{\text{ctx}} \approx 20$ se náhodný model chová skoro identicky jako GP se správnou korelační délkou bez toho, aby mu ji kdokoli sdělil. Pro $\ell = 0,2$ pozorujeme stejný vzor (tabulka 3.8, poměr roste z 9,0× na 801,4×). Právě zde se ukazuje kvalitativní propast mezi oběma modely. Dá se říct, že fixní model má korelační délku „zabudovanou“ do vah, náhodný model ji čte z kontextu.

3.2.7 Predikce na skutečné realizaci a role kontextu

Zopakujeme nakonec i experiment z předchozí sekce, ve kterém oba modely predikují skutečné (šumové) hodnoty neviděné GP realizace, s kontextovými body $n_{\text{ctx}} \in \{5, 10, 15, 20\}$. Připomeňme, že *fixní PFN* je model natrénovaný na jediném GP s pevně danou korelační délkou $\ell = 0,3$. Tu jsme si řekli, že má „zabudovanou“ a nemusí ji z dat zjišťovat. Naproti tomu *náhodné PFN* bylo trénováno na celé rodině GP s různými ℓ , takže správnou korelační délku musí u každého vstupu nejprve rozpoznat z kontextu, a teprve pak predikovat. Tentokrát nás zajímá přímé srovnání obou těchto modelů a *GP oracle*, který zná správné $\ell = 0,3$. Výsledky jsou uvedeny v tabulce 3.9. Samotné predikce a *attention váhy* pro jednu konkrétní realizaci s rostoucím kontextem ukazuje obrázek 3.10.

n_{ctx}	PFN (náhodné)	PFN (fixní)	GP oracle
5	$0,0598 \pm 0,0958$	$0,0408 \pm 0,0698$	$0,0340 \pm 0,0635$
10	$0,0188 \pm 0,0751$	$0,0123 \pm 0,0349$	$0,0112 \pm 0,0360$
15	$0,0063 \pm 0,0193$	$0,0064 \pm 0,0271$	$0,0028 \pm 0,0036$
20	$0,0020 \pm 0,0013$	$0,0021 \pm 0,0025$	$0,0017 \pm 0,0011$

Tabulka 3.9: MSE vůči skutečným hodnotám GP realizace ($\ell = 0,3$) jako funkce velikosti kontextu. Průměr přes 100 instancí. Náhodný model zaostává za fixním pro střední n , ale při $n = 20$ je s ním srovnatelný.



Obrázek 3.10: Náhodný model: predikce (levý sloupec) a *attention* vs RBF (pravý sloupec) pro $n \in \{5, 10, 15, 20\}$. Červené body: kontext, šedé: skutečné neviděné hodnoty, modrá: PFN, zelená přerušovaná: GP se správným ℓ .

Diskuze:

Srovnání tabulky 3.9 s tabulkou 3.2 z předchozí sekce opět odhaluje jakousi daň za generalizaci. Pro střední velikost kontextu ($n = 5$ až 10) je náhodný model viditelně horší než fixní ($0,060$ vs $0,041$ při $n = 5$), protože fixní model zná $\ell = 0,3$, kdežto náhodný ho musí z několika málo bodů teprve odhadnout a při řídkém kontextu je tento odhad zatížen chybou, což se projeví i většími směrodatnými odchylkami v tabulce 3.9. Od $n = 15$ se však oba modely začínají vyrovnávat a při $n = 20$ jsou prakticky nerozlišitelné ($0,0020$ vs $0,0021$), oba těsně za *GP oraclem*. Náhodný model tedy s dostatečným kontextem dožene fixní verzi, aniž by kdy znal správné ℓ předem. Poznamenejme, že nenulové MSE v případě *GP oracle* je v pořádku, tu chybu produkuje zašuměnost dat.

3.2.8 Shrnutí

Přechod od fixního k náhodnému modelu odhalil dvě tváře této architektury. Na jedné straně si náhodný model zachovává všechny kvalitativní mechanismy fixního, a to jsou: kauzální asymetrie *attention* (poslední vrstva dominantní *Test→Train*), model nedělá RBF strategii (korelace $0,32$ vs $0,37$, tabulka 3.4), zachovává se správný tvar profilu nejistoty (korelace $\text{std} > 0,97$, tabulka 3.5). Vnitřní organizace transformeru je tedy robustní vůči šíři trénovací distribuce.

Na druhé straně za tuto obecnost platí jistou daň. Absolutní kalibrace rozptylu se zhorší (*ratio far/near* klesne z $21,3\times$ na $7,49\times$, viz tabulky 3.3 a 3.5), protože jeden model nemůže být dokonale kalibrován pro všechny korelační délky současně. A pro střední velikosti kontextu je náhodný model méně přesný než fixní, neboť korelační délku teprve odhaduje z dat.

Avšak rozhodující je to, co náhodný model umí navíc. Dokáže identifikovat korelační délku z kontextu, a to spolehlivě. Poměr *wrong/correct* roste o řády a dosahuje $1\,691\times$ při $n = 60$ (tabulka 3.7). Od $n_{\text{ctx}} \approx 20$ se chová téměř identicky jako GP se správným ℓ . Náhodný model tedy je amortizovaný Bayesovský inferenční nástroj, který korelační délku a amplitudu odečítá přímo z pozorovaných dat, aniž by mu je kdokoli sdělil předem.

3.3 Specializace vrstev: čtení labelů a kernelové porovnávání

V předchozích dvou sekcích jsme se pohybovali výhradně mezi šestivrstevnými modely. Nejprve šlo o fixní model ze sekce 3.1, poté o náhodný model ze sekce 3.2. Ukázali jsme, že PFN neprovádí prostý *kernel smoothing* a že náhodný model dokáže identifikovat korelační délku z kontextu. Zůstává však otevřená otázka, jak tento výpočet uvnitř transformeru vlastně probíhá. Jinými slovy, co dělá která vrstva a proč jich model vůbec potřebuje víc než jednu. V této sekci proto poprvé budeme systematicky měnit hloubku sítě a zkoumat, zda se mezi vrstvami vytváří jakési přerozdělování práce, závisí-li výsledky na počtu vrstev apod.

Analýzu provedeme na rodině pěti modelů, které se liší pouze počtem vrstev: 1, 2, 4, 6 a 8 vrstev. Všechny sdílejí identickou architekturu s modely z předchozích sekcí (8 *attention* hlav, dimenze embeddingu 512, dimenze skryté vrstvy dopředné sítě 1024, *features_per_group*=1 a výstupní *Full-SupportBarDistribution* s 1000 biny). Všechny byly rovněž trénovány na stejném rozdělení hyperparametrů jako náhodný model:

$$\ell \sim \mathcal{U}(0,05, 1,0), \quad \text{outputscale} \sim \text{LogNormal}(0, 0,5), \quad \sigma^2 \sim 10^{\mathcal{U}(-3, -1)}.$$

Jednovrstvý model konvergoval již za 100 epoch, dvou- a čtyřvrstvé modely byly trénovány 300 epoch a nejhlubší šesti- a osmivrstvé 500 epoch. Tato řada modelů nám umožní sledovat, jak se vnitřní strategie transformeru mění s přidáváním vrstev.

Motivace pro tuto analýzu vychází přímo z Naglerovy teoretické práce, kterou jsme rozebírali v sekci 2.4.4. Připomeňme, že v PFN je každý kontextový bod reprezentován *tokenem* $V_j = (Y_j, X_j)^\top$, který v sobě slučuje jak vstupní souřadnici X_j , tak její pozorovanou hodnotu Y_j . *Attention skóre* jednovrstvého modelu je proto lineární kombinací obojího, zhruba tvaru $\alpha X_j + \beta Y_j$. GP váha $k(x^*, X_j)$ naproti tomu závisí výhradně na vstupní souřadnici. Nagler odtud odvozuje, že jedna vrstva nemůže reprodukovat GP posterior přesně, tzv. *tilted-measure limit* (viz sekce 2.4.4) se totiž nikdy nelokalizuje kolem x^* , a v predikci proto zůstává nenulový bias. To přirozeně vede k hypotéze o rozdělování práce mezi vrstvy. Pokud jedna vrstva nestačí, mohly by se hlubší modely naučit tento úkol rozdělit, a to tak, že rané vrstvy budou číst hodnoty Y_j (tzv. *label reading*) a prostřední vrstvy porovnávat vzdálenosti ve vstupním prostoru X (tzv. *kernel-like chování*). Právě tuto hypotézu budeme v následujících experimentech testovat.

3.3.1 Korelační analýza attention napříč vrstvami

Abychom mohli změřit, na co se která vrstva dívá, spočítáme pro každou vrstvu a každou z jejích osmi hlav dvě korelace *attention vah*:

- korelaci s *RBF/NW kernelem* $k(x^*, X_j)$, která měří, jak moc se hlava chová jako *kernelové porovnání* vstupních vzdáleností,
- korelaci s *labely* Y_j , která měří, jak moc hlava místo toho čte pozorované hodnoty.

První z nich budeme dále stručně nazývat *kernelová korelace*, druhou *labelová korelace*. Vysoká *kernelová korelace* tedy znamená, že hlava váží body podle jejich vzdálenosti od x^* , vysoká *labelová korelace* naopak, že je váží podle jejich hodnot Y_j . Experiment provedeme na 200 instancích se sdílenými daty pro všechny modely, s kontextem $n_{\text{ctx}} = 40$, $n_{\text{test}} = 20$ a referenčním kernelem s $\ell_{\text{NW}} = 0,5$.

Proč Spearman a ne Pearson:

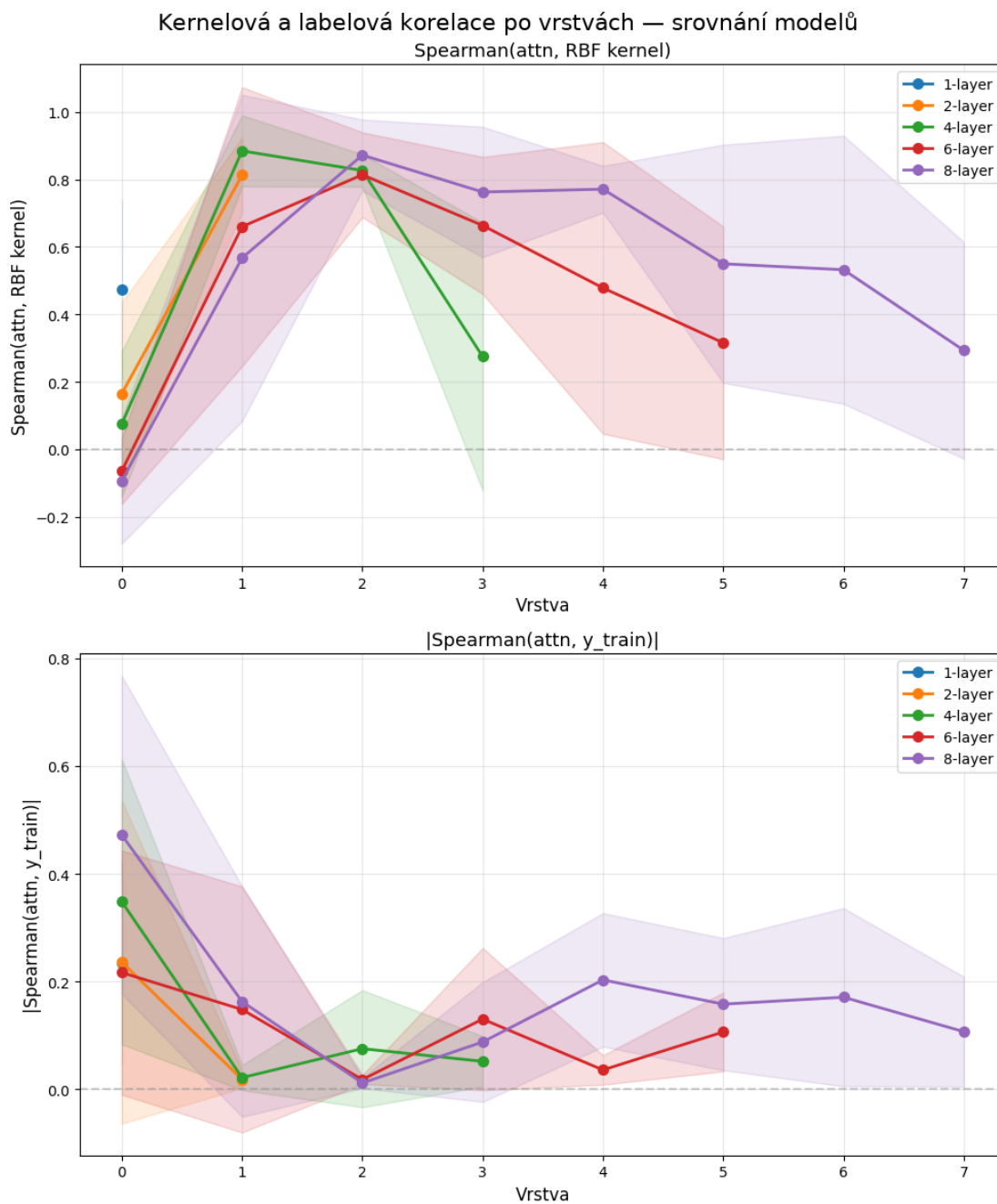
Ukazuje se totiž, že *attention váhy* jsou silně zešikmené. Naprostá většina váhy leží na několika málo bodech a zbytek je skoro nulový. Pearsonova korelace je citlivá na absolutní hodnoty, a proto monotónní vztah mezi *attention* a kernelem systematicky podhodnocuje. Přechodem na *Spearmanovu* korelaci se *kernelové korelace* ve středních vrstvách zvedly z přibližně 0,35 na hodnoty kolem 0,8. Trend zůstává stejný, ale je mnohem výraznější. V celé této sekci proto uvádíme Spearmanovy korelace.

Přehled *kernelových korelací* zprůměrovaných přes hlavy shrnuje tabulka 3.10 a graficky jej spolu s *labelovými korelacemi* zachycuje obrázek 3.11.

Diskuze:

Z tabulky 3.10 a obrázku 3.11 vystupuje jasný a opakující se napříč modely vzor. Vrstva 0 má u všech vícevrstevných modelů nejvyšší *labelovou korelaci* (tj. na dolním grafu obrázku 3.11) a zároveň nejnižší *kernelovou korelaci*. U šestivrstvého modelu je *kernelová korelace* první vrstvy prakticky nulová, u osmivrstvého dokonce mírně záporná. První vrstva tedy primárně čte *labely*, tj. shromažďuje informaci o pozorovaných hodnotách, nikoli o geometrii vstupů. Střední vrstvy tento obraz obracejí. *Kernelová korelace* vyskočí na hodnoty zhruba 0,53 až 0,89 a *labelová* klesne zhruba o řád. To je právě to *kernel-like chování*, které bychom u GP inference čekali, hlava porovnává vzdálenosti ve vstupním prostoru. U modelů se čtyřmi a více vrstvami pak poslední vrstva *kernelovou korelaci* opět snižuje (na 0,27, 0,30 až 0,42), zřejmě proto, že už neslouží k porovnávání, ale k sestavení konečné predikce. Navíc poslední vrstva u vícevrstevných modelů vždy má nejnižší *kernelovou korelaci*. Zajímavé, že u jedno- a dvouvrstvého modelů to neplatí.

Zvláštní pozornost si v tomto případě zaslouží jednovrstvý model. Jeho jediná vrstva má *kernelovou korelaci* 0,480, což je nejvyšší hodnota v celém řádku vrstvy 0 napříč modely. Má to dobrý smysl. Když má model k dispozici jen jednu vrstvu, nemůže si dovolit ji „utrátit“ na pouhé čtení *labelů* a musí v

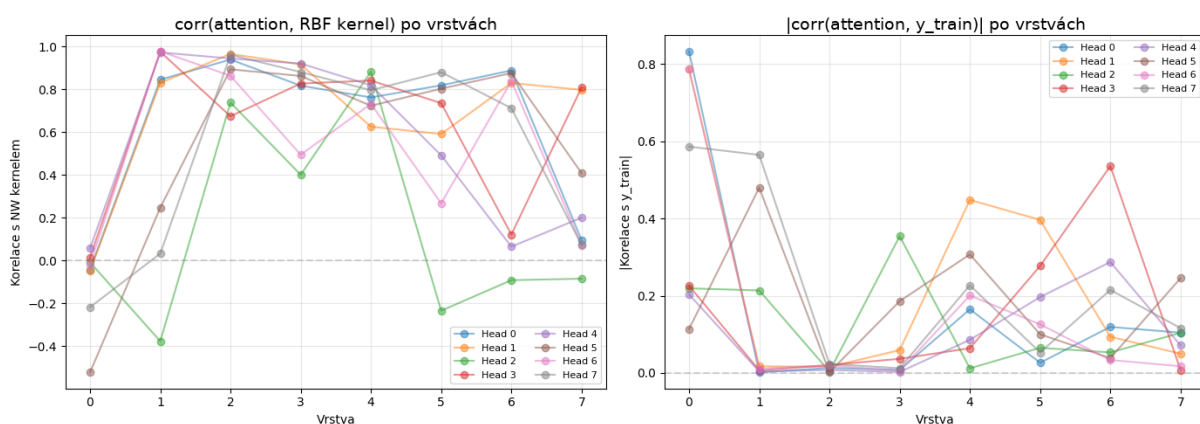


Obrázek 3.11: Srovnání všech pěti modelů. Nahoře: Spearmanova korelace *attention vah* s RBF kernelem jako funkce indexu vrstvy. Dole: absolutní hodnota korelace *attention vah* s *labely* Y_j . Barevné pásy značí ± 1 směrodatnou odchylku přes hlavy. Vzor je u všech modelů stejný: vrstva 0 má nejvyšší *labelovou* a nejnižší *kernelovou* korelaci, střední vrstvy se chovají *kernel-like* (vysoká *kernelová*, nízká *labelová* korelace) a u modelů se čtyřmi a více vrstvami v poslední vrstvě *kernelová* korelace opět klesá.

Vrstva	1 vrstva	2 vrstvy	4 vrstvy	6 vrstev	8 vrstev
0	0,480	0,168	0,074	0,004	-0,099
1	—	0,819	0,887	0,610	0,563
2	—	—	0,828	0,826	0,873
3	—	—	0,270	0,817	0,765
4	—	—	—	0,740	0,773
5	—	—	—	0,416	0,544
6	—	—	—	—	0,530
7	—	—	—	—	0,296

Tabulka 3.10: Spearmanova korelace *attention vah* s RBF/NW kernelem, zprůměrovaná přes 8 hlav dané vrstvy a přes 200 instancí. Pomlčka značí, že daná vrstva u mělčího modelu neexistuje. Ve vrstvě 0 je *kernelová korelace* u vícevrstvých modelů nízká (u 8 vrstev dokonce mírně záporná), zatímco ve středních vrstvách skáče přes 0,8. Výjimkou je jednovrstvý model, jehož jediná vrstva musí zvládnout obojí, a proto má i zde vysokou hodnotu.

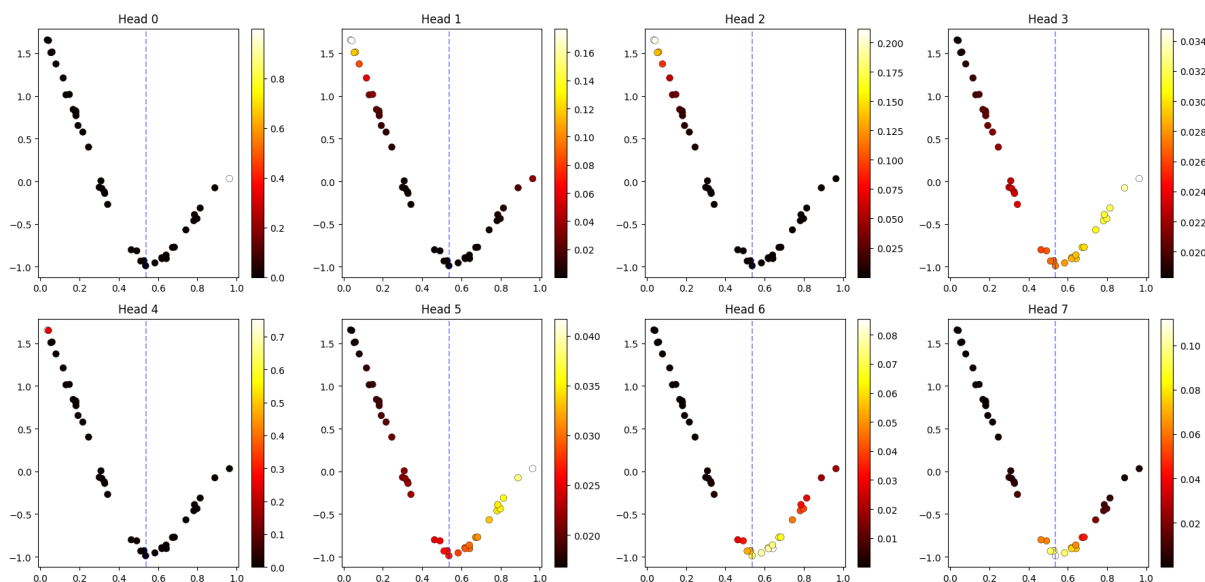
ní stihnout jak čtení *labelů*, tak *kernelové porovnání* současně. Teprve s tím, jak se hloubka modelu zvětšuje, si model může dovolit specializaci, kdy první vrstva jen posbírání *labely* a *kernelovou práci* nechá vrstvám pozdějším. Tuto specializaci na úrovni jednotlivých hlav ilustruje obrázek 3.12. Hlavy uvnitř jedné vrstvy nejsou zaměnitelné, každá pokrývá trochu jiný aspekt vstupu, což je mimo jiné známá vlastnost *multihead-attention* mechanismu v transformerech.



Obrázek 3.12: Osmivrstvý model, rozklad na jednotlivé hlavy. Vlevo: *kernelová korelace*, vpravo: *labelová korelace*, obojí pro každou z 8 hlav zvlášť napříč všemi 8 vrstvami. Ve vrstvě 0 se hlavy rozcházejí, některé silně čtou *labely* (vpravo nahoře), jiné ne, zatímco ve středních vrstvách se většina hlav sbíhá k vysoké *kernelové korelaci*. Rozdílné chování hlav uvnitř jedné vrstvy je motivací pro test důležitosti hlav v sekci 3.3.2.

Toto rozdílné chování hlav, kdy každá sleduje jinou část kontextu, lze nahlédnout i prostorově. Obrázek 3.13 zobrazuje, kam se dívá každá z 8 hlav jednovrstvého modelu. V tomto případě usuzujeme, že je dobré pro čtenáře popsat, co tento obrázek znázorňuje a co znamená. Vysvětlíme tedy nejprve, jak takový graf vzniká a jak ho číst. Zafixujeme jeden testovací bod, v naší realizaci $x^* = 0,54$, a ptáme se, mezi které kontextové body daná hlava rozdělí svou *attention váhu* při predikci v tomto bodě. Každý kontextový bod vyneseme do roviny jako jednu tečku, kde vodorovná souřadnice je jeho poloha X_j a svislá souřadnice jeho pozorovaná hodnota Y_j . Tečku pak obarvíme podle *attention váhy*, kterou tomuto

bodů hlava přidělila, tj. čím světlejší tečka, tím větší váha. Graf tedy čteme tak, že sledujeme, kde se světlé tečky hromadí. Pokud se hromadí ve svislém pruhu kolem x^* , hlava vybírá body podle blízkosti ve vstupním prostoru a hodnota Y_j ji nezajímá. Pokud se naopak hromadí nahoře nebo dole, hlava vybírá body podle jejich hodnoty Y_j , tedy čte *labely*. Kdyby tedy hlava prováděla čisté *kernelové porovnání*, viděli bychom svislý pruh kolem $x^* = 0,54$ nezávisle na Y_j . Skutečnost podle tohoto obrázku je ale jiná. Hlavy míří na různé oblasti roviny. *Head 1* a *2* se dívají na body s nízkým X a vysokým Y (vlevo nahoře). *Head 3*, *5*, *6* a *7* se soustředí u naopak na body s vysokým X a nízkým Y (vpravo dole). Několik hlav (*0*, *4*) dokonce degeneruje na jeden nebo málo bodů. Může to být právě již zmíněný *attention sink*, který jsme popsali v sekci 3.1.1. Připomeňme, že naše kontextové body pocházejí z jedné konkrétní GP realizace, která na tomto intervalu nejprve klesá a poté opět roste, tvoří tedy údolí s minimem kolem $x^* = 0,54$. Body vlevo tak mají vysoké hodnoty Y_j a body v pravé dolní části nízké. Když se tedy hlava dívá do levého horního rohu, vybírá si body s vysokým Y_j , a když do pravého dolního, vybírá si body s nízkým Y_j . Jednotlivé hlavy si vybírají body podle jejich hodnot Y_j . *Attention* tedy závisí i na *labelu*, nikoli jen na vzdálenosti ve vstupním prostoru, což je prostorová obdoba nenulové *labelové korelace* z tabulky 3.10.



Obrázek 3.13: *Iso-weight rozklad* jednovrstvého modelu na 8 hlav. Pro pevný testovací bod $x^* = 0,54$ (modrá přerušovaná čára) je každý kontextový bod vyneseno do roviny (X_j, Y_j) a obarven *attention vahou* dané hlavy (světlejší = vyšší). *Kernelové porovnání* by koncentrovalo váhu kolem x^* nezávisle na Y_j . Místo toho různé hlavy míří do různých rohů roviny (X, Y) , tedy na různé hodnoty Y_j , což je přímý vizuální doklad, že *attention* závisí i na *labelu*.

Klíčový závěr korelační analýzy bychom mohli zformulovat následujícím způsobem. Nelze tvrdit, že PFN provádí čisté *kernel smoothing*. Což je mimo jiné i jeden z výsledků předchozí analýzy v sekci 3.1.3, kde jsme ukázali, že *attention váhy* nekorelují s RBF kernelem a že se predikce PFN zásadně liší od Nadaraya-Watsonova estimátoru. Zde jsme ke stejnému závěru dospěli z opačné strany, tedy pohledem dovnitř jednotlivých vrstev. V žádné vrstvě není *labelová korelace* nulová, model v každém kroku míchá informaci o vstupu X i o hodnotě Y . Avšak střední vrstvy *kernel-like chování* výrazně preferují, ale nikdy ho nedosahují v čisté podobě. Toto pozorování je plně v souladu s Naglerovým teoretickým výsledkem, ke kterému se vrátíme v následující části.

3.3.2 Které hlavy model skutečně potřebuje: head-knockout

Korelační analýza z předchozí části má jedno zásadní omezení, *attention vzor* totiž pouze popisuje. Ignoruje *value vektory*, FFN i reziduální tok, a proto z ní nelze usuzovat na skutečnou funkční roli hlavy v celém modelu. Navíc Nagler ve své práci dokázal (Věta 6.3), že *softmax attention* vždy míchá X_j i Y_j . „Čistý kernel“ je tedy principiálně nedosažitelný a samotná korelace hlavy nám nemusí nutně říct, zda je pro predikci důležitá. Potřebujeme proto metriku, která místo popisu měří skutečný dopad hlavy na výstup. Takovou vlastnost budeme nazývat *kauzální důležitost*. Neptáme se, s čím *attention hlavy* souvisí, ale co se stane, když hlavu z modelu skutečně odebereme. Jde o princip tzv. *ablace*, tedy o cílený zásah do modelu a změření jeho následku. Teprve dopad tohoto zásahu na predikci prozradí, zda byla komponenta pro výpočet nutná, nebo s ním jen náhodně korelovala.

Tuto myšlenku realizuje metoda *head-knockout* (doslova „vyřazení hlavy“). Připomeňme, že každá z 8 *attention hlav* vrstvy pracuje samostatně. Každá hlava si spočítá vlastní výstup, tedy svou vlastní váženou kombinaci kontextových bodů. Těchto 8 dílčích výstupů se pak musí složit dohromady do jedné společné reprezentace, se kterou dál pracuje zbytek sítě. Právě toto zajišťuje naučená výstupní projekce $\mathbf{W}_{\text{out}}^{(h)}$, tj. matice, přes kterou se přičítá příspěvek h -té hlavy do společné reprezentace. Podstatné je, že jiná cesta neexistuje. Cokoli hlava spočítala, dostane se ke zbytku modelu jedinečně vynásobením touto maticí a přičtením ke společné reprezentaci. Konkrétní hlavu proto můžeme odpojit velmi jednoduše, stačí její výstupní projekci vynulovat ($\mathbf{W}_{\text{out}}^{(h)} = \mathbf{0}$). Hlava pak sice dál počítá, ale její výsledek se nikam nepřičte. Ostatní hlavy, dopředné sítě i reziduální spoje přitom zůstanou beze změny. Poté modelem znovu proženeme stejný vstup a měříme, o kolik se zhorší shoda predikce s GP posteriorem. Rozdíl oproti původnímu kompletnímu modelu je pak čistým příspěvkem právě té jedné hlavy. Takto postupně vyřadíme každou hlavu každé vrstvy, vždy právě jednu, zatímco všechny ostatní zůstávají aktivní. U modelu s L vrstvami tedy provedeme $8L$ samostatných *knockoutů*, u osmivrstvého modelu například 64.

Klíčové je použít *relativní* metriku:

$$\text{rel} = \frac{\text{MSE}(\text{knockout}, \mu_{\text{GP}}) - \text{MSE}(\text{full}, \mu_{\text{GP}})}{\text{MSE}(\text{full}, \mu_{\text{GP}})}.$$

Baseline chybou zde rozumíme MSE kompletního modelu vůči GP posterioru, tj. jmenovatel tohoto zlomku. Používáme tuto metriku, jelikož absolutní rozdíl MSE by mohl být zavádějící. Hluboké modely totiž aproximují GP posterior s velmi malou *baseline chybou* (řádově 10^{-4}), takže jejich absolutní zhoršení není výrazné pouze kvůli této architektonické vlastnosti, ať je hlava důležitá, nebo ne. Relativní metrika naproti tomu měří zhoršení v násobcích vlastní *baseline chyby* modelu, a proto je mezi modely srovnatelná. Velká hodnota *rel* znamená, že hlava je důležitá pro aproximaci GP posterioru. Hodnota blízko nuly naopak značí, že model bez ní funguje prakticky stejně. Test provedeme na 50 instancích s kontextem $n_{\text{ctx}} = 40$. Výsledky shrnuje tabulka 3.11 a obrázek 3.14.

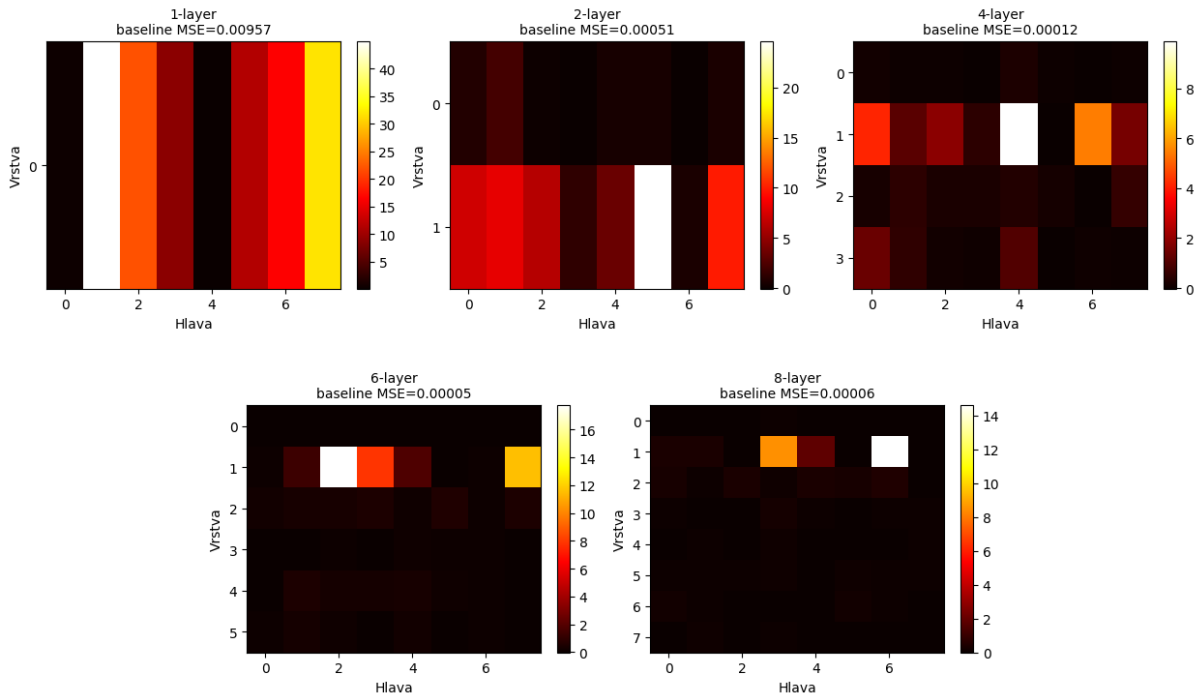
Diskuze:

Obrázek 3.14 ilustruje jasnou strukturu, kauzálně důležité hlavy se koncentrují v raných vrstvách. Je vidět, že dominantní hlava je u každého modelu ve vrstvě 0 nebo 1. To potvrzuje hypotézu o rozdělení práce z korelační analýzy. U hlubokých modelů je absolutní dopad *knockoutu* malý a predikce zůstávají blízko GP posterioru. Avšak odebrání nejdůležitější hlavy zhorší predikci relativně k vlastní *baseline chybě*, a to napříč všemi hloubkami, jak je vidět v tabulce 3.11. Model se bez ní nezhroutí a jeho predikce zůstává použitelná. Tmavá většina políček teplotní mapy zároveň ukazuje, že v hlubokých modelech se najdou i hlavy, jejichž odebrání predikci téměř nezmění. Jednovrstvý model je nejkřehčí, což nás nepřekvapuje, jelikož má jen jednu vrstvu s 8 hlavami a většina z nich je pro predikci důležitá. Výjimkou jsou jen degenerované hlavy 0 a 4.

Poznamenejme, že z výsledků přes všech 50 instancí bereme medián, nikoli průměr. Na většině instancí je totiž chyba PFN vůči GP posterioru nepatrná (řádově zhruba 10^{-4}). Na několika málo instancích

Model	Nejdůležitější (vrstva, hlava)	baseline MSE	max rel Δ MSE
1 vrstva	(0, 1)	0,00957	44,9×
2 vrstvy	(1, 5)	0,00051	24,6×
4 vrstvy	(1, 4)	0,00012	9,9×
6 vrstev	(1, 2)	0,00005	17,7×
8 vrstev	(1, 6)	0,00006	14,6×

Tabulka 3.11: Kauzální důležitost hlav podle *head-knockoutu*, měřená relativně k *baseline chybě* modelu. Uvedena je nejdůležitější hlava každého modelu, jeho *baseline MSE* vůči GP a maximální relativní zhoršení po vynulování této hlavy (např. 44,9× znamená 44,9násobek *baseline chyby*). Agregováno mediánem přes 50 instancí, $n_{\text{ctx}} = 40$.



Obrázek 3.14: *Head-knockout* pro všech pět modelů. Řádky teplotní mapy jsou vrstvy, sloupce hlavy, barva políčka udává relativní zhoršení $\text{rel} = (\text{MSE}_{\text{knockout}} - \text{MSE}_{\text{full}}) / \text{MSE}_{\text{full}}$ po vynulování právě té jedné hlavy. Světlejší políčko znamená větší zhoršení, tedy důležitější hlavu. Každý model má vlastní barevnou škálu, jinak by hluboké modely s malou *baseline chybou* byly celé tmavé. Kauzálně důležité hlavy se koncentrují do raných vrstev (vrstva 0–1) napříč všemi modely. Jednovrstvý model je nejkřehčí (až 44,9×), ale i vícevrstvé modely nesou odebrání jediné hlavy zhruba 10 až 25× vlastní *baseline chyby* podle tabulky 3.11.

ale může být o řády vyšší. Průměr by tak byl zcela ovlivněn těmito *outliery* a mezi modely by kolísal o dva řády už při pouhé změně *seedu*. Medián tuto křehkost odstraňuje a dává stabilní a mezi modely srovnatelný obraz. Což mimochodem je známý jev ze statistiky, že medián je robustnější charakteristika než průměr (viz např. proč se vždy uvádí medián mezd místo průměru)

Podstatné je, že tento obraz nám korelační analýza ukázat nemohla. Hlava může s kernelem korelovat silně, a přesto být nahraditelná. Jiná může korelovat slabě, a přesto být pro predikci důležitá.

Head-knockout tak korelační obraz nejen doplňuje, ale v jistém smyslu ho i koriguje. Tímto usuzujeme, že funkční roli hlavy určuje až její skutečný dopad na predikci, nikoli tvar jejího *attention* vzoru.

3.3.3 Rozklad chyby a role hloubky

Celou tuto sekci jsme zkoumali modely s různým počtem vrstev. Předchozí dvě části této sekce zkoumaly, co vrstvy dělají. Nyní bychom se mohli ptát, kolik jich vlastně potřebujeme, tedy jak hloubka ovlivňuje samotnou přesnost aproximace GP posterioru. K tomu použijeme rozklad střední kvadratické chyby na dvě složky, který přímo navazuje na *bias-variance* rozklad (2.5) z teoretické kapitoly. Pro každý model měříme MSE vůči GP posterioru při rostoucím počtu kontextových bodů $n \in \{5, 10, 20, 40, 80, 120\}$ (200 instancí pro každé n) a naměřené hodnoty proložíme modelem

$$\text{MSE}(n) = \text{bias}^2 + \frac{c}{n}.$$

Člen bias^2 zde představuje tzv. *neredukovatelnou chybu* (anglicky *irreducible*), tj. tu část chyby, která nezmizí ani při nekonečném kontextu ($n \rightarrow \infty$). Připomeňme si, že toto je jedna z vlastností, kterou Nagler popisuje u transformeru, jak jsme již zmiňovali v sekci 2.4.4, bias transformeru totiž obecně nemůže úplně zmizet, zatímco jeho rozptyl s počtem dat klesá. Člen c/n je naopak rozptyl, který s přibývajícím daty klesá. Odhadnuté koeficienty jsou v tabulce 3.12, proložení pak na obrázku 3.15.

Model	bias^2	c (rozptyl)
1 vrstva	0,00476	0,097
2 vrstvy	-0,00152	0,068
4 vrstvy	-0,00089	0,035
6 vrstev	-0,00076	0,029
8 vrstev	-0,00065	0,026

Tabulka 3.12: Rozklad MSE vůči GP posterioru na $\text{bias}^2 + c/n$. Jednovrstvý model má jako jediný zřetelně nenulový bias^2 . Od dvou vrstev výše je neredukovatelná chyba prakticky nulová a hloubka už jen snižuje rozptylový člen c . Poznamenejme dále, že mírně záporné odhady bias^2 u hlubších modelů nemají věcný význam. Proložení totiž nevynucuje, aby bias^2 vyšel nezáporný, a když je skutečná hodnota blízká nule, může odhad vlivem šumu klesnout těsně pod nulu. Tyto hodnoty tedy prakticky znamenají nulový neredukovatelný bias.

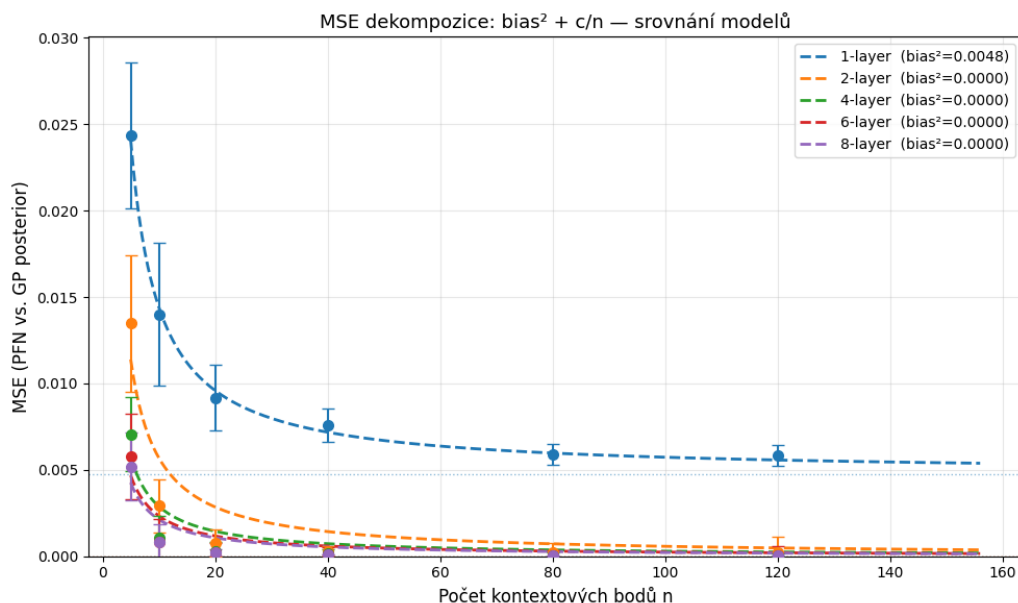
Diskuze:

Grafy na obrázku 3.15 a tabulka 3.12 nám ukazují následující situaci. Za prvé, jednovrstvý model má jako jediný zřetelně nenulový $\text{bias}^2 = 0,00476$. To je přímé empirické potvrzení Naglerova teoretického výsledku ze sekce 2.4.4. Jedna vrstva nestačí k asymptotickému přiblížení GP posterioru, a proto v ní zůstává neredukovatelná chyba, kterou nelze odstranit žádným množstvím kontextových dat. Od dvou vrstev výše je $\text{bias}^2 \approx 0$. Dvě vrstvy už tedy stačí k tomu, aby se model při dostatku dat přiblížil GP posterioru libovolně přesně.

Za druhé, hloubka působí především na rozptylový člen c , který monotónně klesá:

$$c_{1L} = 0,097 > c_{2L} = 0,068 > c_{4L} = 0,035 > c_{6L} = 0,029 > c_{8L} = 0,026.$$

Přidání vrstev tedy zrychluje konvergenci, hlubší model dosáhne stejné přesnosti z menšího kontextu. Zlepšení se však postupně sytí, mezi šesti a osmi vrstvami se c už prakticky nemění. Z praktického hlediska to znamená, že přibližně šest vrstev představuje bod, za kterým další prohlubování sítě nepřináší téměř žádný zisk v přesnosti.



Obrázek 3.15: Rozklad chyby na bias a rozptyl pro všech pět modelů. Vyneseno je MSE predikce PFN vůči GP posterioru jako funkce počtu kontextových bodů n . Body jsou naměřené hodnoty (chybové úsečky $\pm 1 \text{ std}$ přes instance), přerušované křivky jsou jejich proložení tvarem $\text{bias}^2 + c/n$. Jednovrstvý model (modrá) se s rostoucím n neustálí na nule, ale na nenulové hodnotě $\text{bias}^2 \approx 0,0048$ (tečkovaná čára), jeho chyba tedy nezmizí ani s nekonečným kontextem. Hlubší modely naopak klesají prakticky k nule a rozdíly mezi 4, 6 a 8 vrstvami jsou už zanedbatelné.

3.3.4 Shrnutí

Systematické měnění hloubky odhalilo, že PFN organizuje svůj výpočet takovým způsobem, že každá komponenta se zaměří na něco jiného, tj. jistým způsobem si komponenty rozdělují práci během výpočtů.

Korelační analýza ukázala specializaci vrstev. Rané vrstvy čtou *labels* (nejvyšší *labelová* a nejnižší *kernelová korelace*), střední vrstvy se chovají *kernel-like* a poslední vrstva opouští *kernelové porovnávání* ve prospěch sestavení predikce. Současně však platí, že žádná vrstva nedělá čistý *kernel smoothing*. V každém kroku se míchá informace o X i o Y , přesně jak předpovídá Naglerova věta o *softmax attention*.

Head-knockout tento popisný obraz doplnil z jiné perspektivy. Důležité hlavy se koncentrují do raných vrstev a jednovrstvý model je nejkřehčí. Avšak i v hlubokých modelech odebrání nejdůležitější hlavy znatelně zvedne chybu modelu, byť se tam najdou i hlavy postradatelné.

Rozklad chyby nakonec vysvětlil, proč modelu jedna vrstva nestačí. Jednovrstvý model má jako jediný nenulový $\text{bias}^2 \approx 0,0048$, zatímco od dvou vrstev a výše je neredukovatelná chyba nulová a hloubka už jen snižuje rozptylový člen c .

Dohromady nám tato pozorování dávají jasný obraz o tom, jak PFN uvnitř funguje. Model si rozděluje práci mezi vrstvy: rané vrstvy sbírají *labels*, střední vrstvy provádějí *kernelové porovnávání* a poslední vrstva sestavuje konečnou predikci. Žádná vrstva přitom nedělá prostý *kernel smoothing*. Čím je model hlubší, tím je toto rozdělení práce zřetelnější.

3.4 Hloubka jako iterace řešiče: PFN versus gradientní sestup

V předchozí sekci jsme ukázali, co jednotlivé vrstvy dělají. Tam jsme ukázali, že rané vrstvy čtou *labels*, střední vrstvy porovnávají vstupní vzdálenosti a hloubka odstraňuje neredukovatelný bias jednovrstvého modelu. Zůstává však hlubší otázka, proč vlastně přidávání vrstev pomáhá a jaký výpočet odpovídá jedné vrstvě. Připomeňme, že GP *posterior mean* (tj. střední hodnota posteriorního rozdělení, kterou GP vrací jako svou predikci) není nic jiného než řešení lineární soustavy

$$\mu(x^*) = k(x^*, \mathbf{X}) \boldsymbol{\alpha}, \quad \boldsymbol{\alpha} = (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y},$$

tedy inverze matice $\mathbf{K} + \sigma^2 \mathbf{I}$. Tato inverze se dá řešit iterativně. Přirozeně se proto nabízí hypotéza, že hloubka transformeru odpovídá počtu iterací takového řešiče. Tato myšlenka má i teoretickou oporu. Von Oswald et al. [12] ukázali, že jedna vrstva transformeru s lineárním *attention* umí ve svých vahách zakódovat přesně jeden krok gradientního sestupu nad kontextovými daty. Model s L vrstvami pak odpovídá L takovým iteracím. V této sekci tuto hypotézu kvantitativně otestujeme na PFN transformeru.

Nejjednodušším iterativním řešičem je gradientní sestup (dále jen GD) na kvadratické ztrátě

$$\frac{1}{2} \boldsymbol{\alpha}^\top (\mathbf{K} + \sigma^2 \mathbf{I}) \boldsymbol{\alpha} - \mathbf{y}^\top \boldsymbol{\alpha},$$

jehož iterace zní

$$\boldsymbol{\alpha}^{(0)} = \mathbf{0}, \quad \boldsymbol{\alpha}^{(t+1)} = \boldsymbol{\alpha}^{(t)} + \eta (\mathbf{y} - (\mathbf{K} + \sigma^2 \mathbf{I}) \boldsymbol{\alpha}^{(t)}).$$

Tyto iterace mají i druhý výklad. Označme pro přehlednost $\mathbf{A} = \mathbf{K} + \sigma^2 \mathbf{I}$. Stejně jako se pro číslo $|q| < 1$ sčítá geometrická řada $\sum_k q^k = (1 - q)^{-1}$, platí pro matici rozvoj

$$\mathbf{A}^{-1} = \eta \sum_{k=0}^{\infty} (\mathbf{I} - \eta \mathbf{A})^k,$$

kterému se v tomto kontextu říká *Neumannova řada*. Jde o maticovou obdobu geometrické řady, kde roli kvocientu hraje matice $\mathbf{I} - \eta \mathbf{A}$. Řada konverguje, pokud $\|\mathbf{I} - \eta \mathbf{A}\| < 1$, což u naší pozitivně definitní matice zajistí dostatečně malý krok η . Vztah ke gradientnímu sestupu získáme rozepsáním iterací z nulového počátku, čímž dostaneme

$$\boldsymbol{\alpha}^{(t)} = \eta \sum_{k=0}^{t-1} (\mathbf{I} - \eta \mathbf{A})^k \mathbf{y},$$

tedy přesně t -tý částečný součet Neumannovy řady aplikovaný na $\mathbf{y}$. Jeden krok GD tedy znamená jeden přičtený člen řady, a proto budeme oba pohledy volně zaměňovat. Rychlost konvergence řídí podmíněnost soustavy

$$\kappa = \frac{\lambda_{\max}(\mathbf{K}) + \sigma^2}{\lambda_{\min}(\mathbf{K}) + \sigma^2},$$

kde $\lambda_{\max}, \lambda_{\min}$ jsou největší a nejmenší vlastní čísla kernelové matice $\mathbf{K}$. Chyba GD klesá v nejhorším případě jako ρ^t , kde $\rho = (\kappa - 1)/(\kappa + 1)$ a t je počet kroků. Obyčejný GD tak potřebuje k dané přesnosti řádově $O(\kappa)$ kroků, tj. čím hůře je soustava podmíněná, tím víc iterací je potřeba. Odtud plyne i následující hypotéza, kterou bychom chtěli otestovat. Pokud každá vrstva PFN transformeru odpovídá jednomu kroku iterativního řešiče, větší počet vrstev bychom potřebovali pro hůře podmíněné úlohy, tj. úlohy, kde je κ velké.

K ověření této hypotézy potřebujeme kontrolovaně měnit κ , aniž bychom měnili cokoli jiného. Toho dosáhneme přes šum σ^2 a k tomu využijeme následujícího triku. Numericky totiž platí $\lambda_{\min}(\mathbf{K}) \approx 0$, takže z rovnice definice pro κ plyne, že $\kappa \approx 1 + \lambda_{\max}(\mathbf{K})/\sigma^2$ a samotný šum nastavuje podmíněnost prakticky

přímo. Pracujeme proto se dvěma sadami pěti modelů (1, 2, 4, 6 a 8 vrstev), které se liší výhradně šumem, na kterém byly trénovány i testovány: režim *Easy* ($\ell = 0,3$, $\sigma^2 = 0,5$) a režim *Hard* ($\ell = 0,3$, $\sigma^2 = 0,1$). Na rozdíl od předchozích sekcí, kde jsme používali jediný model trénovaný na náhodných hyperparametrech, je zde každá sada trénována přesně na svých testovacích hyperparametrech. Měříme tedy výkon na trénovacím rozdělení, nikoli generalizaci. Všechny pět hloubek v každé sadě bylo trénováno stejných 300 epoch, takže výsledky by neměly být zkreslené nevyrovnaným tréninkem. Na rozdíl od sekce 3.2 zdejší šesti- a osmivrstvé modely konvergovaly i pro 300 epoch, protože opět trénujeme modely na konkrétních hodnotách hyperparametrů. Výslednou podmíněnost obou režimů shrnuje tabulka 3.13.

Režim	ℓ	σ^2	κ ($n=40$)	$\rho = \frac{\kappa-1}{\kappa+1}$
Easy	0,3	0,5	≈ 50	$\approx 0,96$
Hard	0,3	0,1	≈ 245	$\approx 0,99$

Tabulka 3.13: Podmíněnost obou režimů, průměr přes 50 instancí při $n = 40$. Snížení šumu z 0,5 na 0,1 zřetelně zhoršuje podmíněnost. Veličina ρ udává, o kolik se v nejhorším případě zmenší chyba gradientního sestupu za jeden krok.

3.4.1 Hloubka snižuje chybu, ale ne jako obyčejná Neumannova řada

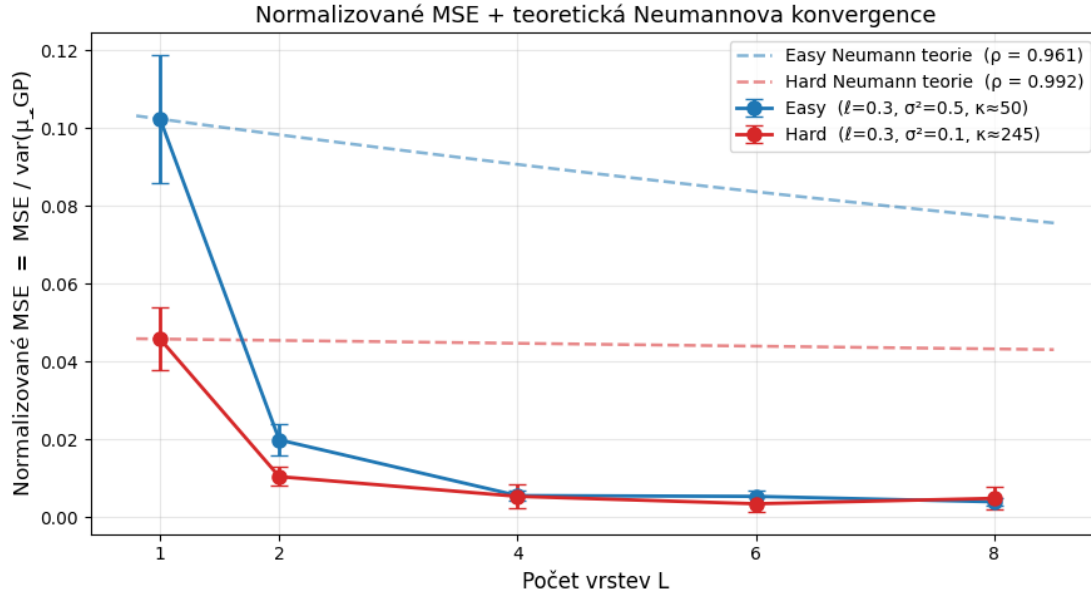
Nejprve ověříme základní předpoklad, že hloubka modelu vůbec zlepšuje aproximaci. Empirický pokles chyby zároveň porovnáme s teoretickou křivkou ρ^L obyčejného iterativního řešiče. Pro každý model a každou hloubku měříme na 200 sdílených instancích ($n_{\text{ctx}} = 40$, $n_{\text{test}} = 10$) tzv. normalizované MSE

$$\text{nMSE} = \frac{\text{MSE}(\hat{\mu}_{\text{PFN}}, \mu_{\text{GP}})}{\text{var}(\mu_{\text{GP}})},$$

tedy chybu predikce PFN vůči přesnému GP posterioru dělenou rozptylem toho, co odhadujeme (GP *posterior mean* přes testovací body). Záměrně za normalizační jmenovatel bereme $\text{var}(\mu_{\text{GP}})$, a nikoli $\text{var}(y)$. Pozorované hodnoty y totiž obsahují šum, který *posterior mean* neobsahuje, a normalizace šumem by uměle zkreslila výsledek. Empirické hodnoty spolu s teoretickou křivkou ρ^L zachycuje obrázek 3.16. Plná čára v něm ukazuje naměřenou chybu PFN jako funkci hloubky L , přerušovaná čára pak hranici ρ^L , tedy jak by chyba klesala, kdyby každá vrstva provedla přesně jeden krok obyčejného řešiče.

Diskuze:

Z obrázku 3.16 plynou dvě pozorování. Za prvé, hloubka modelu skutečně a výrazně zlepšuje aproximaci. Normalizované MSE u obou režimů prudce klesá, přičemž největší pokles připadá na přechod z jedné na čtyři vrstvy, pak se pokles ustálí. Za druhé, empirický pokles je mnohem rychlejší než teoretická křivka ρ^L . Připomeňme, že křivka ρ^L odpovídá nejhoršímu případu, tedy nejpomalejšímu poklesu, jaký teorie připouští. Pro obě hodnoty ρ z tabulky 3.13 předpovídá jen velmi mírný pokles chyby, PFN se však přiblíží GP posterioru už za dvě až čtyři vrstvy. Z toho usuzujeme, že naivní srovnání „vrstva = jeden krok Neumannovy řady“ tedy neplatí. Buď dělá jedna vrstva mnohem víc než jeden krok, anebo řešič, který PFN implementuje, není obyčejný gradientní sestup. Abychom mezi těmito možnostmi rozlišili, potřebujeme jiné srovnání, které zavedeme v sekci 3.4.2.

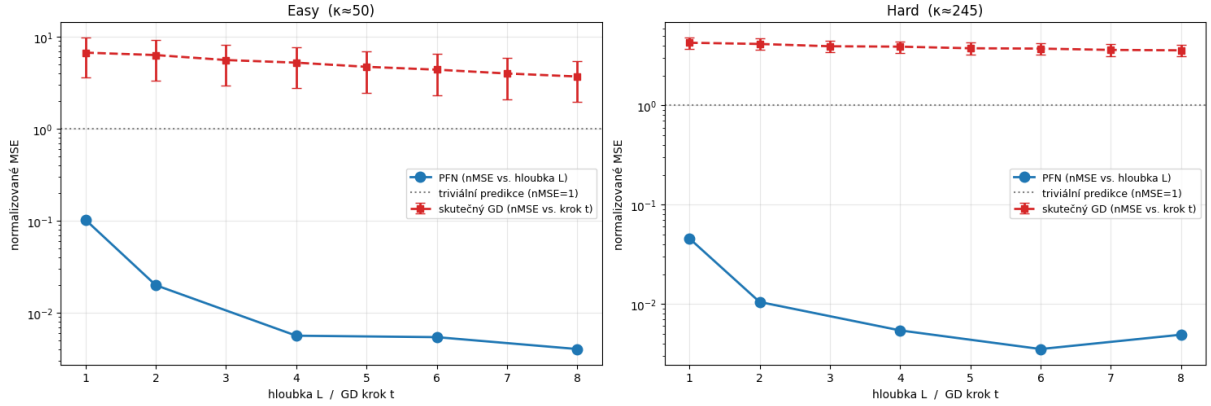


Obrázek 3.16: Normalizované MSE jako funkce hloubky L pro oba režimy (plné čáry, chybové úsečky ± 1 směrodatná chyba) proti teoretické křivce ρ^L , tedy nejpomalejšímu poklesu, který teorie gradientnímu sestupu v nejhorším případě předpovídá (přerušované čáry). Chyba prudce klesá s hloubkou, u *Easy* z 0,10 na 0,004, u *Hard* z 0,046 na 0,005, a hlavní zisk se odehrává mezi jednou a čtyřmi vrstvami. Empirický pokles je ovšem mnohem rychlejší než teoretická křivka, už 2–4 vrstvy dosáhnou přesnosti, kterou by Neumannova řada nedala ani po desítkách kroků.

3.4.2 Srovnání s gradientním sestupem v predikčním prostoru

V předchozí sekci 3.4.1 jsme narazili na zdánlivý paradox, a to že PFN konverguje rychleji, než Neumannova řada dovoluje. Tento paradox má ale jednoduché vysvětlení. Srovnávali jsme dvě různé veličiny. Míra $\rho = (\kappa - 1)/(\kappa + 1)$ popisuje, jak rychle konverguje samotný vektor koeficientů α . Gradientní sestup totiž v každém kroku upravuje přímo souřadnice vektoru $\alpha^{(l)}$ a jeho chybou se rozumí vzdálenost tohoto vektoru od přesného řešení α . Této chybě budeme říkat chyba v α -prostoru, protože ji měříme přímo na koeficientech. Nás ale koeficienty samy o sobě nezajímají. Zajímá nás predikce $\mu(x^*) = k(x^*, \mathbf{X}) \alpha$, která z nich teprve vznikne, a chybu proto měříme až na ní, tedy v *predikčním prostoru*. Tyto dva prostory nejsou totéž. Predikce je *kernelově* vážená kombinace koeficientů α a toto vážení tlumí ty složky, ve kterých gradientní sestup konverguje pomalu. Důvod jen naznačíme. Chybu koeficientů si můžeme rozložit do báze tvořené vlastními vektory kernelové matice $\mathbf{K}$. Gradientní sestup konverguje pomalu právě ve směrech vlastních vektorů s malým vlastním číslem λ_i . Do predikce však tyto směry vstupují s vahou zhruba úměrnou právě λ_i , takže se jejich zbývající chyba v predikci téměř neprojeví. Srovnávat empirickou predikční chybu s křivkou ρ^L proto není zcela korektní, penalizuje totiž řešič za chyby, které se do predikce skoro nepromítanou.

Připomeňme, že v předchozím experimentu jsme žádný gradientní sestup nespouštěli. Empirickou chybu PFN jsme srovnávali pouze s teoretickou křivkou ρ^L , tedy s předpovědí nejhoršího případu, a to navíc pro chybu v α -prostoru. Korektní srovnání vyžaduje postavit proti PFN skutečně spuštěný gradientní sestup a měřit jeho chybu ve stejném predikčním prostoru jako u PFN. Aby případný rozdíl šel přičíst samotnému řešiči, spustíme ho na stejných 200 instancích jako v předchozím experimentu. Na každé z nich iterujeme výše uvedenou rovnici s optimálním fixním krokem $\eta = 2/(\lambda_{\max} + \lambda_{\min})$. Po



Obrázek 3.17: PFN versus skutečný gradientní sestup, obojí v predikčním prostoru, pro *Easy* (vlevo) a *Hard* (vpravo). Modrá: normalizované MSE PFN proti hloubce L . Červená: normalizované MSE gradientního sestupu proti počtu kroků t (stejně osy). Tečkovaná čára značí triviální predikci konstantním průměrem (nMSE = 1). PFN je hluboko pod triviální úrovní už při jedné vrstvě, zatímco skutečný gradientní sestup zůstává nad triviální predikcí i po osmi krocích.

PFN hloubka L	kroky GD, Easy ($\kappa \approx 50$)	kroky GD, Hard ($\kappa \approx 245$)
1	51	271
4	87	402
8	91	408

Tabulka 3.14: Efektivní počet kroků gradientního sestupu, který dosáhne stejné predikční chyby jako L -vrstvý PFN. Už jediná vrstva odpovídá desítkám kroků. Počet potřebných kroků navíc silně roste s podmíněností κ .

každém kroku t zrekonstruujeme predikci $\mu_{\text{GD}}^{(t)}(x^*) = k(x^*, \mathbf{X}) \alpha^{(t)}$ a spočítáme její normalizované MSE vůči GP posterioru, tedy přesně tu metriku, kterou u PFN vynášíme proti hloubce. Výsledek srovnání je na obrázku 3.17.

I v tomto případě usuzujeme, že je dobré pro čtenáře popsat, co obrázek 3.17 znázorňuje a jak ho číst. Obě křivky ukazují stejnou veličinu jako v předchozím experimentu, tedy normalizované MSE predikce vůči GP posterioru. Modrá křivka ji ukazuje pro PFN jako funkci počtu vrstev L . Červená křivka ji ukazuje pro gradientní sestup jako funkci počtu kroků t , vnesenou na stejné ose. Porovnáváme tedy jednu vrstvu PFN s jedním krokem GD přímo proti sobě. Tečkovaná vodorovná čára značí triviální predikci konstantním průměrem, které odpovídá nMSE = 1. Cokoli nad touto čarou je horší, než kdybychom vůbec nepredikovali.

Diskuze:

Obrázek 3.17 dává jednoznačnou odpověď, reálný gradientní sestup je pomalý i v predikčním prostoru. Jeho normalizované MSE zůstává po prvních osm krocích nad triviální hranicí, tj. hůř než predikce konstantním průměrem. PFN je naproti tomu pod touto hranicí už s jedinou vrstvou. Tím je paradox z předchozí části vysvětlen. PFN je rychlejší než gradientní sestup i v korektním srovnání. Abychom rozdíl vyčíslili, dopočítáme pro každou hloubku L tzv. efektivní počet kroků, tedy nejmenší počet kroků gradientního sestupu, po kterém jeho chyba klesne na úroveň L -vrstvého PFN. Výsledky jsou v tabulce 3.14.

Tabulka 3.14 ukazuje dvě věci najednou. Za prvé, jedna vrstva odpovídá řádově desítkám kroků obyčejného gradientního sestupu. Naivní hypotéza „vrstva = jeden krok“ je tedy vyvrácena o celé řády. Za druhé, počet potřebných kroků prudce roste s podmíněností, přesně jak předpovídá $O(\kappa)$ chování

gradientního sestupu. Naproti tomu PFN dosahuje srovnatelné přesnosti stejnými osmi vrstvami v obou režimech. Tuto asymetrii, kdy GD škáluje s κ a PFN nikoli, budeme nazývat κ -necitlivost. Právě ona může být náповědou, jaký řešič se PFN naučil.

Dále představíme jednu z možných interpretací jevu, který zde pozorujeme. V numerické matematice existuje standardní postup, jak konvergenci zbavit závislosti na κ , tzv. *preconditioning*. Vysvětleme nejprve, proč je obyčejný GD na špatně podmíněné soustavě pomalý. Velikost kroku η se musí přizpůsobit největšímu vlastnímu číslu, jinak by iterace divergovaly. Ve směrech s malými vlastními čísly se pak řešení posouvá jen nepatrně a právě kvůli nim je potřeba $O(\kappa)$ kroků. *Preconditioning* tento problém obchází. Soustavu $(\mathbf{K} + \sigma^2 \mathbf{I}) \boldsymbol{\alpha} = \mathbf{y}$ nejprve vynásobíme zleva vhodnou maticí $\mathbf{P}$ a řešíme ekvivalentní soustavu $\mathbf{P}(\mathbf{K} + \sigma^2 \mathbf{I}) \boldsymbol{\alpha} = \mathbf{P}\mathbf{y}$. Matice $\mathbf{P}$ se volí jako aproximace inverze, takže součin $\mathbf{P}(\mathbf{K} + \sigma^2 \mathbf{I})$ je skoro jednotková matice. Vlastní čísla nové soustavy pak leží blízko sebe. Podmíněnost κ spočtená pro tuto novou soustavu, budeme jí říkat efektivní podmíněnost, proto klesne k jedné a všechny směry konvergují zhruba stejně rychle. Počet potřebných kroků tím přestane záviset na původním κ . Přesně takové chování u PFN pozorujeme. Připomeňme navíc, že *attention váhy* závisejí na vstupních datech, takže model si podobnou transformaci může pro každý kontext sestavit sám. Kombinace κ -necitlivosti a rychlosti mnohonásobně převyšující gradientní sestup je proto charakteristickým znakem naučeného *preconditioned* řešiče, nikoli obyčejného gradientního sestupu. Stejný obraz předpovídá i teorie *in-context* učení, viz např. Ahn et al. [13] nebo již zmíněný von Oswald et al. [12].

3.4.3 Shrnutí

Tato sekce testovala hypotézu, že hloubka PFN odpovídá počtu iterací řešiče, který invertuje matici $\mathbf{K} + \sigma^2 \mathbf{I}$.

Hloubka skutečně a výrazně zlepšuje aproximaci GP posterioru, naprostá většina zisku přitom připadá na první čtyři vrstvy. Avšak se ukazuje, že naivní verze hypotézy „vrstva = jeden krok Neumannovy řady“ neplatí. Korektní srovnání se skutečným gradientním sestupem ukázalo, že už jediná vrstva odpovídá desítkám jeho kroků a s horší podmíněností tento počet dále roste, jak je vidět v tabulce 3.14. Hloubka PFN přitom zůstává stejná, model je vůči podmíněnosti prakticky necitlivý.

Dále jsme odvodili předpoklad, že kombinace této κ -necitlivosti a vysoké rychlosti je náznakem naučeného *preconditioned* řešiče, přesně jak pro transformery předpovídá teorie *in-context* učení. Dá se tedy říct, že PFN transformer implementuje κ -necitlivou iteraci, ve které jedna vrstva vykoná řádově tolik práce jako desítky kroků obyčejného gradientního sestupu.

3.5 Robustnost při misspecifikaci prioru

V této poslední sekci budeme zkoumat vlastnost, na které selhává většina Bayesovských metod, a tou je tzv. *misspecifikace prioru*. Situaci, kdy jsme prior apriorně odhadli špatně, tedy když skutečná data pocházejí z jiného rozdělení, než jaké prior předpokládá, nazýváme misspecifikací a tento pojem budeme v celé sekci používat. Metodě s takto špatně zvoleným priorem pak budeme říkat misspecifikovaná. Ve všech dosavadních experimentech pocházela data z GP s RBF kernelem, tedy přesně z té třídy modelů, kterou porovnávané metody předpokládaly. Vždy proto existoval správně nastavený GP (*oracle*), ke kterému jsme mohli PFN vztahovat. V praxi ale taková jistota neexistuje. Skutečná data nepřicházejí se štítkem, ze kterého kernelu byla vygenerována. Nyní proto pro PFN odstraníme i tuto poslední jistotu. Data budeme generovat z tzv. Matérnova kernelu (definice viz dále), zatímco všechny porovnávané metody budou předpokládat RBF. Skutečný generativní proces tak bude ležet mimo support prioru všech metod, včetně implicitního prioru PFN. Budeme tedy zkoumat, co se stane, když po PFN chceme predikovat mimo support jeho prioru. Bude nás zajímat o kolik se predikce zhorší a která

metoda za těchto podmínek selže nejméně. Připomeňme, že přesně touto situací se zabývala věta 2.4.2, podle které se PPD umí učit i tehdy, když skutečné rozdělení leží mimo support prioru. Tato sekce je tedy zároveň empirickým testem této teoretické vlastnosti.

Nejprve popíšeme uspořádání experimentu. Data generujeme z GP s Matérnovým kernelem s korelační délkou ℓ_{true} a šumem $\sigma^2 = 0,1$. Matérnův kernel je v praxi nejběžnější alternativou RBF a jeho tvar řídí parametr hladkosti ν . My volíme $\nu = 2,5$, pro který má kernel tvar

$$k_{\text{Matérn}}(x, x') = \left(1 + \frac{\sqrt{5}r}{\ell} + \frac{5r^2}{3\ell^2}\right) \exp\left(-\frac{\sqrt{5}r}{\ell}\right), \quad r = |x - x'|,$$

kde ℓ je opět korelační délka. Realizace GP s tímto kernelem jsou jen dvakrát diferencovatelné, tedy znatelně hrubší než nekonečně hladké realizace RBF kernelu. Právě proto se Matérnův kernel považuje za realističtější model skutečných dat. Nekonečná hladkost RBF je totiž pro reálné procesy příliš silný předpoklad [4]. Náš test misspecifikace tak napodobuje typickou praktickou situaci, kdy jsou data hrubší, než model předpokládá. Pro tento účel je Matérnův kernel ideální i z druhého důvodu. Pro $\nu \rightarrow \infty$ přechází v RBF, oba kernely jsou si tedy velmi podobné. Situace nebo omyl, kdy data pocházející z Matérnova kernelu modelujeme pomocí RBF, se proto může snadno přihodit i v praxi. Současně jsou oba kernely dost odlišné na to, aby tento omyl žádná volba hyperparametrů RBF nedokázala úplně napravit. Misspecifikace prioru zde tedy spočívá v rozdílu mezi skutečným a předpokládaným kernelem. Žádná z testovaných metod nemá k dispozici správnou třídu funkcí. Porovnáváme následující metody:

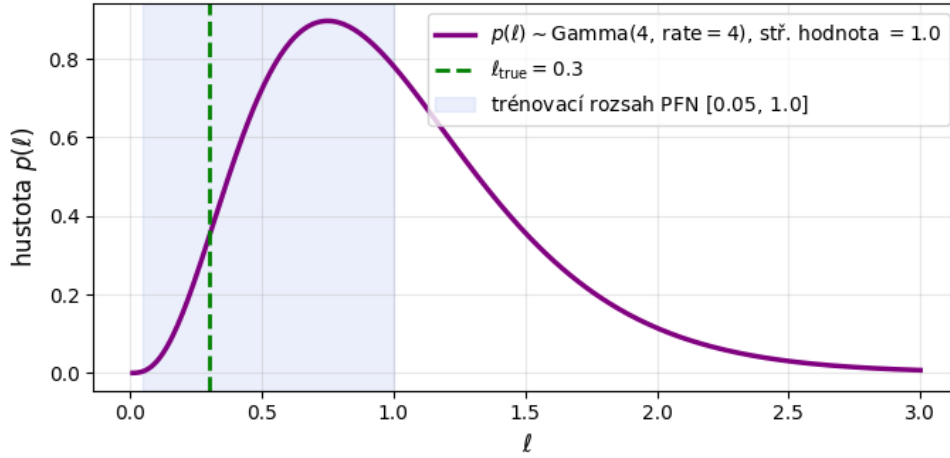
- *PFN*, náš šestivrstvý model s náhodnými hyperparametry ze sekce 3.2, jehož prior je implicitně daný trénovacím rozdělením (RBF, $\ell \sim \mathcal{U}(0,05, 1,0)$) a za běhu ho nelze nijak opravit,
- *GP_fixed*, GP s RBF kernelem a pevnou korelační délkou $\ell = 0,7$, tedy špatný kernel i špatná korelační délka a žádná adaptace,
- *GP_ML*, GP s RBF kernelem, který korelační délku odhaduje z dat maximalizací marginální věrohodnosti (Type II – MLE ze sekce 1.4.2),
- *GP_Bayes*, Bayesovský GP s RBF kernelem a priorem $p(\ell) \sim \text{Gamma}(4, 4)$, vzorkovaný metodou NUTS ze sekce 1.4.3,
- *GP_oracle*, referenční GP se správným Matérnovým kernelem i správnými hyperparametry.

Poznamenejme ještě, jak je nastavený prior metody *GP_Bayes*. Rozdělení $\text{Gamma}(4, 4)$ má střední hodnotu u $\ell = 1,0$ a většinu své pravděpodobnostní hmoty soustřeďuje kolem ní. Pravá hodnota $\ell_{\text{true}} = 0,3$ leží až v jeho levém chvostu, jak ukazuje obrázek 3.18. Prior tedy pravé hodnotě přiřazuje jen malou pravděpodobnost. *GP_Bayes* je tím pádem misspecifikovaný dvakrát. Za prvé předpokládá špatný kernel, stejně jako všechny ostatní metody. Za druhé jeho prior tvrdí, že správná korelační délka je nepravděpodobná, a metoda se od tohoto přesvědčení může odchýlit až na základě pozorovaných dat. Experiment se tím blíží reálné situaci, kdy prior volíme bez znalosti pravdy a zvolíme ho nešťastně.

Kvalitu predikce měříme dvěma metrikami. První je MSE predikce vůči skutečným hodnotám. Druhou je kalibrační metrika, které budeme říkat ΔNLL :

$$\Delta\text{NLL} = \text{NLL}_{\text{metoda}} - \text{NLL}_{\text{oracle}},$$

tedy o kolik je záporná log-věrohodnost predikcí dané metody horší než u *oracle*. *Oracle* má z definice $\Delta\text{NLL} = 0$ a čím blíže nule metoda je, tím lépe je kalibrována. Vysvětleme, v čem se obě metriky liší a proč potřebujeme obě. MSE hodnotí pouze bodovou predikci, tedy střed prediktivního rozdělení. Dvě metody se stejným středem, ale různě širokou nejistotou, mají proto stejné MSE. NLL naproti tomu



Obrázek 3.18: Prior $p(\ell) \sim \text{Gamma}(4, 4)$ metody GP_Bayes (fialová) se střední hodnotou u $\ell = 1,0$. Zelená přerušovaná čára značí pravou hodnotu $\ell_{\text{true}} = 0,3$, která leží v chvostu prioru. Modrý pás vyznačuje trénovací rozsah PFN $\ell \in [0,05, 1,0]$, pravá hodnota leží uvnitř něj.

hodnotí celé prediktivní rozdělení. Trestá nejen špatný střed, ale i špatně odhadnutou nejistotu, a to zejména příliš sebejisté intervaly, ze kterých skutečná hodnota vypadne. Pořadí metod se tak v obou metrikách může lišit, metoda s dobrým středem a přehnaně úzkou nejistotou dopadne v MSE dobře a v ΔNLL špatně. Právě ΔNLL přitom testuje to podstatné, smyslem Bayesovské inference je totiž vracet celou prediktivní nejistotu, nikoli jen bodový odhad. MSE pak slouží jako kontrola, že případná lepší kalibrace není zaplacená horší bodovou predikcí. Všechny výsledky průměrujeme přes 150 instancí, v každé predikujeme $n_{\text{test}} = 10$ testovacích bodů.

Než přejdeme k samotným testům, ujasňme si, co přesně zde misspecifikujeme. Priorem každé metody rozumíme její představu o tom, z jaké třídy funkcí data pocházejí. U GP metod je to kernel spolu s hodnotou nebo rozdělením hyperparametrů, u PFN je to implicitní prior daný trénovacím rozdělením. V tomto experimentu tedy nejde o špatně odhadnutou korelační délku, kterou by šlo z dat opravit. Špatně je samotná třída funkcí, Matérnovy realizace totiž žádná volba hyperparametrů RBF přesně nepopíše.

Jinou roli zde hraje i velikost kontextu. V sekcích 3.1 a 3.2 bylo vše správně specifikované a n měřilo, jak rychle PFN konverguje ke správnému posterioru. Zde je prior všech metod špatně a n měří, jak rychle se metody ze špatného prioru vzpamatují. S rostoucím n totiž likelihood postupně převáží prior. Adaptivní metody GP_ML a GP_Bayes si korelační délku mohou z dat znovu odhadnout, jejich handicap tedy s daty mizí. PFN svůj prior za běhu změnit nemůže a GP_fixed se nepřizpůsobuje vůbec. Nejzajímavější proto budou malé kontexty, kde prior dominuje a ukazuje se, čím prior škodí nejméně.

3.5.1 Vliv velikosti kontextu

V prvním testu zafixujeme $\ell_{\text{true}} = 0,3$ a měníme velikost kontextu $n \in \{5, 10, 20, 40, 64, 100, 128\}$. Ptáme se, jak rychle se jednotlivé metody s rostoucím množstvím dat vzpamatují ze špatného prioru. Výsledky shrnují tabulky 3.15 a 3.16 a obrázek 3.19. Horní panel obrázku ukazuje MSE jako funkci n , dolní panel ΔNLL . Tečkovaná čára v dolním panelu značí *oracle*, tedy nulu, ke které se chtějí všechny metody přiblížit.

Diskuze:

Z tabulek 3.15 a 3.16 plynou dvě pozorování. Za prvé, v MSE je PFN plně konkurenceschopné s adaptivními metodami GP_ML a GP_Bayes. Pro střední velikosti kontextu je dokonce nejlepší ze všech misspecifikovaných metod a pro velká n jsou všechny tři prakticky shodné, jak je vidět v tabulce 3.15. GP_fixed, který se datům nijak nepřizpůsobuje, zaostává za všemi výrazně a s rostoucím n se jeho odstup nezmenšuje. Za druhé, v kalibraci je obraz ještě víc jednoznačnější. PFN má nejmenší ΔNLL ze všech misspecifikovaných metod pro všechna $n \leq 40$ i pro $n = 100$, jak ukazuje tabulka 3.16. U velkých kontextů už mezi PFN a GP_ML není jednoznačné pořadí, GP_ML vede při $n = 64$ a $n = 128$. Toto chování má přirozené vysvětlení. Adaptivní metody musí korelační délku odhadnout z několika málo bodů a při malém kontextu ji přefitují, jejich prediktivní intervaly jsou pak příliš sebejisté. PFN naproti tomu marginalizuje přes celé trénovací rozdělení hyperparametrů, a proto vrací opatrnější a lépe kalibrovanou nejistotu. Přesně v režimu malých dat, kde na kalibraci záleží nejvíce, se tato vlastnost vyplácí.

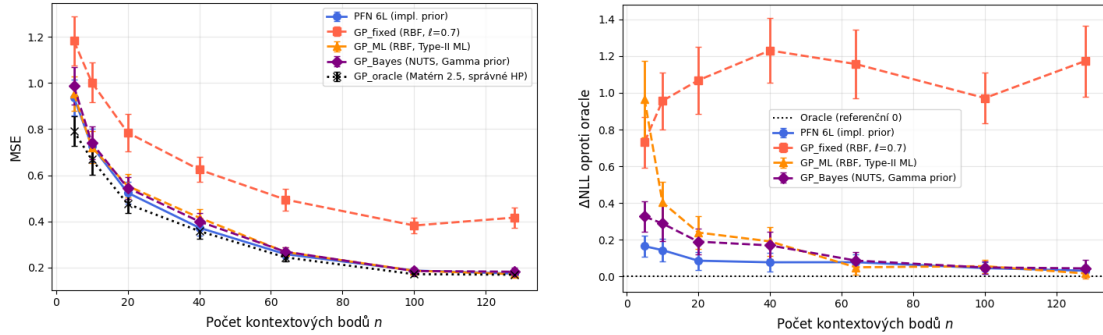
Za povšimnutí stojí i chování obou krajních metod. GP_fixed sice s rostoucím kontextem svou bodovou chybu snižuje, jeho ΔNLL ale podle tabulky 3.16 s daty neklesá, šířku prediktivních intervalů totiž určuje pevně zvolený kernel, na který žádné pozorování nesáhne. Naopak GP_Bayes je při malých kontextech kalibrovaný znatelně lépe než GP_ML, přestože jeho prior pravé hodnotě korelační délky téměř nevěří. Jediný rozdíl mezi nimi je přitom marginalizace, GP_ML sází vše na jediný bodový odhad korelační délky, GP_Bayes přes její nejistotu průměruje. Vidíme tedy stejný mechanismus, který pomáhá PFN, jen v menším měřítku.

n	PFN	GP_fixed	GP_ML	GP_Bayes	GP_oracle
5	0,93699	1,18172	0,95305	0,98592	0,79025
10	0,72713	1,00189	0,72144	0,73947	0,66794
20	0,52432	0,78485	0,55354	0,54532	0,47545
40	0,37256	0,62423	0,41614	0,39967	0,35816
64	0,25835	0,49483	0,26753	0,26891	0,24434
100	0,18719	0,38220	0,18626	0,18506	0,17135
128	0,17385	0,41661	0,17169	0,18200	0,16918

Tabulka 3.15: MSE vůči skutečným hodnotám jako funkce velikosti kontextu n při misspecifikaci kernelu ($\ell_{\text{true}} = 0,3$, průměr přes 150 instancí). PFN je pro $n \in \{5, 20, 40, 64\}$ nejlepší ze všech misspecifikovaných metod, pro velká n je s GP_ML a GP_Bayes prakticky shodné.

n	PFN	GP_fixed	GP_ML	GP_Bayes
5	+0,165	+0,732	+0,968	+0,326
10	+0,143	+0,956	+0,403	+0,288
20	+0,086	+1,068	+0,239	+0,190
40	+0,077	+1,231	+0,191	+0,169
64	+0,077	+1,157	+0,050	+0,086
100	+0,045	+0,972	+0,056	+0,049
128	+0,031	+1,174	+0,016	+0,044

Tabulka 3.16: Kalibrace ΔNLL vůči *oracle* jako funkce velikosti kontextu n ($\ell_{\text{true}} = 0,3$, průměr přes 150 instancí, *oracle* má z definice hodnotu 0). PFN je nejbližší *oracle* pro všechna $n \leq 40$ i pro $n = 100$, GP_ML jej mírně předstihne jen při $n = 64$ a $n = 128$.



Obrázek 3.19: Vliv velikosti kontextu při misspecifikaci kernelu (data z Matérn 2,5, $\ell_{\text{true}} = 0,3$). Vlevo: MSE jako funkce n , černá tečkovaná čára je GP_{oracle} . Vpravo: ΔNLL vůči $oracle$, tečkovaná čára značí nulu ($oracle$). Chybové úsečky ± 1 směrodatná chyba. PFN (modrá) se v MSE drží těsně u $oracle$ a v ΔNLL je pro malé a střední kontexty ze všech metod nejnižší.

3.5.2 Vliv síly misspecifikace

Ve druhém testu zafixujeme $n = 40$ a měníme pravou korelační délku $\ell_{\text{true}} \in [0,05, 2,0]$. Tím měníme sílu misspecifikace. Pro malá ℓ_{true} jsou realizace Matérnova kernelu hrubé a RBF je aproximuje špatně. Pro velká ℓ_{true} jsou realizace hladké a rozdíl mezi oběma kernely se stírá. Hodnoty $\ell_{\text{true}} > 1,0$ navíc leží mimo trénovací rozsah PFN, takže zároveň testujeme, jak model degraduje mimo svůj implicitní prior.

ℓ_{true}	PFN	GP_fixed	GP_ML	GP_Bayes	GP_oracle
0,05	1,06612	2,00588	0,99193	0,98868	0,98116
0,10	0,85773	1,87695	0,81220	0,81922	0,78140
0,20	0,55296	1,13399	0,57635	0,56389	0,50696
0,30	0,37256	0,62423	0,41614	0,39967	0,35816
0,50	0,24395	0,27506	0,25383	0,23612	0,23189
0,70	0,19586	0,18896	0,19154	0,18329	0,18401
1,00	0,16668	0,15438	0,15891	0,15302	0,15353
1,50	0,15210	0,14165	0,13702	0,13590	0,13366
2,00	0,14713	0,13830	0,13038	0,12932	0,12545

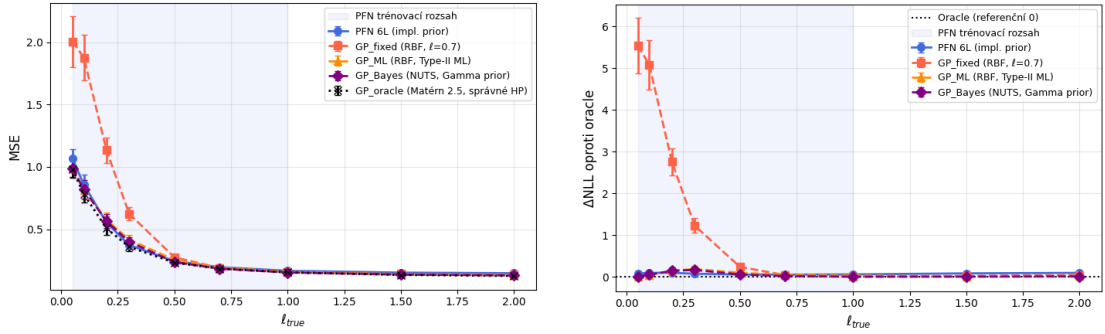
Tabulka 3.17: MSE vůči skutečným hodnotám jako funkce pravé korelační délky ℓ_{true} ($n = 40$, průměr přes 150 instancí). Pro krátké korelační délky vede PFN spolu s adaptivními metodami, pro dlouhé se všechny metody sbíhají a GP_fixed přestává být špatný, jeho $\ell = 0,7$ se totiž blíží pravdě.

Diskuze:

Výsledky shrnují tabulky 3.17 a 3.18 a obrázek 3.20. Za první, síla misspecifikace se projevuje přesně tam, kde bychom čekali. Pro krátké korelační délky, kde jsou Matérnovy realizace hrubé, selhává GP_fixed v kalibraci o řády, jak je vidět v tabulce 3.18. Pro dlouhé korelační délky jsou realizace hladké, RBF s $\ell = 0,7$ se stává rozumnou volbou a všechny metody se sbíhají. Za druhé, PFN je nejlépe kalibrován v okolí pravé hodnoty $\ell_{\text{true}} \approx 0,3$, tedy uprostřed svého trénovacího rozsahu. Pro extrémně krátké délky ho přebijí ostatní adaptivní metody, kterým hrubá data poskytují silný signál pro odhad ℓ . Za třetí, mimo trénovací rozsah ($\ell_{\text{true}} > 1,0$) PFN degraduje jen zlehka. Jeho ΔNLL zůstává malý i pro dvojnásobek horní hranice trénovacího rozdělení, žádný náhlý kolaps nenastává. To je stejná plynulá degradace, jakou jsme u identifikace hyperparametrů pozorovali v sekci 3.2.

ℓ_{true}	PFN	GP_fixed	GP_ML	GP_Bayes
0,05	+0,083	+5,544	+0,007	+0,007
0,10	+0,098	+5,078	+0,041	+0,058
0,20	+0,099	+2,758	+0,148	+0,145
0,30	+0,077	+1,231	+0,191	+0,169
0,50	+0,057	+0,241	+0,098	+0,058
0,70	+0,055	+0,044	+0,052	+0,022
1,00	+0,061	+0,005	+0,034	+0,006
1,50	+0,085	+0,025	+0,015	+0,007
2,00	+0,097	+0,043	+0,020	+0,013

Tabulka 3.18: Kalibrace ΔNLL vůči *oracle* jako funkce ℓ_{true} ($n = 40$, průměr přes 150 instancí). Pro střední korelační délky kolem pravé hodnoty 0,2 až 0,3 je nejbližší *oracle* PFN. Pro velmi krátké i dlouhé délky vedou adaptivní metody, ΔNLL PFN však zůstává malé v celém rozsahu, a to i mimo jeho trénovací rozsah $\ell > 1,0$.



Obrázek 3.20: Vliv síly misspecifikace ($n = 40$). Vlevo: MSE jako funkce ℓ_{true} . Vpravo: ΔNLL vůči *oracle*. Modrý pás značí trénovací rozsah PFN $\ell \in [0,05, 1,0]$. GP_fixed (oranžová) pro krátké korelační délky selhává v obou metrikách, ostatní metody se s rostoucím ℓ_{true} sbíhají k *oracle*. PFN zůstává blízko *oracle* v celém rozsahu včetně hodnot mimo svůj trénovací rozsah.

3.5.3 Chování mimo trénovací rozsah korelační délky

Předchozí část naznačovala, že PFN může relativně dobře zvládnout i odhady pro korelační délky mimo svůj trénovací rozsah. Avšak pozorný čtenář by mohl hned oponovat řadou argumentů. Uvedený test totiž pro takový závěr není korektní. Jedním z důvodů, který leží na povrchu, je tento. Ačkoli některé hodnoty leží mimo rozsah, jsou to hodnoty na hladkém konci škály pro parametr ℓ , kde je úloha sama o sobě snadná. Protože odhadovat hladké funkce není těžká úloha a případné selhání by se skoro neprojevovalo. Čistý test vyžaduje dvě věci. Kernel musí být správně specifikovaný, aby jediným zdrojem chyby byla extrapolace. Dále bychom chtěli, aby korelační délka opustila trénovací rozsah na obou stranách. Přesně takový test nyní provedeme.

Data generujeme z GP s RBF kernelem, tedy ze stejné třídy funkcí, na které byl model trénován. Pravou korelační délku měníme na logaritmické mřížce $\ell \in [0,01, 5,0]$, která přesahuje trénovací rozsah $[0,05, 1,0]$ na obou stranách. Použijeme šestivrstvý náhodný model, $n = 40$ kontextových bodů, šum $\sigma^2 = 0,01$ a 200 instancí pro každou hodnotu ℓ . Chybu měříme jako *relativní MSE*:

$$\text{rel_MSE} = \frac{\text{MSE}(\hat{\mu}_{\text{PFN}}, \mu_{\text{oracle}})}{\text{MSE}(\mu_{\text{oracle}}, \mathbf{y}_{\text{test}})},$$

73

tedy odchylku PFN od *oracle* vydělenou chybou, kterou má sám *oracle* vůči skutečným zašuměným hodnotám. Jmenovatel je tedy chyba, které se nedá zbavit, neboť je způsobena samotným šumem v datech, a žádná metoda se pod ni nedostane. Hodnota $\text{rel_MSE} = 1$ by tedy znamenala, že se PFN liší od *oracle* o tolik, kolik činí tato nevyhnutelná chyba. Hodnoty hluboko pod 1 by znamenaly, že PFN *oracle* prakticky kopíruje.

ℓ	$\text{MSE}(\hat{\mu}_{\text{PFN}}, \mu_{\text{oracle}})$	$\text{MSE}(\mu_{\text{oracle}}, \mathbf{y}_{\text{test}})$	rel_MSE	Stav
0,01	0,35358	0,46192	0,765	mimo rozsah
0,03	0,07729	0,09744	0,793	mimo rozsah
0,05	0,01634	0,03569	0,458	hranice
0,10	0,00389	0,01583	0,246	
0,20	0,00198	0,01233	0,161	
0,30	0,00117	0,01154	0,101	minimum
0,50	0,00244	0,01091	0,224	
0,70	0,00318	0,01069	0,297	
1,00	0,00356	0,01053	0,338	hranice
1,50	0,00294	0,01043	0,282	mimo rozsah
2,00	0,00234	0,01035	0,226	mimo rozsah
3,00	0,00197	0,01028	0,191	mimo rozsah
5,00	0,00192	0,01024	0,188	mimo rozsah

Tabulka 3.19: Chování PFN mimo trénovací rozsah korelační délky (RBF data, $n = 40$, $\sigma^2 = 0,01$, průměr přes 200 instancí). Relativní MSE je odchylka PFN od *oracle* dělená chybou, kterou má sám *oracle* kvůli šumu v datech. Minimum leží uprostřed trénovacího rozsahu a ani mimo rozsah na žádné straně relativní MSE nepřekročí referenční hodnotu 1.

Diskuze:

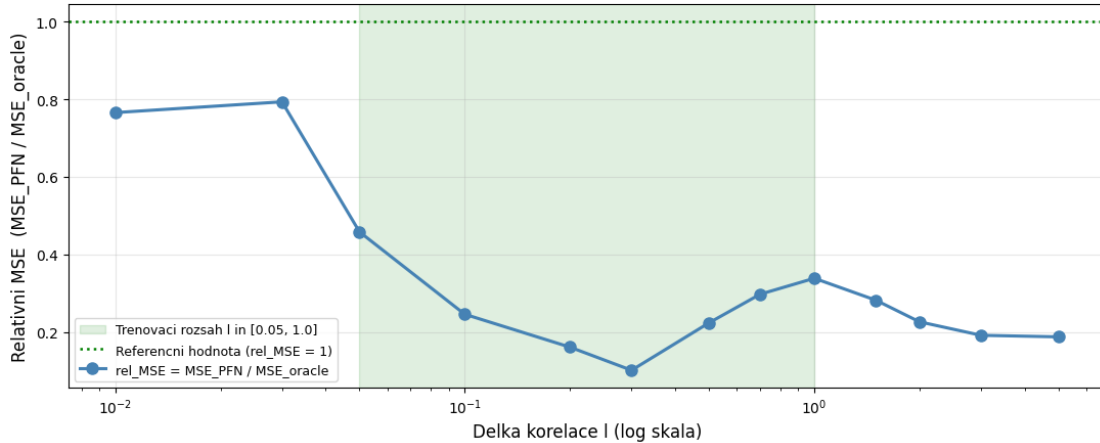
Výsledky shrnuje tabulka 3.19 a obrázek 3.21. Z tabulky 3.19 můžeme učinit jasné závěry. Uvnitř trénovacího rozsahu je relativní MSE hluboko pod referenční hodnotou $\text{rel_MSE} = 1$, tedy pod úrovní chyby způsobené samotným šumem v datech. Nejmenší je uprostřed rozsahu. Přesnou příčinu polohy minima neurčujeme, prior nad ℓ je rovnoměrný, takže minimum nelze vysvětlit tím, že by model střední délky viděl při tréninku častěji. Zde jde spíše o souhru obtížnosti úlohy a normalizace *oracle*.

Za krátkým okrajem rozsahu chyba prudce vzroste. Krátké realizace se rychle mění, kontext 40 bodů je nedokáže zachytit a navíc jde o délky, které model při tréninku neviděl. Zajímavé je, že pro úplně nejkratší délky už relativní chyba dále neroste, dokonce mírně klesá, jak je vidět v tabulce 3.19. Nejde o to, že by se PFN zlepšovalo. V tomto režimu je úloha prakticky nepredikovatelná a velkou chybu dělá i samotný *oracle*, jmenovatel relativní MSE proto rychle roste. Obě metody se zároveň uchylují ke stejné opatrné predikci blízké apriorní střední hodnotě, tedy nule, a jejich vzájemná vzdálenost se přestává zvětšovat. Relativní chyba se tak ustálí hluboko pod referenční hodnotou, ne proto, že by PFN úlohu zvládlo, ale proto, že ji v tomto režimu nezvládá nikdo.

Za dlouhým okrajem chyba naopak opět klesá. Pro velmi dlouhá ℓ jsou realizace hladké a úloha je snadná i pro PFN, které takové délky při tréninku nevidělo. Referenční hodnotu chyba nepřekročí na žádné straně, a to ani pro případy hodně vzdálené od hranice trénovací množiny. Odchylka PFN od *oracle* tedy všude zůstává menší než chyba způsobená samotným šumem v datech. Z experimentu vyplývá, že k degradaci sice dochází, ale žádný náhlý kolaps nenastává.

Z toho usuzujeme, že implicitní prior PFN nefunguje jako ostrá hranice, za kterou model přestává fungovat. Chová se spíše jako měkké těžiště, od kterého přesnost pozvolna klesá. Pro hladké funkce za horní hranicí rozsahu to dává dobrý smysl. Model se během tréninku naučil reprezentovat hladkost a

delší korelační délka pro něj není kvalitativně nová situace. Tím je zpětně podložený i závěr předchozí části. Plynulá degradace mimo trénovací rozsah není artefakt misspecifikovaného kernelu, platí to i v čistém testu.



Obrázek 3.21: Relativní MSE jako funkce pravé korelační délky ℓ (logaritmická osa). Zelený pás značí trénovací rozsah $\ell \in [0.05, 1.0]$, zelená tečkovaná čára referenční hodnotu $\text{rel_MSE} = 1$. Křivka má minimum uprostřed trénovacího rozsahu a k jeho okrajům roste. Za krátkým okrajem chyba prudce vzroste a ustálí se, za dlouhým okrajem opět mírně klesá. Referenční hodnotu nikde nepřekročí.

3.5.4 Shrnutí

Tato sekce zkoumala otázku, co se stane, když správný prior nezná žádná z metod a jak se chová PFN v porovnání s nimi.

Zjistili jsme, že ve smyslu MSE je PFN srovnatelné s adaptivními metodami GP_ML a GP_Bayes. Pro střední velikosti kontextu je dokonce nejlepší, a to přesto, že na rozdíl od nich nemůže svůj prior za běhu nijak přizpůsobit datům. V kalibraci je PFN nejlepší ze všech misspecifikovaných metod pro malý a střední kontext i pro korelační délky v okolí pravé hodnoty. GP_fixed, který nemá žádný mechanismus adaptace, selhává nejvíce pro krátké a střední korelační délky a v MSE pro všechna n . Pro dlouhé korelační délky, kde se jeho pevné $\ell = 0.7$ blíží pravé hodnotě, přestává být nejhorší. Čistý test s korektně specifikovaným kernelem navíc ukázal, že mimo trénovací rozsah korelační délky PFN degraduje plynule a jeho odchylka od *oracle* zůstává menší než chyba způsobená samotným šumem v datech, jak je vidět v tabulce 3.19.

Vysvětlení je stejné jako u kalibrace rozptylu v sekci 3.2. PFN neprovádí bodový odhad hyperparametrů, ale marginalizuje přes celé trénovací rozdělení. Tam, kde adaptivní metody z malého kontextu korelační délku přefitují a vracejí příliš sebejisté intervaly, drží PFN opatrnější nejistotu, a právě proto je jeho ΔNLL nejmenší. Za tuto robustnost platí jen malou cenu v MSE pro velké kontexty, kde už mají adaptivní metody dost dat na spolehlivý odhad.

Dohromady tedy platí, že amortizovaná Bayesovská inference PFN není křehká vůči volbě prioru, alespoň v těchto syntetických experimentech. I když se skutečný generativní proces liší od trénovacího prioru samotným kernelem, PFN zůstává přesné a ze všech srovnávaných metod nejlépe kalibrované právě v režimu malých dat.

Kapitola 4

PFN pro segmentaci obrazu

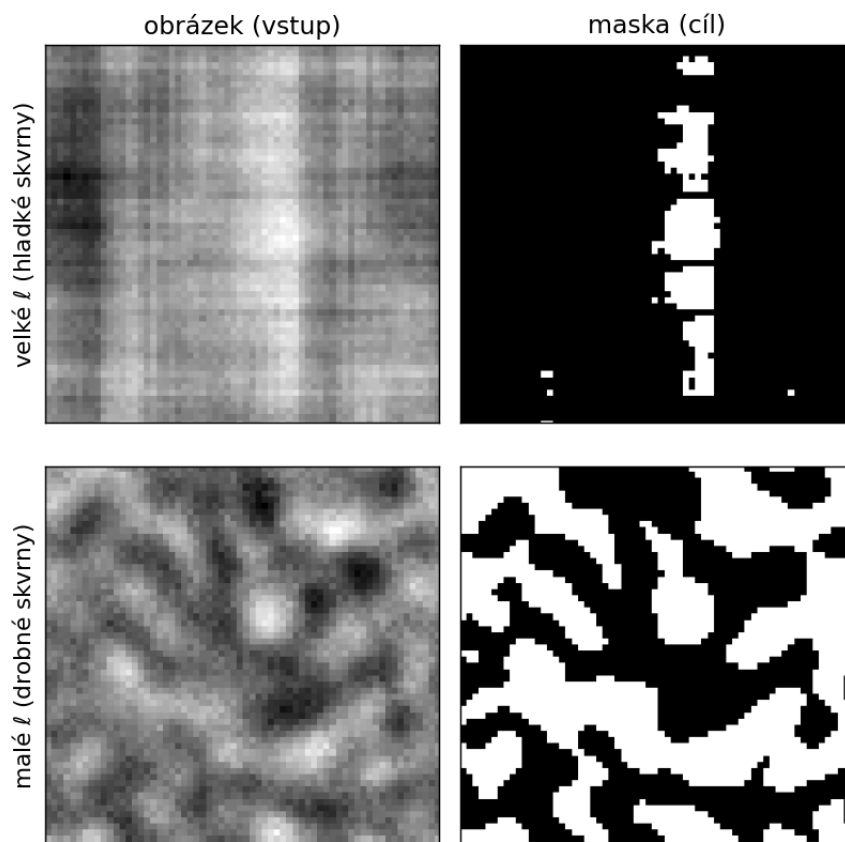
V předchozí kapitole jsme PFN zkoumali jen na jednorozměrné regresi. Tam jsme se měli o co opřít, pro Gaussovský proces umíme spočítat přesný posterior. V této kapitole dáme PFN těžší a praktičtější úlohu, a to segmentaci obrazu. Ukážeme, že není to zcela nová úloha, ale přirozené pokračování celé práce. Jde totiž o stejnou amortizovanou Bayesovskou inferenci, jen o jednu dimenzi výše a s binárním cílem. Současně tím získáme nové, nezávislé prostředí, ve kterém můžeme znovu ověřit předpovědi Naglerovy teorie ze sekce 2.4.

4.1 Od jednorozměrné regrese k segmentaci obrazu

V této úvodní sekci popíšeme tuto novou úlohu a přeložíme do ní pojmy, se kterými jsme dosud pracovali. Doposud PFN dostávalo body na přímce a předpovídalo spojitou hodnotu. Segmentace je úloha jiného typu. Na vstupu máme obrázek a chceme u každého jeho bodu (pixelu) rozhodnout, zda patří do hledané oblasti nebo ne. Tomuto rozhodování pro celý obrázek najednou se říká segmentace, tedy rozdělení obrazu na oblast zájmu a pozadí. Oblasti zájmu, tedy pixelům hledaného útvaru, budeme říkat popředí (anglicky *foreground*), všem ostatním pixelům pozadí (*background*). Výsledkem je tzv. maska, což je obrázek stejného rozměru jako vstup, ve kterém má pixel popředí hodnotu jedna a pixel pozadí hodnotu nula. Masku si můžeme představit jako černobílý obrys obtažený přes původní obrázek.

Hlavní rozdíl oproti regresi je ve výstupu. V regresi PFN předpovídalo rozdělení jedné spojité hodnoty pro jeden vstupní bod. Zde předpovídá pro každý pixel pravděpodobnost, že leží uvnitř oblasti, tedy číslo mezi 0 a 1. Model tedy nevrací rovnou tvrdou černobílou masku, ale pravděpodobnostní mapu, kde hodnota blízká jedné znamená „téměř jistě uvnitř“ a hodnota blízká nule „téměř jistě mimo oblast zájmu“. Právě tato mapa nás zajímá, protože u ní můžeme stejně jako v jedné dimenzi zkoumat přesnost predikce.

Ukažme nyní, že segmentace není zcela nová úloha, ale přirozené pokračování toho, co už jsme viděli v kapitole 3. V celé předchozí kapitole jsme pracovali s funkcí jedné proměnné vzorkovanou z Gaussovského procesu. Představme si nyní stejnou myšlenku, ale s funkcí dvou proměnných, tedy s náhodným polem přes celou plochu obrázku. Hladkost tohoto skrytého pole řídí stejná korelační délka ℓ jako dříve. Malé ℓ dává drobné a ostré útvary, velké ℓ velké hladké plochy. Zatím je to jen dvourozměrná obdoba Gaussovského procesu z předchozí kapitoly. Z tohoto spojitého pole vytvoříme masku jednoduchým krokem. Zvolíme práh a řekneme, že pixel patří do popředí právě tehdy, když hodnota pole v něm práh překročí. Tím se ze spojitého pole stane binární maska, tedy náš cíl. Vstupní obrázek pak vznikne tak, že k poli přidáme šum. Obrázek je tedy rozmazaná verze skryté pravdy a model má z něj odhadnout ostrou masku pod ní. Obrázek 4.1 ukazuje dva takové páry obrázku a jeho masky, jeden pro velké ℓ (hladké skvrny) a jeden pro malé ℓ (drobné skvrny).



Obrázek 4.1: Ukázka úlohy segmentace. Vlevo vstupní obrázek (zašuměné skryté pole), vpravo odpovídající maska (cíl predikce), kterou získáme prahováním skrytého pole. Horní řádek odpovídá velké korelační délce ℓ , tedy hladkým plochám, dolní řádek malé ℓ , tedy drobným ostrým útvarům. Model dostává na vstupu levý obrázek a má předpovědět pravou masku.

Zbývá vysvětlit, jak modelu řekneme, jaká pravidla pro danou sérii obrázků platí, tedy jaká je korelační délka, kde leží práh a jak silný je šum. Stejně jako v regresi mu to neřekneme přímo, ale ukážeme mu několik hotových příkladů. Zde opět postupujeme podle standardního postupu. Dáme mu sadu dvojic obrázku a už správně nakreslené masky a model si z nich má odvodit, jaká pravidla v této sérii platí. Této sadě příkladů budeme říkat *support set* (kontextová sada), je to přímá obdoba kontextových bodů D_n z jednorozměrného případu. Model tedy funguje a zpracovává vstup podle stejného principu, jak jsme již popsali v kapitole 2.

Celkově tedy jde o stejnou amortizovanou Bayesovskou inferenci, jen ve dvou dimenzích a s binárním cílem. Posteriorní prediktivní rozdělení (PPD), které bylo v regresi Gaussovou hustotou, je zde v každém pixelu Bernoulliho rozdělení, tedy pravděpodobnost, že pixel patří do oblasti. Trénovací kritérium, kterým byla u regrese záporná log-věrohodnost výstupní hlavy *BarDistribution*, se zde stává pixelovou binární křížovou entropií (BCE). Všechny ostatní pojmy zůstávají beze změny, jen je čteme v novém kontextu.

Nyní můžeme říct, co přesně budeme v této kapitole zkoumat. Připomeňme, že Naglerovy výsledky ze sekce 2.4 nezávisí na dimenzi vstupu. Věta 2.4.6 o mizení rozptylu i neredukovatelný bias z podmínky lokality (věta 2.4.5) jsou tvrzení o *attention mechanismu* obecně, a měla by tedy platit i pro segmentaci. Právě tohoto faktu využijeme. Nejprve vezmeme hotový, už natrénovaný *in-context* segmenter

UniverSeg [14]. Není to PFN, ale funguje na stejném principu, predikuje z kontextové sady přes globální *attention*. Ověříme, zda se u něj objeví tzv. kolaps k prioru. Jde o jev, kdy model s příliš malým kontextem přestane odpovídat na konkrétní vstup a vrátí se k výchozí odpovědi, tedy k průměrné masce, kterou zná z tréninku. Intuice je jednoduchá. Když má model málo příkladů, nedokáže z nich poznat pravidla úlohy, a nejbezpečnější volbou je pro něj průměr. S podmínkou lokality tento jev souvisí následovně. *Softmax attention* nikdy nedá žádnému bodu kontextu váhu přesně nula. Predikce proto vždy obsahuje složku tvořenou průměrem přes celý kontext, která na sousedech konkrétního bodu nezávisí. Při dostatku dat se tato globální složka projevuje jen jako malý neredukovatelný bias. Při malém nebo neinformativním kontextu z predikce zbude právě jen zprůměrovaná odpověď známá z tréninku. Pokud se kolaps objeví i u modelu, který jsme sami netrénovali, nejde o artefakt našeho tréninku, ale o obecnou vlastnost amortizované inference. Poté natrénujeme vlastní PFN přímo na úloze popsané výše. Zde využijeme hlavní výhodu syntetických dat, pravidla jejich generování známe, a proto umíme pro každý obrázek spočítat i přesný posterior. Tomu budeme stejně jako v jedné dimenzi říkat *oracle*. U reálných segmentačních dat takovou pravdu nikdy nemáme. Zde avšak proti ní můžeme přesně změřit, jak moc dobře ji PFN aproximuje a kde vzniká odchylka, tedy bias, rozptyl, kalibraci a chování mimo trénovací rozdělení. Zkoumáme tedy, jak dobře PFN aproximuje ideální Bayesovskou inferenci a kde se od ní začíná odchylovat, jen nyní na náročnější úloze.

4.2 Kolaps k prioru u předtrénovaného modelu

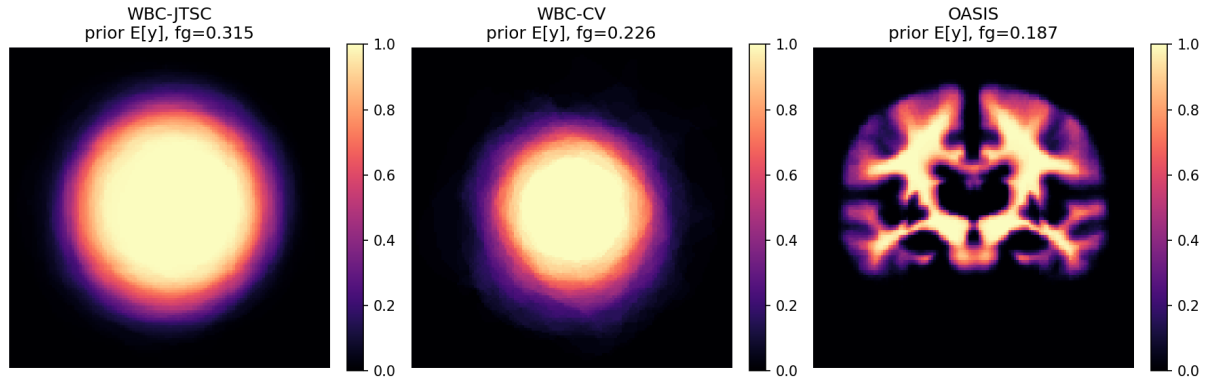
V tomto prvním experimentu nebudeme trénovat vlastní model, ale vezmeme hotový *in-context* segmenter a otestujeme na něm předpověď Naglerovy podmínky lokality.

4.2.1 Uspořádání experimentu

Jako model použijeme UniverSeg, což je předtrénovaný *in-context* segmenter. Model necháme zmražený, žádnou jeho váhu neupravujeme. UniverSeg nabízí interní průměrování přes více kontextových sad. Voláme ho vždy nad jedním *support setem*, tedy bez tohoto interního průměrování. Kdybychom průměrovali, zaměnili bychom vliv velikosti kontextu s vlivem tohoto průměrování, a to nechceme.

Kolaps budeme sledovat podél dvou os. Hlavní a čistou osou je velikost *support setu* n_{supp} , kterou měníme od jednoho příkladu po 64. Na ní stojí hlavní tvrzení experimentu, protože přímo řídí, kolik informace model dostane, a nezávisí na tom, o jaká data jde. Druhou, doplňkovou osou je změna domény, tedy typu a zdroje obrázků. Postupujeme přes tři domény, snímky bílých krvinek (WBC-JTSC [15]), jinou akvizici stejných buněk (WBC-CV [15]) a mozkové MRI (OASIS [16]). Poznamenejme, že touto osou neměříme vzdálenost dat od tréninku, model všechny tři modalitty při tréninku viděl, ale jen robustnost napříč různými úlohami. U krvinek je cílem segmentace maska celé buňky, u mozku maska bílé hmoty.

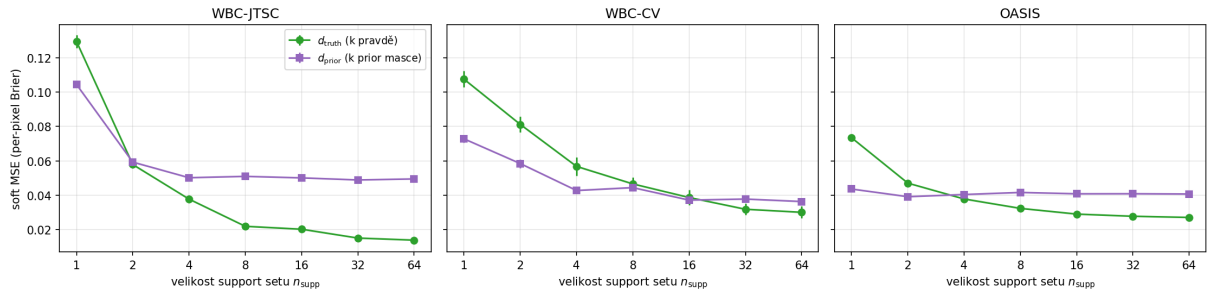
Abychom mohli měřit posun predikce k prioru, potřebujeme prior nějak vyjádřit. UniverSeg žádný explicitní prior nemá. To obejdeme tímto způsobem. Nahradíme ho tzv. operační maskou, tzn. průměrnou maskou přes všechny příklady dané domény. Ta má smysl jen pro prostorově zarovnaná data, kde průměr dává rozumný tvar. U krvinek je buňka vždy zhruba uprostřed, takže průměrná maska je kompaktní útvar uprostřed, jak ukazuje obrázek 4.2. U mozku je struktura složitější a zašuměnější, ale stále čitelná.



Obrázek 4.2: Operační prior masky, tedy průměrné masky přes všechny příklady dané domény. Hodnota v pixelu udává, jak často v něm napříč doménou leží popředí. U krvinek (WBC) je maska kompaktní útvar uprostřed, u mozku (OASIS) je strukturovanější. Hodnota fg je podíl popředí v průměrné masce.

4.2.2 Kolaps k prioru

Chceme zjistit, zda se predikce s ubývajícím kontextem vrací k prioru, jak předpovídá věta 2.4.5. K tomu zavedeme dvě vzdálenosti. První je vzdálenost predikce od pravé masky, značíme ji d_{truth} . Druhá je vzdálenost predikce od operační prior masky, značíme ji d_{prior} . Obě počítáme jako průměrnou pixelovou kvadratickou odchylku. Malé d_{truth} tedy znamená, že model předpověděl blízko pravdě, malé d_{prior} , že se drží výchozí průměrné masky.



Obrázek 4.3: Kolaps k prioru. Zelená křivka je vzdálenost predikce od pravé masky d_{truth} , fialová vzdálenost od operační prior masky d_{prior} , obojí jako funkce velikosti *support setu*. Při jednom příkladu je predikce blíž prioru než pravdě (kolaps), s rostoucím kontextem se křivky protnou a predikce je blíže k pravdě. Chybové úsečky jsou ± 1 směrodatná chyba.

Diskuze:

Na obrázku 4.3 kolapsem k prioru rozumíme stav, kdy je predikce blíž prior masce než pravdě, tedy $d_{\text{prior}} < d_{\text{truth}}$. Poznáme ho proto z uspořádání obou křivek. Zde tento stav nastává pouze při malém kontextu. Při kontextu rovnajícím se jedné ukázce leží ve všech třech doménách fialová křivka pod zelenou. S rostoucím *support setem* d_{truth} prudce klesá, až se obě křivky protnou. Křížení avšak neznačí kolaps samotný, ale jeho konec. Od té chvíle je predikce blíž pravdě než prioru, vzdálenost k pravdě přebije vzdálenost k prioru. Poloha křížení pak měří, kolik kontextu model k tomuto úniku potřebuje. Mírně se liší podle domény, nejdříve k němu dochází u snímků WBC-JTSC, nejpозději u WBC-CV. Kolaps tedy řídí velikost kontextu, nikoli konkrétní úloha. Právě tato blízkost k prioru při slabé podmiňující informaci Naglerova teorie předpovídá, a potvrzuje se tak i na modelu, který není úplně *PFN-type*. Podstatné

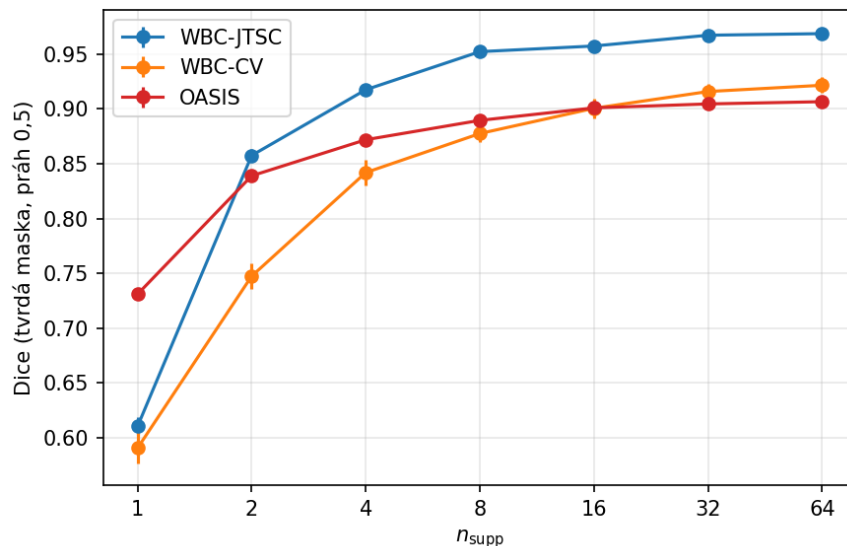
je také, že d_{prior} neklesá k nule, ale ustálí se na nenulové hodnotě. Model totiž ani při velkém kontextu nepredikuje přesně průměrnou masku, jeho predikce je specifická pro daný obrázek.

4.2.3 Výkon a saturace kontextem

Mimo jiné by nás také mohla zajímat i samotná kvalita masky. Měříme ji *Dice* koeficientem, tedy mírou překryvu predikované a pravé masky, kde jedna znamená dokonalý překryv a nula žádný.

Diskuze:

Obrázek 4.4 ukazuje, že *Dice* s rostoucím kontextem roste a poté saturuje. Saturací rozumíme stav, kdy křivka přestane růst a ustálí se na plató, další příklady v kontextu už tedy kvalitu masky nezlepšují. V našem případě se přidávat další kontext přestane vyplácet zhruba od osmi až šestnácti příkladů. Je to přímá dvourozměrná obdoba saturace přesnosti s velikostí kontextu, kterou jsme viděli u regrese v kapitole 3. Plató se mírně liší podle domény, ale kvalitativní tvar křivky, tedy růst a následná saturace, zůstává stejný.



Obrázek 4.4: *Dice* koeficient jako funkce velikosti *support setu* pro tři domény. Křivka roste a saturuje, výška plató se mírně liší podle domény. Chybové úsečky jsou ± 1 směrodatná chyba.

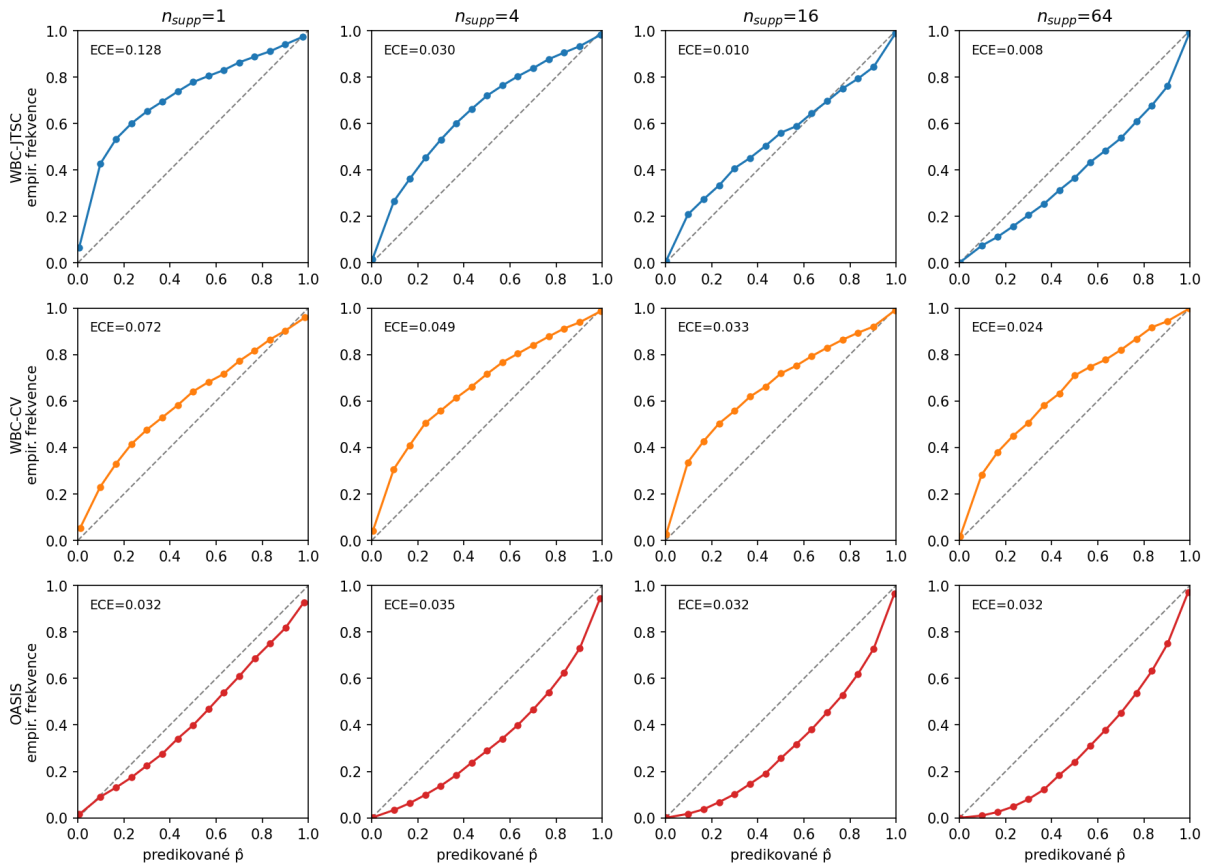
4.2.4 Kalibrace

Nakonec nás zajímá, zda pravděpodobnosti, které model vrací, odpovídají skutečnosti. Řekne-li model u nějakých pixelů hodnotu 0,8, mělo by z nich popředí obsahovat zhruba 80 %, jinak jsou jeho pravděpodobnosti zavádějící. Tuto shodu měříme dvěma metrikami a u obou platí, že směr případné chyby nepředpokládáme, ale měříme. První je *reliability diagram*. Vezmeme všechny pixely, u kterých model hlásí zhruba stejnou pravděpodobnost popředí, například $\hat{p} \approx 0,7$, a spočítáme, jaký podíl z nich v pravé masce popředí skutečně je. U dokonale kalibrovaného modelu by vyšlo právě 70 %. Diagram tyto dvojice vynáší proti sobě, dokonalá kalibrace tedy leží na přímce. Leží-li křivka nad přímkou, je skutečný podíl popředí vyšší, než model hlásí, a model tedy pravděpodobnost popředí podhodnocuje. Odchylku od této přímky shrnuje jedno číslo, očekávaná kalibrační chyba (ECE). Druhou metrikou je tzv. *znaménkový rozdíl*, který hodnotí tvrdá rozhodnutí. Pixel prohlásíme za popředí, pokud $\hat{p} > 0,5$, jinak za pozadí. Každé takové rozhodnutí má svou jistotu, a tou je pravděpodobnost, kterou model zvolené třídě přiřadil, tedy

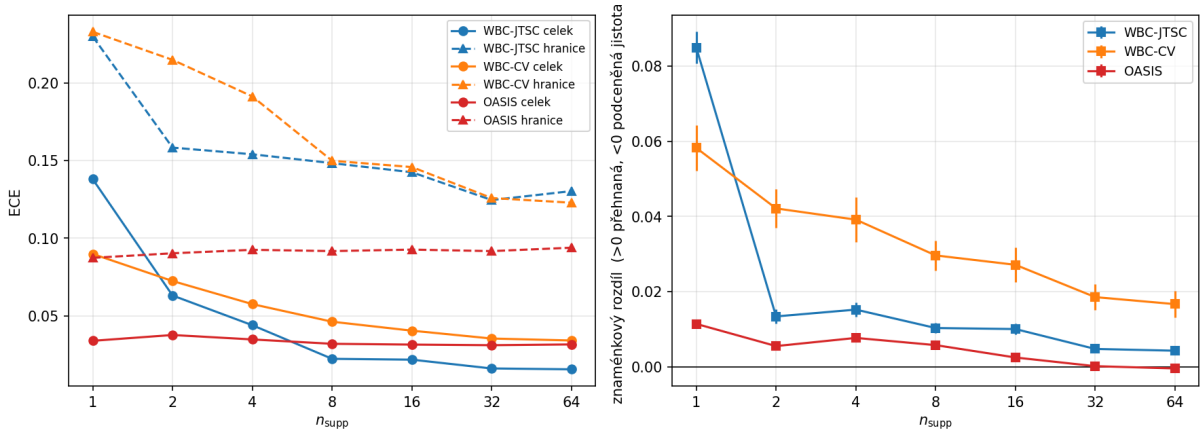
$\max(\hat{p}, 1 - \hat{p})$. Řekne-li model například $\hat{p} = 0,9$, rozhodne pro popředí s jistotou 0,9, řekne-li $\hat{p} = 0,2$, rozhodne pro pozadí s jistotou 0,8. Znaménkový rozdíl je pak průměrná jistota minus podíl správných rozhodnutí. Kladná hodnota znamená, že si model svými rozhodnutími věří víc, než jak často má pravdu, záporná naopak.

Diskuze:

Měření na obrázcích 4.5 a 4.6 ukazuje, že směr kalibrace není univerzální, ale závisí na doméně. U krvinek můžeme říct, že je model podhodnocený. Model je opatrný a vrací mírně jiné pravděpodobnosti, než by odpovídalo skutečnosti. U mozku je naopak přehnaně sebejistý, a to téměř nezávisle na velikosti kontextu. Znaménkový rozdíl na obrázku 4.6 vpravo je přitom kladný ve všech doménách, nejsilnější je při jednom příkladu a s rostoucím kontextem klesá k nule. To není ve sporu s reliability diagramy, obě metriky totiž odpovídají na jinou otázku. *Reliability diagram* se ptá, zda sedí hlášená pravděpodobnost popředí, znaménkový rozdíl se ptá, zda sedí jistota zvolené třídy. Model tak může u krvinek pravděpodobnost popředí podhodnocovat, tedy mít křivku nad úhlopříčkou, a současně si svými tvrdými rozhodnutími věřit víc, než jak často má pravdu. Když navíc oddělíme vnitřek objektu od úzkého pásma kolem hranice, ukáže se, že se kalibrační chyba soustředí právě na hranici. Kalibrační chyba je tam řádově vyšší než uvnitř a s kontextem klesá mnohem pomaleji, u mozku dokonce zůstává prakticky konstantní. Poznamenejme, že tento závěr je v souladu se vzorem, který jsme viděli i v jedné dimenzi, model dobře pozná, kde je nejistý, ale absolutní míru nejistoty odhaduje hůře.



Obrázek 4.5: *Reliability* diagramy pro tři domény (řádky) a vybrané velikosti *support setu* (sloupce). Křivka nad úhlopříčkou znamená podhodnocenou jistotu, pod úhlopříčkou přehnanou.



Obrázek 4.6: Vlevo očekávaná kalibrační chyba (ECE) zvláště pro celý obrázek a pro hraniční pásmo, vpravo znaménkový rozdíl mezi jistotou a správností (kladný znamená přehnanou jistotu). Kalibrační chyba je na hranici řádově vyšší než uvnitř a klesá s kontextem pomaleji.

4.2.5 Shrnutí

Tento experiment ukázal, že hotový *in-context* segmenter dědí přesně ty samé patologie, které Nagle-rova teorie předpovídá pro každý amortizovaný model s globálním *attention*. S ubývajícím kontextem se predikce vrací k prioru a kalibrace je nestabilní. Protože jsme model sami netrénováli, nemůže jít o artefakt naší trénovací procedury, ale o obecnou vlastnost amortizace. Podstatné je, že jev je napříč všemi třemi úlohami kvalitativně stejný. Kolaps tedy řídí velikost kontextu, nikoli konkrétní úloha.

Současně se ukázala i mez tohoto testu. Poznamenejme, že UniverSeg nemá pravý prior ani dostupný posterior, proto jsme se museli spolehnout na operační prior masku a nemohli jsme ji porovnat s přesným posteriorem. Přesně tuto mez odstraní následující experiment, ve kterém natrénujeme vlastní PFN na úloze, kde prior i přesný posterior známe, a stejné patologie změříme rigorózně proti *oraculu*.

4.3 Srovnání vlastního PFN s přesným posteriorem

V této sekci provedeme hlavní experiment kapitoly. Natrénujeme vlastní PFN na úloze ze sekce 4.1, ve které pro každý obrázek umíme spočítat přesný posterior. Změříme, jak moc přesně PFN jej aproximuje a kde se od něj odchyluje.

4.3.1 Uspořádání experimentu

Data generujeme přesně podle postupu z úvodu kapitoly. Vzorkujeme dvourozměrné pole z Gaussovskeho procesu, prahováním z něj vytvoříme masku a přidáním šumu vstupní obrázek. Hyperparametry úlohy při tréninku nedržíme pevné. Korelační délku, sílu šumu i podíl popředí (tedy jak velkou část obrázku zabírá oblast zájmu) losujeme pro každou dávku znovu, aby se model musel naučit celou rodinu úloh, ne jednu konkrétní. Pole vzorkujeme na čtvercové mřížce 64×64 pixelů, obrázek má tedy stranu $L = 64$ pixelů. Korelační délku ℓ přitom neudáváme v pixelech, ale relativně, jako zlomek ℓ/L strany obrázku. Takové vyjádření totiž přímo říká, jak velké jsou útvary vzhledem k celému obrázku, a nezávisí na zvoleném rozlišení. Například $\ell/L = 0,05$ znamená drobné útvary o velikosti kolem pěti procent obrázku, $\ell/L = 0,40$ velké hladké plochy přes téměř polovinu obrázku. Losujeme $\ell/L \in [0,05, 0,40]$ log-uniformně, směrodatnou odchylku šumu $\sigma \in [0,01, 0,30]$ a podíl popředí z intervalu $[0,15, 0,85]$.

Velikost *support setu* pak volíme náhodně z množiny $\{1, 2, 4, 8, 16, 32, 64\}$. Tyto intervaly tvoří trénovací rozsah, ke kterému budeme výsledky později vztahovat. Jako architekturu záměrně volíme stejnou rodinu jako v předchozí sekci, tedy UniverSeg, tentokrát ovšem s náhodnou inicializací, bez předtrénovaných vah. Model trénujeme od nuly na datech z našeho prioru po dobu 250 epoch, s pixelovou Bernoulliho hlavou a s binární křížovou entropií jako kritériem. Právě tento trénink na explicitním prioru dělá z modelu PFN. Obě sekce se tak liší jen trénovací procedurou a daty, případné rozdíly v chování proto nelze svést na architekturu.

Technická poznámka:

Podstatná je jedna technická volba, díky které máme přesnou pravdu k dispozici. Výpočet přesného posterioru totiž vyžaduje inverzi kernelové matice, která má řádek a sloupec pro každý pixel, a její přímá inverze by proto byla příliš pomalá. Proto pole vzorkujeme na *toru*, tedy na mřížce se slepenými okraji, kde levý okraj sousedí s pravým a horní s dolním. Na toru vypadá okolí každého pixelu stejně, takže kovariance dvou pixelů závisí jen na jejich vzájemném posunu. Takovou matici diagonalizuje Fourierova transformace a inverze se rozpadne na obyčejné dělení frekvencí po frekvenci, přesný posterior proto spočítáme rychle a bez aproximace. Výsledek je náš *oracle* z úvodu kapitoly, tedy nejlepší možná pravděpodobnostní mapa, jakou lze z daného obrázku vůbec získat. Torus je jen pomůcka pro tento výpočet a na interpretaci výsledků nemá podstatný vliv.

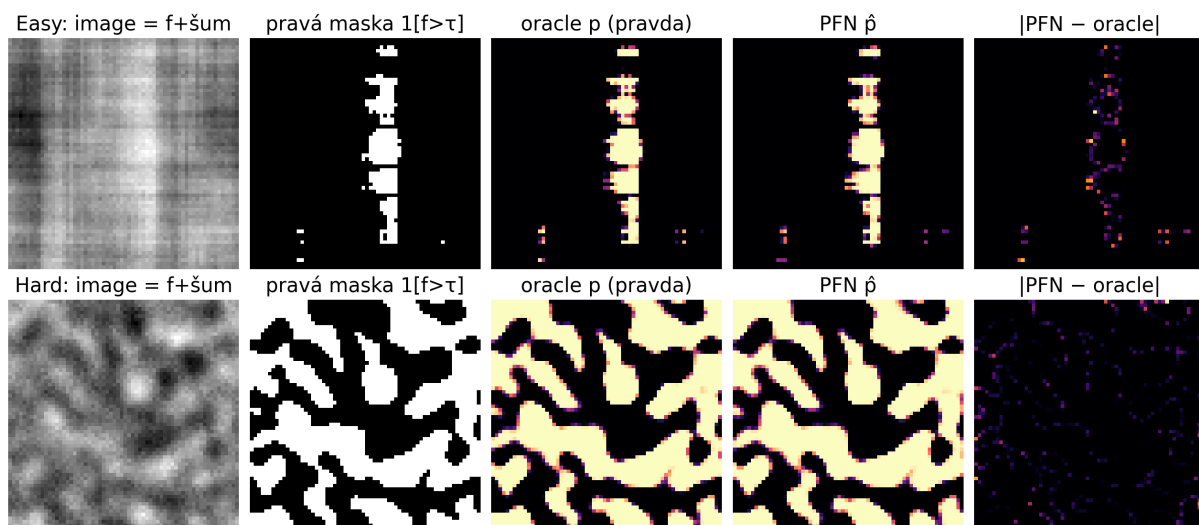
Nakonec zaved’me značení režimů, podle kterého budeme výsledky číst. Náročnost úlohy řídí korelační délka, čím je kratší, tím členitější je pole a tím těžší jeho predikce. Uvnitř trénovacího rozsahu proto rozlišujeme úlohy snadné (*Easy*, $\ell/L = 0,40$), střední (*Medium*, $\ell/L = 0,15$) a náročné (*Hard*, $\ell/L = 0,05$). Snadný a náročný režim tak leží na okrajích trénovacího rozsahu. K nim přidáme dva režimy mimo trénovací rozsah, ve kterých je korelační délka kratší (*OOD-short*, $\ell/L = 0,03$, členitější pole) nebo delší (*OOD-long*, $\ell/L = 0,60$, hladší pole), než jakou model při tréninku viděl. Zkratka OOD znamená *out-of-distribution*, tedy data mimo trénovací rozdělení. Není-li uvedeno jinak, každý vykreslený bod průměrujeme přes 40 až 48 nezávisle vygenerovaných úloh.

4.3.2 Shoda predikce s oraclem

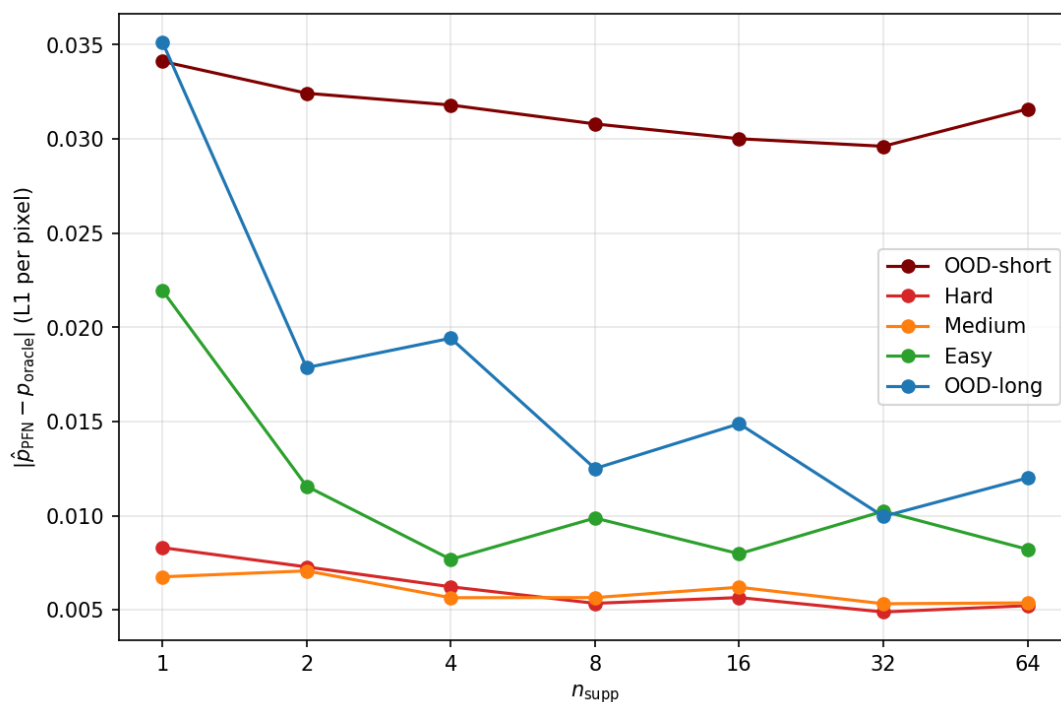
Nejprve se ptáme, jak blízko je predikce PFN k pravému posterioru. Vzdálenost měříme absolutní odchylkou obou pravděpodobnostních map zprůměrovanou přes všechny pixely. Na obrázku 4.7 jsou vedle sebe vstupní obrázek, pravá maska, *oracle* a predikce PFN pro snadnou a náročnou úlohu. Mapy PFN a *oracle* jsou na pohled téměř k nerozeznání. Viditelný rozdíl zůstává jen na tenkých hranicích mezi popředím a pozadím.

Diskuze:

Rozdíl mezi PFN a *oracle*m znázorňuje graf na obrázku 4.8, který vynáší jejich průměrnou vzdálenost jako funkci velikosti kontextu pro všech pět režimů. Plyne z něj poznatek, který se v této sekci bude opakovat. Velikost kontextu téměř nehraje roli, predikce je dostatečně dobrá i s jediným *support párem*, tedy s jedinou dvojicí obrázku a masky v kontextu. Vysvětlení je přímočaré a plyne přímo z toho, jak naše úloha vzniká. Připomeňme, že vstupní obrázek jsme vytvořili tak, že jsme ke skrytému poli, tedy k té hladké funkci, ze které se prahováním dělá maska, přičetli šum. Hodnota každého pixelu je proto hodnota tohoto pole v daném bodě plus šum, tedy jeho přímé, jen zašuměné měření. Obrázek jako celek tak zachycuje pole ve všech bodech najednou. To je zásadní rozdíl oproti jednorozměrné regresi, kde model o funkci věděl jen z několika kontextových bodů. Zde si model dokáže pole z velké části zrekonstruovat už ze samotného vstupního obrázku, bez pomoci kontextu. Ze *support setu* se dozvídá jen to, co ze samotného obrázku poznat nejde, tedy kde leží práh mezi popředím a pozadím a jak silný je šum. Co naopak rozhoduje, je náročnost úlohy. Uvnitř trénovacího rozsahu je vzdálenost od *oracle*u malá, mimo rozsah roste. Nejhuře dopadají pole s kratší korelační délkou, než jakou model viděl při tréninku. U nich chyba zůstává velká i s velkým kontextem. Naopak u hladších polí se model přiblíží *oracle*u,



Obrázek 4.7: Ukázka predikcí pro snadnou (nahore) a náročnou (dole) úlohu. Zleva vstupní obrázek, pravá maska, *oracle* (pravý posterior), predikce PFN a jejich absolutní rozdíl. Mapy PFN a *oracle* téměř splývají, rozdíl je soustředěn na hranici mezi popředím a pozadím, tedy na obrysu segmentované oblasti.



Obrázek 4.8: Průměrná vzdálenost predikce PFN od *oracle* jako funkce velikosti kontextu pro pět režimů náročnosti. Uvnitř trénovacího rozsahu (Hard, Medium, Easy) je vzdálenost malá. U režimů Hard a Medium na velikosti kontextu téměř nezávisí, u režimu Easy s kontextem znatelně klesá. Mimo rozsah (OOD-short, OOD-long) je větší, členitější pole (OOD-short) se *oracle* nepřiblíží ani s velkým kontextem.

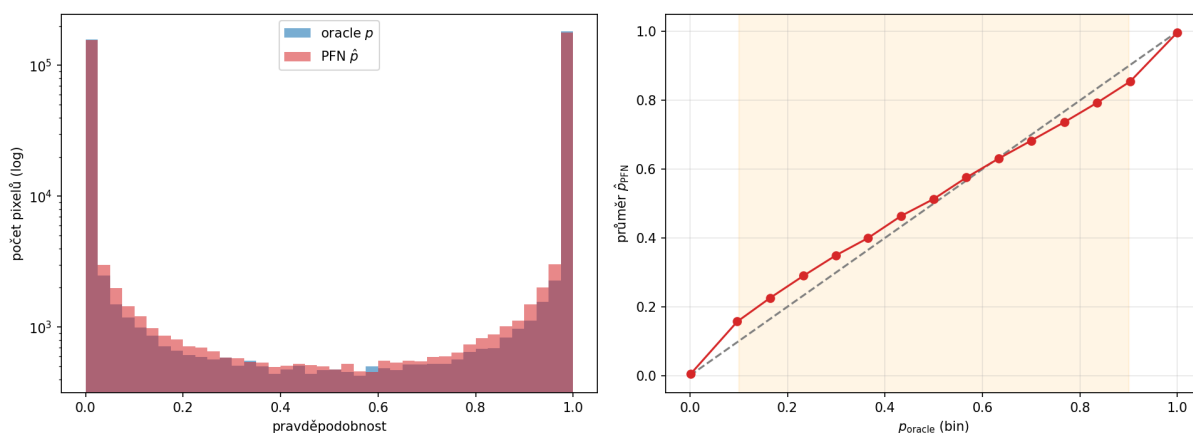
jakmile dostane malý kontext. Stejnou asymetrii extrapolace jsme u regrese viděli v části 3.5.3, i tam byly krátké korelační délky pro model těžší než dlouhé. Důvod je také stejný. Pole s krátkou korelační délkou obsahují jemnější útvary, než jaké model při tréninku viděl, a tento rozdíl nevyrovná ani hustý vstupní obrázek.

4.3.3 Kalibrace

To, že střední hodnota vychází smysluplně, ještě nemusí nutně znamenat, že PFN správně reprezentuje i nejistotu. Proto se ptáme, zda se pravděpodobnosti, které PFN vrací, shodují s pravděpodobnostmi *oracelu*, především na nejednoznačných hranicích.

Diskuze:

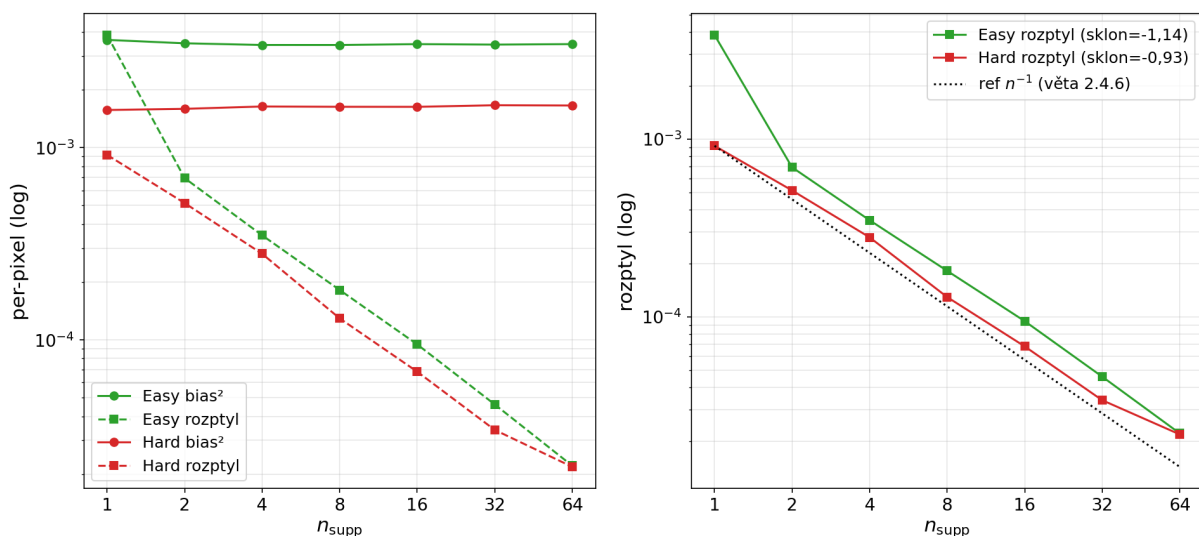
Obrázek 4.9 vlevo ukazuje, že rozdělení predikovaných pravděpodobností PFN a *oracelu* se téměř překrývají. Obě rozdělení mají výrazné vrcholy u nuly a u jedné, tedy tam, kde je pixel jistě mimo nebo jistě uvnitř. Mezi vrcholy leží nízké údolí. Prostřední hodnoty pravděpodobnosti má jen malý zlomek pixelů. To přesně odpovídá geometrii úlohy. Nejisté jsou jen pixely v úzkém pásmu podél hranic mezi popředím a pozadím, a těch je oproti vnitřkům oblastí málo. Že jde právě o hraniční pixely, přitom nevyčteme z histogramu, ale z konstrukce úlohy. Prostřední pravděpodobnost dostane pixel právě tehdy, když je odhad pole blízko prahu, a body, kde pole protíná práh, tvoří z definice masky právě její hranici. Stejně pásmo středních hodnot podél hranic je ostatně přímo vidět na mapách na obrázku 4.7. Podstatné je, že PFN tuto nejistotu na hranicích zachycuje stejně jako *oracle*, nesnaží se ji uměle potlačit. Zbývá jen mírná odchylka, na hranicích je totiž PFN o něco sebejistější, než by odpovídalo *oracelu*, jak ukazuje pravá část obrázku. Tato přehnaná ostrost je jediná systematická vada kalibrace, kterou jsme naměřili, ale je malá. Připomeňme, že jde o stejný vzor jako v jedné dimenzi, model dobře pozná, kde je nejistota, a jen mírně chybí v její velikosti.



Obrázek 4.9: Vlevo rozdělení predikovaných pravděpodobností PFN a *oracelu* přes všechny pixely, obě křivky se téměř překrývají. Vpravo srovnání průměrné predikce PFN s *oraclem* po intervalech, mírné vybočení od úhlopříčky odpovídá přehnané ostrosti na hranicích.

4.3.4 Rozklad chyby na bias a rozptyl

Nyní výsledky napojíme přímo na teorii ze sekce 2.4. Chybu predikce rozložíme na dvě části. První je rozptyl, tedy kolísání predikce podle toho, které konkrétní *support páry* model dostal. Druhou je bias, tedy systematická odchylka, která zůstane i po zprůměrování přes všechny volby kontextu. Měříme je tak, že pro pevný vstupní obrázek opakovaně losujeme *support set*, celkem 24 tahů pro každou velikost kontextu, a sledujeme, jak se predikce mění. Pro obě části máme z Naglerovy teorie předpověď, kterou jsme v jedné dimenzi potvrdili, a nyní ji ověříme znovu ve dvou.

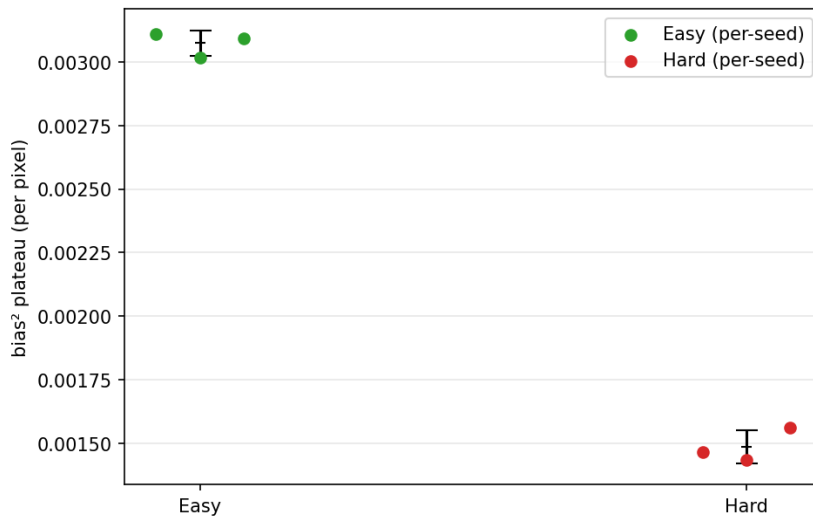


Obrázek 4.10: Rozklad chyby na bias a rozptyl jako funkce velikosti kontextu pro snadnou a náročnou úlohu (logaritmické osy). Vlevo plné čáry jsou bias, který zůstává na plató, přerušované rozptyl, který klesá. Vpravo rozptyl proti teoretickému řádu n^{-1} plynoucímu z věty 2.4.6, empirický pokles tomuto řádu zhruba odpovídá.

Diskuze:

Obrázek 4.10 ukazuje obě části zvlášť. Rozptyl s rostoucím kontextem prudce klesá. Připomeňme, že věta 2.4.6 omezuje odchylku predikce od jejího průměru řádem $n^{-1/2}$. Rozptyl je střední hodnota kvadrátu této odchylky, a kvadrát $n^{-1/2}$ je n^{-1} . Rozptyl by tedy měl mizet alespoň jako n^{-1} . Pokles odečtený z grafu tomuto řádu zhruba odpovídá. U některých úloh se zdá být i o něco rychlejší, což horní mez věty neporušuje. Přidávání dalších *support párů* tedy skutečně ustaluje predikci. Bias se naopak drží na malé kladné hodnotě a s kontextem nemizí. To je přesně neredukovatelný bias, který plyne z podmínky lokality (věta 2.4.5). Jak jsme již popisovali v kapitole 3, globální *attention* přiděluje váhu i vzdáleným bodům, takže drobná systematická odchylka nikdy úplně nezmizí.

Z jediného tréninku bychom ale nemohli tvrdit, že je bias vlastností architektury. Mohl by to být jen zbytek konkrétního běhu. Proto jsme natrénovali tři modely s různou inicializací a různými trénovacími daty a bias změřili na stejné sadě úloh. Obrázek 4.11 ukazuje, že se hodnota biasu napříč běhy shoduje na jednotky procent. Tímto usuzujeme, že neredukovatelný bias je strukturální vlastnost architektury, a ne náhoda jednoho modelu. Máme konkrétní praktické potvrzení jevu, který Naglerova teorie přesně předpovídá.



Obrázek 4.11: Hodnota biasu na plató pro tři nezávislé tréninky (různá inicializace i data) na stejné sadě úloh. Body se shodují na jednotky procent, chybová úsečka je směrodatná odchylka přes běhy. Shoda ukazuje, že bias je strukturální vlastnost architektury.

4.3.5 Vzdálenost od Bayesova optima

Dosud jsme PFN porovnávali s *oracle*m přímo. Nyní se budeme dívat z jiné perspektivy, a to jak daleko je PFN od nejlepšího možného prediktora. I *oracle* totiž dělá nenulovou chybu. Obrázek je zašuměný, takže u pixelů na hranici nejde s jistotou určit, zda patří do popředí. Existuje tedy nejmenší možná chyba, pod kterou se nedostane žádný prediktor, budeme jí říkat *Bayes floor*. Chybu měříme binární křížovou entropií (BCE) vůči pravé masce, pro PFN i pro *oracle* stejně. Rozdíl mezi chybou PFN a touto mezí je cena, kterou platíme za to, že PFN úlohu jen odhaduje z kontextu.

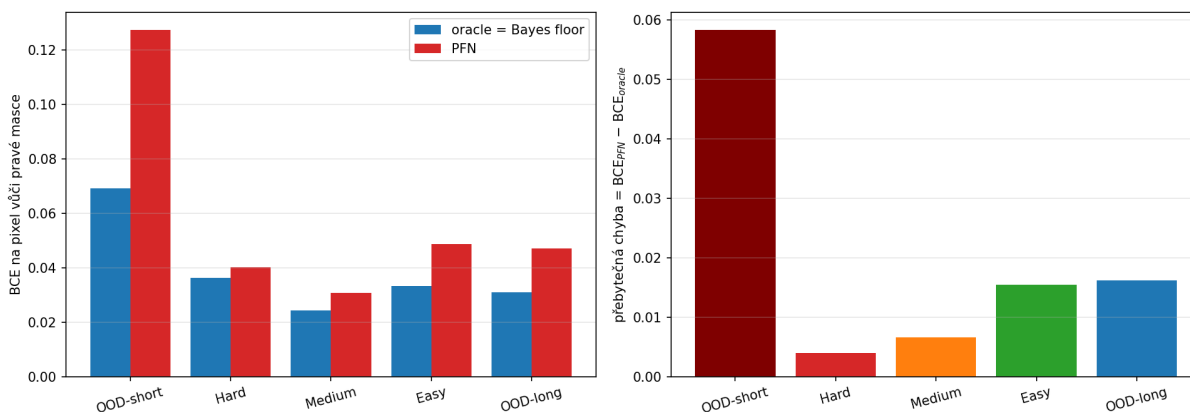
Popišme nejprve, co obrázek 4.12 zobrazuje a jak ho číst. Na vodorovné ose obou grafů je pět režimů náročnosti, které jsme zavedli v části 4.3.1. Zleva stojí OOD-short, tam je korelační délka kratší, než jakou model viděl při tréninku, a pole je proto členitější. Následují tři režimy uvnitř trénovacího rozsahu, Hard, Medium a Easy, seřazené od nejkratší korelační délky po nejdelší. Vpravo stojí OOD-long, tam je korelační délka naopak delší a pole hladší. Levý graf ukazuje pro každý režim dva sloupce. Modrý sloupec je chyba samotného *oracul*, tedy *Bayes floor*, nejmenší chyba, jaké lze v daném režimu vůbec dosáhnout. Červený sloupec je chyba PFN. Čím menší je rozdíl mezi nimi, tím blíže je PFN nejlepšímu možnému prediktoru. Pravý graf vynáší tento rozdíl, tedy přebytečnou chybu (angl. *excess risk*), přímo.

Diskuze:

Nyní k výsledkům. Uvnitř trénovacího rozsahu (sloupce Hard, Medium a Easy) je přebytečná chyba malá. U režimů Hard a Medium sloupce PFN a *Bayes floor* téměř splývají, u režimu Easy je rozdíl viditelný, ale je stále malý. Model se tedy nenaučil jen dobrý prediktor, ale téměř nejlepší možný. Poznamenejme, že tento výsledek zpětně vysvětluje i jedno pozorování z tréninku, které jsme dosud nekomentovali. Trénovací chyba se totiž ke konci tréninku ustálila na malé, ale nenulové hodnotě. Nebylo to tím, že by model uvázl, ale tím, že dosedl těsně nad *Bayes floor*, pod který se žádný prediktor nedostane.

Za povšimnutí stojí, že uvnitř rozsahu je přebytečná chyba největší právě u hladkých polí (Easy), jeho sloupec je mezi režimy uvnitř rozsahu nejvyšší. Graf pouze ukazuje tento fakt, příčinu z něj nevyčteme. Vysvětlení ale nabízí spojení dvou dřívějších pozorování. Z části 4.3.3 víme, že PFN je na nejistých pixelech na hranicích o něco ostřejší než *oracle*. A hladká pole mají nejisté hranice nejširší, takže pixelů, na kterých se tato vada projevuje, je v nich nejvíc. Součet mnoha malých chyb přes širokou hranici pak

dává největší přebytnou chybu. Mimo trénovací rozsah (krajní sloupce OOD-short a OOD-long) se oba směry chovají různě. Hladší pole (OOD-long) zůstávají srovnatelná s režimem Easy, kdežto u členitějších polí (OOD-short) přebytná chyba vzroste na několiknásobek, tedy přesně tam, kde amortizace selhává.



Obrázek 4.12: Vlevo chyba PFN a *Bayes floor* (chyba samotného *oraculu*) pro jednotlivé režimy, měřená binární křížovou entropií (BCE) na pixel. Vpravo jejich rozdíl $BCE_{PFN} - BCE_{oracle}$, tedy přebytná chyba PFN nad optimem. Uvnitř trénovacího rozsahu je malá, prudce roste jen u členitějších polí mimo rozsah (OOD-short).

4.3.6 Kolaps k prioru nenastává

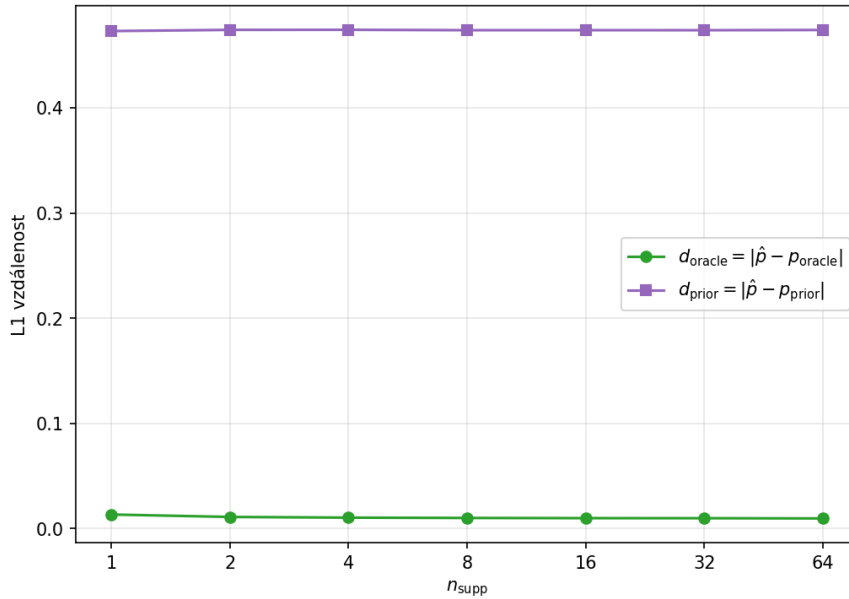
V sekci 4.2 jsme viděli kolaps k prioru, tedy že se predikce při malém kontextu vrací k průměrné masce. Zde stejné měření provedeme na vlastním PFN, a to s pravým priorem místo operační prior masky. Prior masku totiž tentokrát známe přesně. Připomeňme, co je prior maskou. Je to první predikce, kdy vstupní obrázek ještě nevidíme, ale známe pravidla této úlohy. Pro každý pixel se tedy ptáme, s jakou pravděpodobností patří do popředí. Tímto modelu říkáme, kolik zhruba oblasti popředí je na obrázku. V naší úloze je odpověď pro všechny pixely stejná. Skryté pole totiž vzniká v každém místě obrázku podle stejných pravidel (vzorkování z GP). Útvary tvořené GP nemají žádný typickou polohu, na rozdíl od např. krvinek. Tedy jediná informace, kterou můžeme modelu sdělit, je celkový podíl popředí. Zabírá-li popředí například třetinu obrázku, patří do něj každý pixel s pravděpodobností jedna třetina. Prior maska je proto mapa se stejnou hodnotou ve všech pixelech, tedy jednobarevná plocha bez jakéhokoli tvaru. Měření provedeme na jednom zástupci uvnitř trénovacího rozsahu, konkrétně na náročné úloze (Hard). Sledujeme vzdálenost predikce od *oraculu* a od této prior masky jako funkci velikosti kontextu.

Opět zkusme popsat, co tento na první pohled zvláštní graf zobrazuje na obrázku 4.13. Ve skutečnosti používáme zde stejný princip jako v 4.3. Zelená křivka je vzdálenost predikce od *oraculu*, fialová křivka je vzdálenost od uniformní prior masky (naš prior v tomto případě), obě jako funkce velikosti kontextu.

Diskuze:

Podle popisu grafu 4.13 výše můžeme udělat následující závěr. Kdyby kolaps k prioru nastával i zde, viděli bychom fialovou křivku klesat k nule při malém kontextu a zelenou naopak stoupat. Predikce by se totiž podobala prior masce, a ne skutečnosti. Avšak nic takového se zde neděje. Zelená křivka zůstává nízko (blízko nuly) a nemění se s velikostí kontextu, dokonce se drží u nuly už i při jediném *support páru*. Predikce je tedy po celou dobu mnohem blíže *oraculu* než prior masce. Kolaps zde tedy nenastává. Důvod je stejný jako v části 4.3.2, hustý vstupní obrázek prozradí úlohu hned u jednoho příkladu, takže model nemá důvod, proč se vracet k průměru.

Zdůrazněme, že tento negativní výsledek Naglerově teorii neodporuje. Vyžaduje ale přesnější čtení toho, co teorie předpovídá. Jelikož je PFN trénováno aproximovat PPD, přiblížení k prioru je třeba čekat tam, kde je samotný posterior blízko prioru, tedy tam, kde je podmiňující informace o úloze slabá. V naší úloze však nese vstupní obrázek zašuměné pozorování skrytého pole v každém pixelu, tedy přímo tu veličinu, kterou prahujeme. Většinu informace potřebné k predikci tak model získá už ze samotného vstupního obrázku, z jeho pixelů si totiž dokáže skryté pole zrekonstruovat, a tím zná i tvar hledané masky. Ze *support setu* se dozvídá jen to, co v obrázku obsaženo není, tedy polohu prahu a sílu šumu (viz část 4.3.2). Posterior je tak daleko od prioru i při jediném *support páru* a žádný kolaps proto teorie nepředpovídá, což naše měření potvrzuje. Právě v tom se tato úloha liší od experimentu se zmrazeným modelem v sekci 4.2. Tam je vstupní obrázek sice rovněž hustý, ale sám o sobě neurčuje, co se má segmentovat. Stejnou buňku lze segmentovat po jádru, cytoplazmě i celé buňce a cílovou strukturu definuje až *support set*. Je-li *support set* řídký, je řídký i jediný nosič informace o úloze. Posterior přes možné úlohy zůstává široký a nejbezpečnější volbou je průměrná maska. Kdyby naše PFN přesto kolabovalo, nebylo by to potvrzení teorie, ale selhání aproximace posterioru. Stopa příklonu k prioru nicméně zůstává i zde. Je jí malé bias plató z části 4.3.4, které s kontextem nemizí a jehož neodstranitelnost vysvětluje právě podmínka lokality (věta 2.4.5). Podobný mechanismus, který u řídkého a málo informativního kontextu způsobí kolaps, se u informativního vstupu projevuje jen touto drobnou systematickou odchylkou. Kolaps k prioru tedy není univerzální vlastnost amortizace. Nastává tam, kde je jediným zdrojem informace o úloze řídký kontext.



Obrázek 4.13: Vzdálenost predikce PFN od *oraculu* (zelená, p_{oracle}) a od uniformní prior masky (fialová, p_{prior} , tedy konstantní podíl popředí) jako funkce velikosti kontextu pro náročnou úlohu. Predikce je po celou dobu blízko *oraculu* a daleko od prioru, ke kolapsu nedochází ani při jediném *support páru*.

4.3.7 Shrnutí

Tento experiment ukázal, že jádro myšlenky PFN funguje i ve dvou dimenzích. Skutečné PFN natréované na explicitním prioru dostatečně dobře aproximuje pravý Bayesovský posterior, a to ve střední hodnotě i v nejistotě. Uvnitř trénovacího rozsahu je dokonce téměř Bayesovsky optimální. Od nejlepššího

možného prediktoru jej dělí jen malá chyba. Rozklad chyby přitom odpovídá Naglerově teorii, rozptyl s kontextem mizí alespoň jako n^{-1} a bias zůstává na malé, ale strukturální hodnotě, a to bylo potvrzené shodou přes tři nezávislé tréninky.

Současně se ukázalo, kde má metoda meze. Amortizační chyba neroste s ubývajícím kontextem, ale s náročností úlohy a s odchýlením hyperparametrů mimo trénovací rozsah. Hustý vstupní obrázek totiž úlohu prozradí i s malým počtem příkladů. Oba experimenty této kapitoly tak vedou ke stejnému závěru. Kolaps k prioru u předtrénovaného modelu ani selhání mimo trénovací rozsah nejsou vadou architektury, v čistém režimu je totiž model prokazatelně téměř optimální. Jsou to patologie podmíněné režimem, které nastupují až při řídkém kontextu nebo mimo naučenou rodinu úloh.

Závěr

Jak již jsme řekli v úvodu, PFN je dalším pokročilým nástrojem z rodiny architektur strojového učení, které stavějí na Bayesovském principu a statistice [2]. Cílem této práce bylo tuto architekturu představit a porozumět tomu, jakým způsobem Bayesovskou inferenci skutečně aproximuje. V první kapitole jsme proto shrnuli klasický aparát, od Gaussovských procesů a Bayesovského výběru modelu po variální inferenci, Type II MLE a metody MCMC. Na příkladu Gaussovských procesů jsme viděli, že plně Bayesovský přístup vede na integrály, které analyticky spočítat nelze, a právě to je důvod, proč aproximační metody vůbec potřebujeme. Každá z těchto metod navíc něco vyžaduje, ať už hustotu prioru, spočitatelný likelihood, nebo nákladnou optimalizaci.

Ve druhé kapitole jsme postupně zavedli PFN jako koncept a nástroj. Vysvětlili jsme jeho trénink na datech vzorkovaných z prioru a porovnali ho s těmito klasickými metodami. Ukázali jsme také, proč minimalizace záporné log-věrohodnosti vede právě k posteriornímu prediktivnímu rozdělení a jak model celé toto rozdělení vrací v jediném dopředném průchodu. Uvedli jsme rovněž Naglerovu statistickou teorii [3], která pro tuto architekturu předpovídá mizení rozptylu s velikostí kontextu a neredukovatelný bias plynoucí z porušení podmínky lokality.

Následně jsme provedli řadu experimentů. Nejprve jsme vyvrátili triviální hypotézu, že PFN provádí prostý *kernel smoothing*. *Attention váhy* s RBF kernelem nekorelují a predikce PFN je o řády blíže přesnému GP posterioru než Nadaraya-Watsonův estimátor. Pohledem dovnitř transformeru jsme objevili přerozdělování práce mezi vrstvami. Rané vrstvy čtou *labels*, střední vrstvy porovnávají vstupní vzdálenosti a kauzálně nejdůležitější hlavy leží právě ve vrstvách raných. Ukázali jsme, že hloubka funguje jako iterace řešiče jen přibližně a spíše koncepčně. PFN konverguje výrazně rychleji, než by odpovídalo jednomu kroku Neumannovy řady na vrstvu. Model trénovaný na celé rodině hyperparametrů navíc spolehlivě identifikuje korelační délku přímo z kontextu, a to za cenu horší absolutní kalibrace rozptylu. Při misspecifikaci prioru se PFN ukázalo jako konkurenceschopné adaptivním metodám a v kalibraci pro malé kontexty jako nejlepší z misspecifikovaných metod. Degradace mimo trénovací rozsah je přitom plynulá, bez náhlého kolapsu.

V poslední kapitole jsme stejné otázky přenesli do dvou dimenzí, a to konkrétně na segmentaci obrazu. U předtrénovaného *in-context* segmenteru jsme pozorovali kolaps k prioru při řídkém kontextu, tedy jev, který souvisí s nelokalitou *attention* jako neredukovatelný bias. Vlastní PFN natrénované na explicitním prioru pak bylo uvnitř trénovacího rozdělení skoro optimální a dostatečně dobře approximovalo přesný posterior včetně nejistoty. Rozptyl predikce mizel s kontextem v souladu s teorií, zatímco bias zůstával na malém plató, jehož shoda přes nezávislé tréninky ukazuje, že jde o strukturální vlastnost architektury. Naglerovy předpovědi se tak potvrdily i mimo jednorozměrnou regresi.

Celkově tedy platí, že PFN provádí věrnou amortizovanou Bayesovskou inferenci v jediném dopředném průchodu. Jeho slabiny, jako jsou kolaps při neinformativním kontextu a selhání mimo naučenou rodinu úloh, nejsou vadou architektury, ale vlastností režimu, ve kterém model pracuje. Přírozeným pokračováním práce je hlubší teoretické pochopení toho, jaký výpočet vrstvy transformeru skutečně provádějí, a rozšíření experimentů na větší modely a další třídy úloh.

Literatura

- [1] C. M. Bishop: *Pattern Recognition and Machine Learning*. Springer, New York, 2006.
- [2] S. Müller, N. Hollmann, S. Pineda Arango, J. Grabocka, F. Hutter: *Transformers Can Do Bayesian Inference*. In: International Conference on Learning Representations (ICLR), 2022.
- [3] T. Nagler: *Statistical Foundations of Prior-Data Fitted Networks*. In: Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR 202, 2023.
- [4] C. E. Rasmussen, C. K. I. Williams: *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA, 2006.
- [5] S. J. D. Prince: *Understanding Deep Learning*. MIT Press, Cambridge, MA, 2023.
- [6] N. Hollmann, S. Müller, K. Eggenberger, F. Hutter: *TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second*. In: International Conference on Learning Representations (ICLR), 2023.
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin: *Attention Is All You Need*. In: Advances in Neural Information Processing Systems 30 (NIPS), 2017.
- [8] G. Xiao, Y. Tian, B. Chen, S. Han, M. Lewis: *Efficient Streaming Language Models with Attention Sinks*. In: International Conference on Learning Representations (ICLR), 2024.
- [9] R. M. Neal: *MCMC Using Hamiltonian Dynamics*. In: S. Brooks, A. Gelman, G. L. Jones, X.-L. Meng (eds.), *Handbook of Markov Chain Monte Carlo*, Chapman & Hall/CRC, 2011.
- [10] S. Brooks, A. Gelman, G. L. Jones, X.-L. Meng (eds.): *Handbook of Markov Chain Monte Carlo*. Chapman & Hall/CRC, Boca Raton, 2011.
- [11] M. D. Hoffman, A. Gelman: *The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo*. Journal of Machine Learning Research, 15(47):1593–1623, 2014.
- [12] J. von Oswald, E. Niklasson, E. Randazzo, J. Sacramento, A. Mordvintsev, A. Zhmoginov, M. Vladymyrov: *Transformers Learn In-Context by Gradient Descent*. In: Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR 202, 2023.
- [13] K. Ahn, X. Cheng, H. Daneshmand, S. Sra: *Transformers Learn to Implement Preconditioned Gradient Descent for In-Context Learning*. In: Advances in Neural Information Processing Systems 36 (NeurIPS), 2023.
- [14] V. I. Butoi, J. J. Gonzalez Ortiz, T. Ma, M. R. Sabuncu, J. Guttag, A. V. Dalca: *UniverSeg: Universal Medical Image Segmentation*. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023.

- [15] X. Zheng, Y. Wang, G. Wang, J. Liu: *Fast and Robust Segmentation of White Blood Cell Images by Self-Supervised Learning*. Micron, 107:55–71, 2018.
- [16] D. S. Marcus, T. H. Wang, J. Parker, J. G. Csernansky, J. C. Morris, R. L. Buckner: *Open Access Series of Imaging Studies (OASIS): Cross-Sectional MRI Data in Young, Middle Aged, Nondemented, and Demented Older Adults*. Journal of Cognitive Neuroscience, 19(9):1498–1507, 2007.